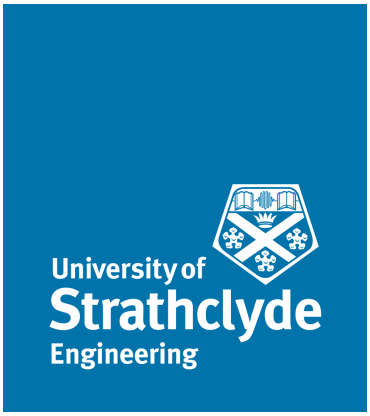
EE106 Engineering design for software development 1

Robert C Atkinson

October 10, 2025



Chapter 1

Introduction

This course will examine computer programming foundations using Python as a language. The initial focus will be on proper understanding of three key components of basic programming:

- types and operators;
- decisions;
- loops.

Functions will be introduced as a means of structuring code and permitting pre-written code to be more readily reused. User defined data types will be explored to permit more powerful expression of concepts, and object-oriented design will be covered to permit intelligent reuse of user defined data types.

Container objects (ready made data structures) will be introduced as a means of organising information in computer programs.

1.1 Logistics

This class has a strong practical element which reflects the necessity to practice programming to become effective at it. Therefore, three hours per week will be allocated to laboratory sessions. They will take place in the Royal College Room RC 446 & RC 448 Fridays 9:00 — 12:00. The lab sessions will commence on Week 1 of the semester. Lectures will take place on Tuesdays in the Thomas Graham Building Room TG 223 12:00 — 13:00. Please be prompt! Each week we will cover a new topic (chapter of the course notes), read the appropriate chapter and work your way through the worked examples of code. Next, attempt the problem sets. These should be thoroughly tested to ensure they work properly. Each problem must be demonstrated during the following Friday lab slot between 9:00 and 11:00.

Please attempt all problems in a set prior to attending the Friday session. During each Friday lab session you will be permitted to demonstrate that week's problem sets, and also the problem sets for the week

before — please do not email me any code to assess! From 11:00 until 12:00 we will run a coding clinic where you can stay behind and ask questions on deeper aspects of computer programming; if there is some aspect of coding you are not fully grasping, this is the perfect opportunity to seek help. Finally, a catch-up session will be organised for Week 10 whereby any outstanding problem sets can be assessed; this provide three distinct opportunities to have your code assessed. No code will be assessed after the catch-up session in Week 10, furthermore, no code will be assessed during the tutorial sessions.

When you would like to demonstrate your problem sets, put your hand up to summon a Teaching Assistant. **Please state clearly which problems you wish to demonstrate**. The Teaching Assistants will ask you a number of questions about the code you have written to ensure you understand best practice, and to ensure you are the author of the code that you are demonstrating. Unfortunately, in the past some students have sought to demonstrate code taken from the Internet or provided by class mates; this is not permitted. Any student caught supplying or receiving code may be subject to disciplinary action from the University. All code must be demonstrated to a tutor or teaching assistant; to reiterate, please do not email any code to be assessed offline — I refuse to open any attachments containing executable code.

It is YOUR responsibility to ensure that your code has been demonstrated. The teaching assistants will keep a record of all exercises completed; for your own records you may use Table 1.1.

Exercise	1	2	3	4
Types and operators				•
Branches				•
Loops				
Input/output			•	•
Strings				
Data structures				
Functions				•
Abstract data types				
OOP				•

Table 1.1: Progress checker

It is imperative that you can fully explain all code that is being demostrated, including (i) what each line does, (ii) why you chose a particular approach, and (iii) how you tested your code. You must be the author of all code demonstrated, it is recommended that familiarise yourself with the University's policy on academic dishonesty. In recent years, we have seen more students turning up to labs or demonstrations with code that they clearly didn't write themselves, and, more importantly, didn't understand. Sometimes this code might have been copied from a friend, a website, or more recently, from Generative AI. The exact source doesn't actually matter. What matters is that when you cannot explain what your own code is doing, or why you made particular design choices, it is immediately clear that the learning outcomes have not been met.

This is not just an issue of academic integrity, it undermines your own development. The purpose of this class is not to get a program that 'just works', but to build your own understanding of how and why it works. Employers, future module registrars (e.g. EE273) will expect you to be able to reason through your code, debug errors, and make informed design decisions. If you bypass that process by submitting work you don't understand, you are robbing yourself of the chance to develop those essential skills.

For this reason, code that you cannot explain or justify cannot be accepted in exchange for marks. In other

words, if during a demonstration you are unable to talk through your logic and choices, I cannot award you credit. Passing this class requires evidence that you personally have achieved the learning objectives. Tools like Generative AI may seem like a shortcut, but they are no substitute for genuine understanding and relying on them risks failure, both in assessment and in your long-term growth as an engineer.

If you are finding difficulty in completing a particular exercise, by all means seek assistance; however, it is not the role of Teaching Assistants to write your code for you — this would undermine the opportunity for you to learn. The Teaching Assistants have been specifically asked NOT to provide you with a solution to a problem, rather they will guide you towards an appropriate solution of your own. It is important that you embrace this opportunity to develop your initiative, self-reliance, and problem solving skills. With this in mind, some tips ....

1. If the Python interpretor displays an error, it is probable that you have misspelled a variable/function/class name incorrectly; check the spelling matches both at the place of definition and place of execution error.
2. Another common source of error is incorrect indentation, which defines blocks of code. We will learn more about this later.
3. The interpretor will attempt to identify the location (line number) of the error, but it is not infallable. If you can see no error where it is indicating, check a line or two above.
4. If you don't understand a particular error, copy and paste it into Google — that's what most programmers do!
5. A great recource for solving computing problems is the website www.stackoverflow.com — check it out.

In order to pass this class you need to achieve an overall class mark above 40%. As part of the Project Phase in the second semester, you will be allocated a team mate to work together on a problem that you will select from a list of options. Assessment of the project will be based on an interim specification, a final report, and a demonstration of code. Please note, each of these elements are considered as integrated course work, which means that you will not receive a score for each element individually. The following submission deadlines apply: the interim report is to be submitted via myplace by Thursday (5PM) of Week 5 of Semester 2, and the final report is to be submitted via myplace by Thursday (5PM) of Week 11. The demonstration are due on week 11 of Semester 2 (9AM sharp).

Your overall score will be based on the following components:

- Semester 1 lab exercises: 20%
- Semester 1 class test: 10%
- Interim Specification: 20%
- Final Report: 25%
- Demonstration: 25%

Should you be required to resit the module, you will be allocated a project on an individual basis to work on over the summer period. The project topic will be different from any topic you have attempted previously. This will be assessed by a Final Report (50%) and Demonstration (50%).

1.1.1 Interim and Final Reports

The interim and final reports are worth 20% and 25% of the overall mark, respectively. Therefore, it is important that you invest effort in writing these reports, do not leave them until the last minute. A specimen template is provided online, it is strongly suggested that you use it.

The interim report should provide an accessible description of the project, that is, a description of the problem you are solving written in non-technical language such that a 14 year old with no prior exposure to computation could understand it. Avoid any overly technical language in this description. Next a more technical description of the problem should be provided, one that a competent computer programmer would comprehend. The descriptions should make clear what you are attempting to achieve and why. It is also important to explain how you will solve the problem, this explanation should describe how you break the overall problem down into smaller problems using functions, and class methods. Any classes that you intend to utilise to manipulate and store data should be described.

It is recommended that you include diagrams to describe the structure of your code where possible, flowcharts are a good way to describe the flow of logic. However, the inclusion of a diagram or picture does not remove the need for a textual description. Another way to help the reader understand the code you intend to write is to describe algorithms using pseudocode. Make reference to any formal or informal software testing strategy you intend to use. How are you going to ensure that your code functions as expected? Include a Gantt chart as part of a time plan that clearly shows the development phases of each of the tasks that you have already undertaken and will undertake in the remainder of the project. It should also include who is doing what. Include references to any material that has informed your software architecture or implementation. Any code written so far should be included in an appendix. Finally, come to an agreement with your partner over your contribution to the submitted report, if both equal then this should be 50% each.

The expectation is that the interim report should be a minimum of 10 pages excluding table of contents and references, etc. The final report should follow a similar structure to the interim report, however a time plan is not required. In this report you should include strong evidence of software testing, all code should be well documented and included in an appendix. Myplace contains two speciment reports — one which received a good grade and one that received a poor grade. Please compare and contrast them.

1.1.2 Project Demonstration

The project demonstration will take place during the final lab session of Semester 2, and is worth 25% of the overall mark — attendance is mandatory. The format of the demonstration is broadly similar to that of the demonstration of exercises in Semester 1: you will be required to run your code and demonstrate an understanding of each line of code and the overall architecture. It is insufficient that the code works — the onus is on you to demonstrate a full understanding of the code in exchange for marks. During the demonstration, you will asked a series of questions to test this understanding. At the end you may leave.

Please be advised that each of the individial elements of the second semester: (i) interim report, (ii) final report, and (iii) demonstration are class as integrated coursework, thus you will not receive a score for each element individually. You will, however, receive qualitative feedback on your interim report.

1.1.3 Recommended Texts and Online Material

There is a variety of information on programming using Python available free on the Internet. A good source of information is Dive into Python 3 by Mark Pilgrim; it is available online free at:

http://getpython3.com/diveintopython3/

Other notable texts include:

- Practical Programming, 3e: An Introduction to Computer Science Using Python 3.6 by Gries, Campbell and Montojo.
- Introduction to Computer Science using Python by Dierbach
- Python Programming An Introduction to Computer Science by Zelle

Therre is a great deal of free material available online, either in written or video format. I would recommend sentdex's video series of a range of Python topics, especially his Introduction to Python 3, available here: "https://youtu.be/eXBD2bB9-RA?si=iYL-4JOFPwSJp7NI".

1.1.4 Some Notes On Conduct

1. Please be punctual!
2. Please be polite to the teaching staff. The Teaching Assistants are there to support your learning NOT to write your code, this would be counter-productive. A common approach to problem-centric teaching is to adopt a Socratic approach to solving problems whereby the student is asked a series of questions to guide them towards understanding. In this respect, the Teaching Assistants are not being unhelpful.
3. Inevitably, a small number of students (for a number of reasons) will simply opt to copy the code of a neighbour or use Generative AI to gain credit for completing a problem set. I must ask that you refrain from any such activity, since this course of action will have a negative impact on your ability to learn to program effectively. As a consequence, such students tend to struggle to pass the project assessment that takes place in semester 2. Furthermore, this act is a form of plagiarism, which constitutes academic dishonesty. The university regulations prohibit this behaviour and penalties will be applied where discovered.
4. During the project phase (semester 2), it is equally important that you are the author of any code you submit. There are occasions where it is permissible to use code from another source as long as it is referenced both in the code itself, but also in the submitted report. However, the expectation is that you should be the author of over 90 percent of any code submitted. Using a library is fine, copying and pasting code from elsewhere is not.
5. Please refrain from emailing code to be assessed or to seek support; you must be present (remotely) to explain your code. Email attachments containing code will not be opened.
6. Please resepect the deadline regarding the demonstration of problem sets, and the submission of interim and final reports.

1.2 Getting Started With Spyder

To begin programming the computer, we must first launch the Python Integrated Development Environment (IDE), Spyder. On the Start Menu locate an application called *anaconda*, and click on it. Hopefully, a window similar to that depicted in Figure 1.1 should appear.

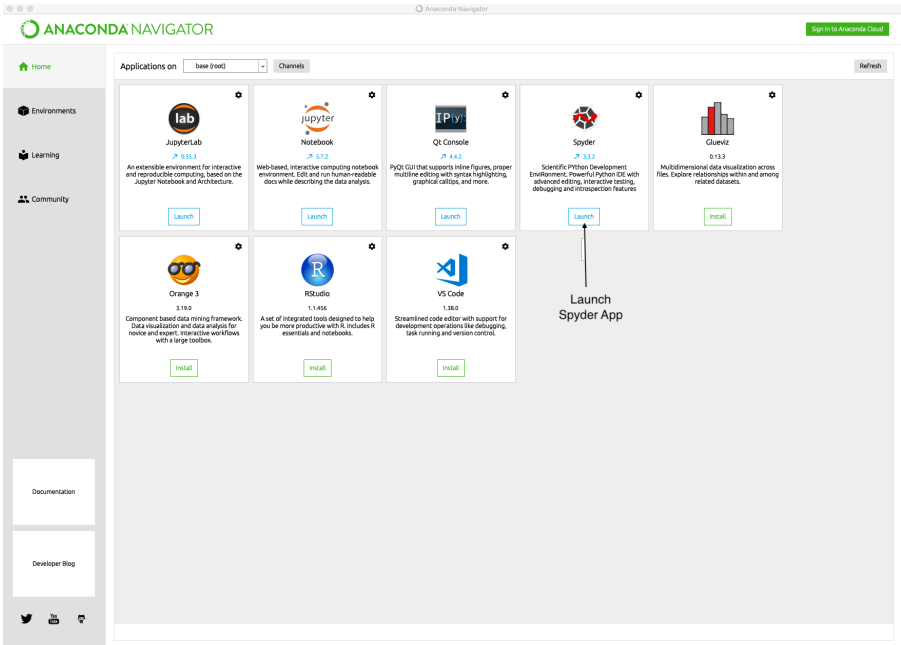


Figure 1.1: The Anaconda IDE

Within this window, locate Spyder and click to launch. At this point another window similar to that depicted in Figure 1.2 should appear. Within the window, three distinct panes should be visible: an editor pane, interactive console, and one to manage files and view the values of variables, etc. The far left pane is used by the programmer to write and modify source code. The code can later be run through the Python interpreter by clicking the green triangle or 'F5'; you should save your code before running it. In the bottom right pane is the interactive console; this is used by the programmer to interact with running code such as entering information from a command prompt or reading information printed to the screen. When you launch Spyder, ensure you are running the correct version of the interpretor 3.x.x, and not 2.7.x. Finally, the upper right pane contains three tabs; one can be used for help and the other can be used to load and view files, or importantly examine program context (view the values of variables).

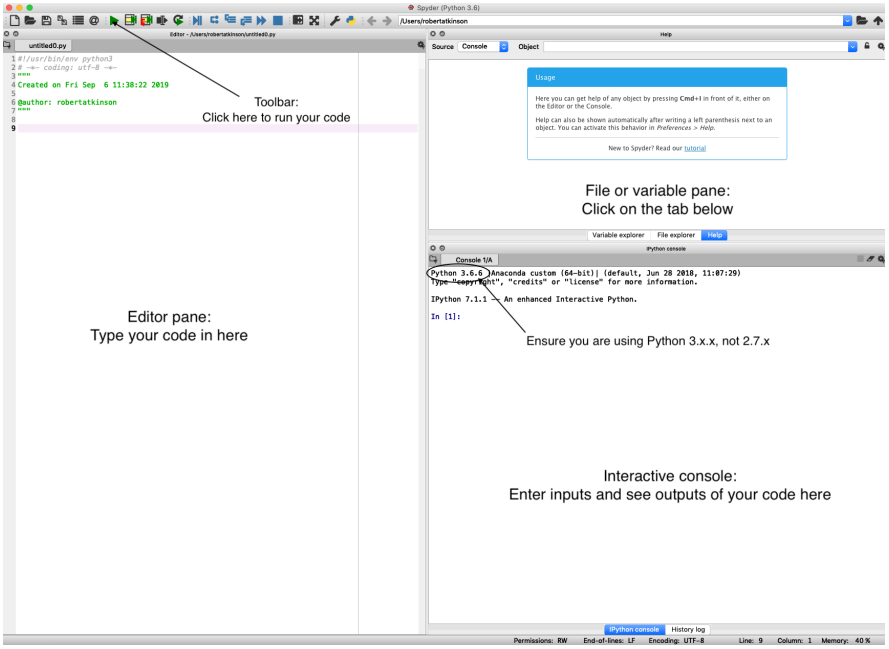


Figure 1.2: The Spyder IDE

Create an appropriate folder to store all EE106 Python code you write; I suggest you organise it by topic. This will be used as the default location to store any files you save during this Spyder session. The file pane can also be used to load code you have written previously by double clicking on the file name. By way of example, selecting and double clicking the integer.py file (you do not have this file) loads the code shown in Figure 1.3.

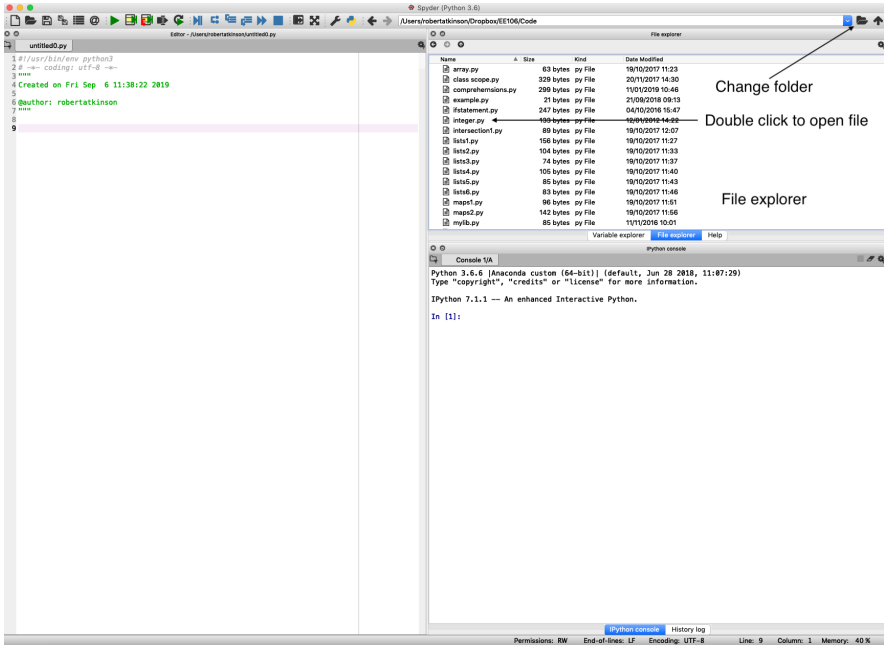


Figure 1.3: Spyder in File Pane

To illustrate the console and variable panes, Figure 1.4 shows the code to a simple Python program (integer.py) When the code is run, we can see the output of the print statements in the console pane and the value (and types) of the variables in the variable explorer pane.

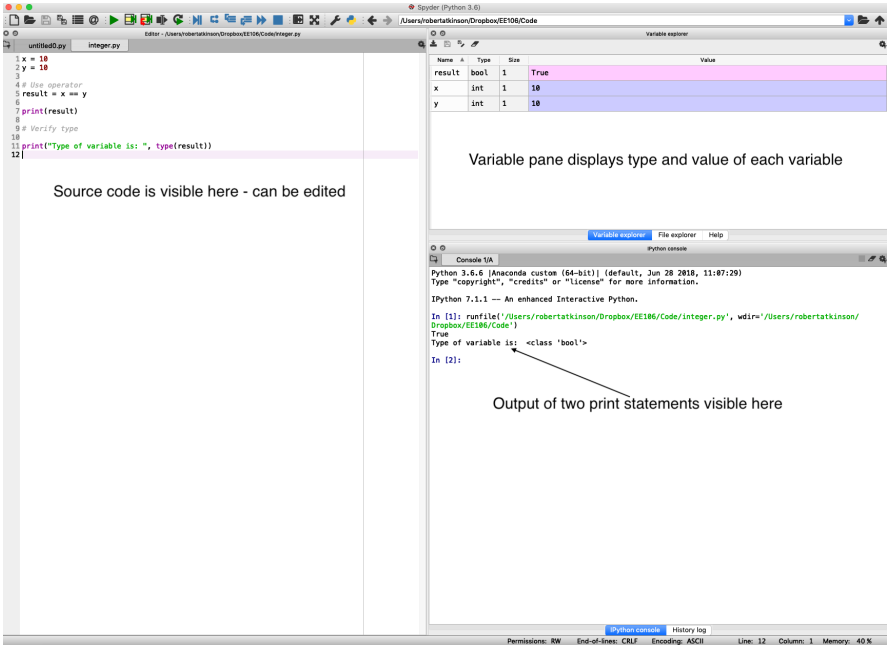


Figure 1.4: Spyder in Operation

If you encounter a problem at this stage please seek the assistance during your Friday lab slot.

1.3 Home Installation Of Anaconda

If you wish to install Python on your home computer to experiment with programming, it is available free of charge online. The software is available at: https://www.anaconda.com

1.4 What's Next

Once you have a simple hello world program running move on to the next section which should be available on myplace.

Chapter 2

Types and Operators

Digital computers operate by storing and manipulating numbers; within computer programs numbers are stored in memory and are accessed (i.e. referred to within the code) by a label or variable name. For example, the excerpt of Python code, in Listing 2.1, defines an integer variable called 'first_var' and stores the value of 10 in it.

```
first_var = 10
```

Listing 2.1: Variable assignment

When this code is executed, the computer's operating system will allocate space for the integer in system memory then store the value of '10' at that memory location. It is worthwhile examining what actually happens when a variable is declared within a computer program. Each memory location has a unique memory address, but the programmer need not know the precise address at which variable values are stored, this is handled by the operating system. Instead, the programmer only requires some form of reference to that memory location, and the variable name is used as this reference. Thus, the example code, in Listing 2.1, stores an integer value at some memory location (say address 1002), and uses the variable name 'first_var' to access the value stored at that location as shown in Figure 2.1. This is a convenient abstraction that frees the programmer to concentrate on solving the problem of interest.

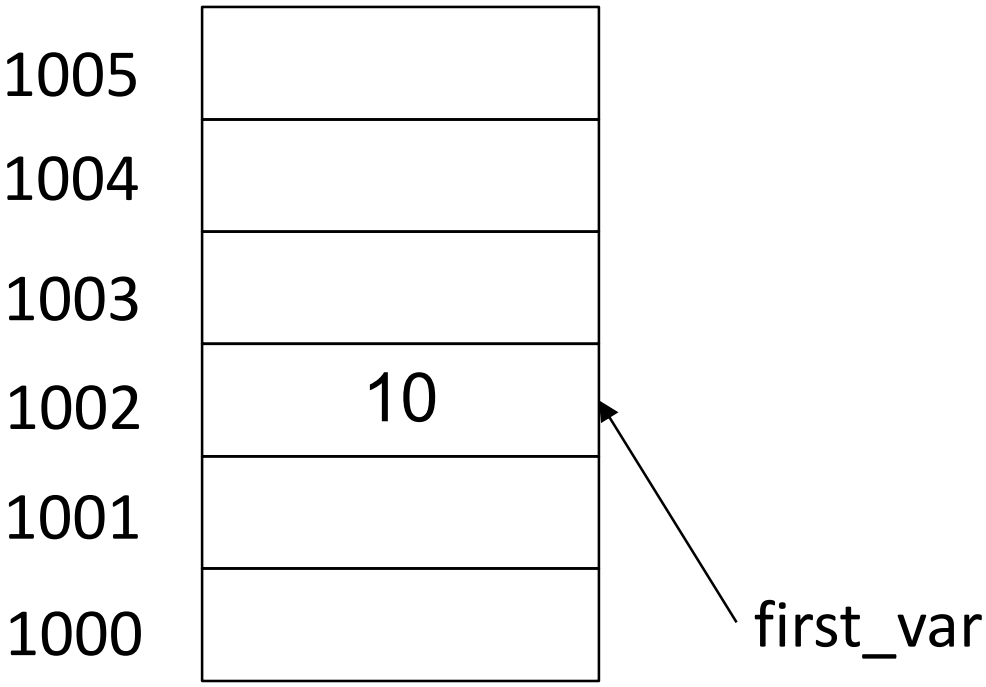


Figure 2.1: Integer stored in memory

It is important to use meaningful variable names, for example if a variable stores someone's age, then call that variable 'age' rather than, say 'a'. More readable code is good code. It is worthwhile pointing out that Python has many built in keywords (see Table 2.1) that we use for programming as well as some variables, constants and functions (see Table 2.2) that should NOT be used as a variable name.

Just as mathematicians have classified numbers into different sets: real numbers, integers, natural numbers, etc., computer programmers use different **types** of variable within their programs. It is important to choose the correct type of variable because the operators that are used to manipulate them within computer programs behave in a different manner dependent on the type of variable: **operators are type-specific!**

To highlight these subtle differences in behaviour, we shall consider the result of applying the '+', '-', '/', and '*' operators on a range of different variable types in Sections 2.1 and 2.2.

False	class	global	pass
None	continue	if	raise
True	def	import	return
and	del	in	try
as	elif	is	type
assert	else	lambda	while
async	except	match	with
await	finally	nonlocal	yield
break	for	not	
case	from	or	

Table 2.1: Python 3.12 keywords

ArithmeticError	IsADirectoryError	UnicodeEncodeError	complex	list
AssertionError	KeyError	UnicodeError	copyright	locals
AttributeError	KeyboardInterrupt	UnicodeTranslateError	credits	map
BaseException	LookupError	UnicodeWarning	debugcell	max
BaseExceptionGroup	MemoryError	UserWarning	debugfile	memoryview
BlockingIOError	ModuleNotFoundError	ValueError	delattr	min
BrokenPipeError	NameError	Warning	dict	next
BufferError	None	ZeroDivisionError	dir	object
BytesWarning	NotADirectoryError	__IPYTHON__	display	oct
ChildProcessError	NotImplemented	__build_class__	divmod	open
ConnectionAbortedError	NotImplementedError	__debug__	enumerate	ord
ConnectionError	OSError	__doc__	eval	pow
ConnectionRefusedError	OverflowError	__import__	exec	print
ConnectionResetError	PendingDeprecationWarning	__loader__	execfile	property
DeprecationWarning	PermissionError	__name__	filter	range
EOFError	ProcessLookupError	__package__	float	repr
Ellipsis	RecursionError	__spec__	format	reversed
EncodingWarning	ReferenceError	abs	frozenset	round
EnvironmentError	ResourceWarning	aiter	get_ipython	runcell
Exception	RuntimeError	all	getattr	runfile
ExceptionGroup	RuntimeWarning	anext	globals	set
False	StopAsyncIteration	any	hasattr	setattr
FileExistsError	StopIteration	ascii	hash	slice
FileNotFoundError	SyntaxError	bin	help	sorted
FloatingPointError	SyntaxWarning	bool	hex	staticmethod
FutureWarning	SystemError	breakpoint	id	str
GeneratorExit	SystemExit	bytearray	input	sum
IOError	TabError	bytes	int	super
ImportError	TimeoutError	callable	isinstance	tuple
ImportWarning	True	cell_count	issubclass	type
IndentationError	TypeError	chr	iter	vars
IndexError	UnboundLocalError	classmethod	len	zip
InterruptedError	UnicodeDecodeError	compile	license	

Table 2.2: Python built-in variables, constants and functions

2.1 Numerical Data Types

The use of the '+' operator on two (or more) integers adds them and produces an integer data type as a result. Check this by running the code fragment in Listing 2.2. The code should indicate that an integer type (referred to simply as an *int*) has been created. In computer programming parlance the data type produced by an operation (or function call) is referred to as the **return type**.

```
int_a = 10
int_b = 20

int_c = int_a + int_b

print(''Result = '', int_c)

# Verify type

print(''Type of variable is: '', type(int_c))
```

Listing 2.2: Integer type

Listing 2.2 uses two built-in functions specific to Python: *print()* and *type()*. The former is used to output information to the screen and the latter is used to determine the data type of a variable.

The use of the '+' operator on two (or more) real numbers (known as a *float* in computer programming) adds them and produces a float as a result. Check this by running the code fragment in Listing 2.3.

```
real_a = 10.1
real_b= 20.1

real_c = real_a + real_b

print(''Result = '', real_c)

# Verify type

print(''Type of variable is: '', type(real_c))
```

Listing 2.3: Float type

It should be apparent that a float data type is produced. The value returned indicates that precision errors often creep into computer programs and these have to be monitored, thus testing is integral to good programming practice.

At this point it is reasonable to ask: how does the computer know that a variable should hold a real number or an integer? In Python, data types are *implicitly* defined. That is, there is a difference between a line of

code that reads *x = 10*, and one that reads *x = 10.0*, although they will both hold the same numerical value, the former will be treated as an integer and the latter as a float. Most other general purpose computing languages require types to be *explicitly* defined. The code fragment below is for the C++ language and produces the same behaviour as Listing 2.3.

```
float real_a = 10.1;
float real_b = 20.1;

float real_c = real_a + real_b;

cout << ``Result = '' << real_c;
```

Unsurprisingly, the subtraction of two integers results in an integer return type, and subtracting floats produces a float data type. The same rule applies to multiplication, but what about division? Clearly, division of one real number by another is expected to yield a real number as a result, but the same cannot be guaranteed for integers. For example, the division of 13 by 5 does not result in an integer value. Type in the code in Listing 2.4 to determine the type returned when applying the '/' operator to divide two integers.

```
int_a = 13
int_b = 5

result = int_a / int_b

print(``Result = '', result)

# Verify type

print(``Type of variable is: '', type(result))
```

Listing 2.4: Integer division

Listing 2.4 should demonstrate that a real number (e.g. a float) is produced by default when dividing two integers. However, there are circumstances when it is desirable to specifically use integer arithmetic and Python (and indeed many other programming languages) provides additional operators to address these situations. These are the quotient operator ('//') and the remainder operator ('%'). Substitute these operators in place of the '/' operator in Listing 2.4 and notice the result in both cases. What do these do?

2.2 Strings

Thus far we have considered only numerical types. Computer programming languages have data types that are used to store and manipulate textual information, these are known as strings. Type in the code in Listing 2.5, and notice the result. The programmer uses either single quotes (' ') or double quotes (" ") to tell the computer that the data type is a string.

```
a_str=``hello world!''

print(a_str)

# Verify type

print(``Type of variable is: '', type(a_str))
```

Listing 2.5: A simple string

In order to fully understand string manipulation, it is necessary to understand how strings are represented and stored in memory. A string can the thought of as a sequence of characters, for example “hello world!” can be represented as shown in Figure 2.2.

h	e	l	l	o		w	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11

Figure 2.2: String as a sequence

Each of the individual characters is also a string in Python, note this is not always the case in other programming languages! The individual characters can be accessed using a special index notation, ‘[]’. Most computer programming languages use indices that start at 0, therefore the value of a_str[0] is ‘h’. Confirm this by modifying Listing 2.5 to be the same as that in Listing 2.6. Experiment with different index values and notice the outcome when the index exceeds the length of the string.

```
a_str=``hello world!''

print(a_str[0])

# Verify type

print(``Type of variable is: '', type(a_str[0]))
```

Listing 2.6: Elements of a string

When stored in memory, each of the individual elements of a string is stored in a separate memory location. The code fragment shown in Listing 2.7 creates a string that is stored at the hypothetical memory address of 1000, as shown in Figure 2.3.

```
my_str = 'hello'
```

Listing 2.7: String stored in memory

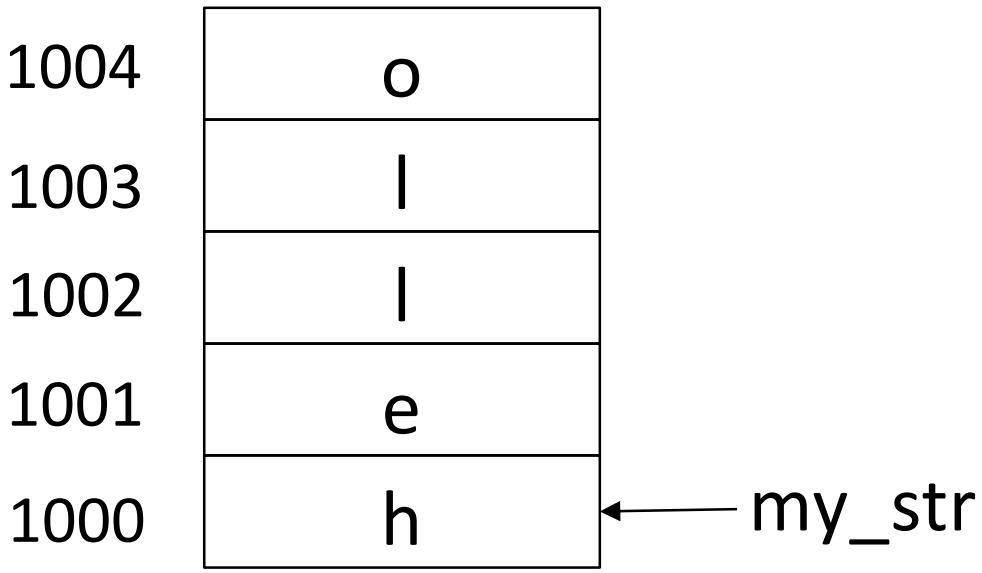


Figure 2.3: String stored in memory

Now that a basic understanding of string structure has been achieved, it is now appropriate to consider how strings can be manipulated using standard operators. Type in the code in Listing 2.8 and notice the result. It should be apparent that the '+' operator in this instance performs *concatenation* and the result (i.e. return type) is another string. What about the other operators ('-', '*', '/'), can they be used on strings; substitute them into the code to discover the result.

```
a_str=''hello''
b_str=''world!''

result = a_str + b_str

print(result)

```

```
# Verify type

print(''Type of variable is: '', type(result))
```

Listing 2.8: Applying the '+' operator to a string

It should be apparent that only the '+' operator has any meaningful application to two or more strings. However, the '*' operator can be used to duplicate a string many times. Type in the code in Listing 2.9 and notice the result. Notice what happens if you change the value of multiplier from 3 to 3.0, i.e. change the data type from an integer to a floating point (real) number.

```
a_str=''hello''
multiplier = 3

result = a_str * multiplier

print(result)

# Verify type

print(''Type of variable is: '', type(result))
```

Listing 2.9: Applying the '*' operator to a string

Appreciation of the type-specific behaviour of operators avoids many common pitfalls. For example enter and run the code in Listing 2.10.

```
# Define a two strings with numerical values
a_str = ''10''
b_str = ''20''

# Use the '+' operator
result = a_str + b_str

print(result)

# Verify type

print(''Type of variable is: '', type(result))
```

Listing 2.10: Incorrect string manipulation

The program will not print out the value of 20 as a novice programmer might intuitively expect. Rather

it prints the string “1020”, i.e. it prints the character[1] “1”, followed by the character “0”, followed by the character “2”, and followed by the character “0”. The reason behind this result is the the computer uses the ‘+’ operator to perform concatenation rather than addition when applied to strings.

2.3 Boolean

Yet another type that is often encountered in programming is the boolean type; it can hold only one of two values: True or False[2]. Type in the code shown in Listing 2.11 to verify that a simple boolean data type has been created.

```
1  my_bool = True
2
3  print(my_bool)
4
5  # Verify type
6
7  print(``Type of variable is: '', type(my_bool))
```

Listing 2.11: Boolean type

Just as the ‘+’, ‘-’, ‘*’, ‘/’ operators can be used on numerical types to perform standard arithmetic operations, boolean operators are defined that operate on boolean data types: *and*, *or*, and *not*. These have precisely the same operation as AND, OR and NOT gates encountered in digital circuit design. Modify Listing 2.11 to produce the code in Listing 2.12 and note the result of the *not* operator.

```
1  my_bool = True
2
3  print(not my_bool)
4
5  # Verify type
6
7  print(``Type of variable is: '', type(my_bool))
```

Listing 2.12: Boolean *not* operator

The code in Listing 2.13 defines two boolean data types then applies the boolean *and* operator (which has a boolean return type). Run the code and change the values of variables ‘x’ and ‘y’ to ensure that the output is consistent with a two input AND gate.

[1] Or more correctly, it prints the string “1”.
[2] Note the capitalisation used in Python for these values.

```
1  # Boolean inputs
2  x = True
3  y = True
4
5  # Use boolean operator
6  result = x and y
7
8  print(result)
9
10 # Verify type
11
12 print(``Type of variable is: '', type(result))
```

Listing 2.13: Boolean *and* operator

Modify the code in Listing 2.13 to match that in Listing 2.14; once again vary the values of ‘x’ and ‘y’ to ensure that the output is consistent with a two input OR gate.

```
1  # Boolean inputs
2  x = True
3  y = True
4
5  # Use boolean operator
6  result = x or y
7
8  print(result)
9
10 # Verify type
11
12 print(``Type of variable is: '', type(result))
```

Listing 2.14: Boolean *or* operator

Boolean data types can be used in conjunction with boolean operators to solve a variety of mathematical problems in their own right. However they are also useful for decision making and controlling loops as will be discussed later. For now it is sufficient to know that decisions and loops can be controlled based on some form of comparison which produces a boolean return type. Therefore, it is often useful to know if the value of one variable is greater-than, less-than, or equal-to the value of another. By way of example run the code as shown in Listing 2.15.

```
1  x = 20
2  y = 10
3
4  # Use operator
```

```
5  result = x > y
6
7  print(result)
8
9  # Verify type
10
11 print(''Type of variable is: '', type(result))
```

Listing 2.15: Using the '>' operator

It should be apparent from Listing 2.15 that even though 'x' and 'y' are integers, the return type of the '>' is always boolean. Indeed the return type will be boolean even if both 'x' and 'y' are floats or strings. Try modifying this code by changing the data types and by using the less-than operator ('<'). To determine if two variables have the same value the equal-to operator is used. Unfortunately, the assignment operator already uses the '=' symbol therefore another symbol is used: '==''. All programming languages use subtly different operators to distinguish between assignment and equality testing. Modify Listing 2.15 by using the equal-to operator and vary the values of the variables.

2.4 Type Conversion

Occasionally, it is useful to be able to convert from one data type to another; this process is known as **casting**. Casting is often required when using the code written by others, or when using some built-in functions. The code in Listing 2.16 converts an integer data type to a float by using the *float()* function on line 5. Hopefully, the result of running this code is fairly intuitive since real numbers can be viewed as a superset of integers, consequently all integers have a real number equivalent.

```
1  # Define an integer type
2  my_int = 20
3
4  # cast to a float
5  my_float = float(my_int)
6
7  print(my_float)
8
9  # Verify type
10
11 print(''Type of variable is: '', type(my_float))
```

Listing 2.16: Casting to a float type

The code in Listing 2.17 performs the reverse operation, it converts a float data type to an integer type. This begs the question: what if the float does not contain a whole number, e.g. 20.0, how can it be converted to an integer value? This is a reasonable question because real numbers do not necessarily have an integer equivalent. To answer this question, type in the code and view the result.

```
# Define a float type
my_float = 20.9

# cast to an integer
my_int = int(my_float)

print(my_int)

# Verify type

print(``Type of variable is: '', type(my_int))
```

Listing 2.17: Casting to an integer type

The output should demonstrate that only the integer component of the original number is retained. One of the most frequently used forms of casting is to convert from a string to either a float or an integer. Of course, this can only be achieved if the programmer has prior knowledge that the string only contains numerical characters. Run the code in Listing 2.18 to verify the result.

```
# Define a two strings with numerical values
a_str = ``10''
b_str = ``20''

# Cast strings to integers
a_int = int(a_str)
b_int = int(b_str)

# Use the `+' operator
result = a_int + b_int

print(result)

# Verify type

print(``Type of variable is: '', type(result))
```

Listing 2.18: Casting a string to an integer

Listing 2.19 depicts equivalent code for converting from a string to a floating point number.

```
# Define a two strings with numerical values
a_str = ``10.1''
b_str = ``20.3''

```

```
5  # Cast strings to floats
6  a_float = float(a_str)
7  b_float = float(b_str)
8
9  # Use the '+' operator
10 result = a_float + b_float
11
12 print(result)
13
14 # Verify type
15
16 print(''Type of variable is: '', type(result))
```

Listing 2.19: Casting a string to a float

Converting from a string to another (numerical) type is often necessary when using the built-in function, *input()*, to read in key strokes from the keyboard. By default the *input()* function returns a string data type. This is fine if the user is being prompted to enter, say, a name or address; however, there are circumstances when the user is being prompted for a numerical input that will be subject to some form of mathematical manipulation. Type in the code in Listing 2.20 and notice the result.

```
1  # Define two strings with numerical values
2  # from keyboard
3  a_str = input(''Enter an integer:'')
4  b_str = input(''Enter an integer:'')
5
6  # Cast strings to ints
7  a_int = int(a_str)
8  b_int = int(b_str)
9
10 # Use the '+' operator
11 result = a_int + b_int
12
13 print(''Result = '', result)
14
15 # Verify type
16
17 print(''Type of variable is: '', type(result))
```

Listing 2.20: Converting input strings

Run the code in Listing 2.20 again, however, this time enter a string of characters from the alphabet, verify that this produces an error. It should be clear that an error occurs because it make no logical sense to convert a sequence of characters to a number unless of course these characters are numerals. This realisation highlights that fact that it is not permissible to convert from all data types to all others, and further that some knowledge of the value of the data type may be needed at the time the code is being written.

2.5 Advanced Material

Where applicable, each lesson will be accompanied with some advanced material that is not examinable and may be of interest to those who can already program in Python or another language. This material is often quite Python-specific; therefore, unlike the mandatory material, it cannot be readily ported to other languages. Please refrain from using this material in any exercises demonstrated in the first semester.

2.5.1 Tuple

A data type that is quite useful in Python is a tuple. A tuple is essentially an ordered immutable (not changeable) collection of variables (of other data types). We will examine collections later in the course in Section 7.

To define a tuple, simply enclose your other variables (data types) in parenthesis as shown in Listing 2.21. Square brackets are used to accessed the internal elements of a tuple (remember the first element is at zero). This demonstrates that tuples are ordered.

```
1 # Create a tuple
2 my_tuple = (1, 2, 3)
3 print(my_tuple[1])
```

Listing 2.21: Create a tuple of integers

As mentioned, tuples can contain various data types: in the example shown in Listing 2.22, a tuple of strings is created, and one element is printed to the screen.

```
1 # Create a tuple
2 my_tuple = ('one', 'two', 'three')
3 print(my_tuple[1])
```

Listing 2.22: Create a tuple of strings

In the example shown in Listing 2.23, a tuple of strings is created. We attempt to modify the tuple (overwrite one of the elements), but this generates an error (TypeError: 'tuple' object does not support item assignment). This demonstrates the immutable nature of tuples: the elements of a tuple cannot be changed after it is created.

```
1 # Create a tuple
2 my_tuple = ('one', 'two', 'three')
3 my_tuple[1] = 'four' # This will generate an error!
```

Listing 2.23: Immutable data type

It is possible to collectively access the individual items in a tuple as shown in Listing 2.24. However, the number of variables must match the number of elements in the tuple.

```
# Create a tuple
my_tuple = ('one', 'two', 'three')
x, y, z = my_tuple
```

Listing 2.24: Accessing elements within a tuple

2.6 Summary

Based on the lessons presented in this section the following key points should be remembered.

1. There are a variety of simple data types: floats, integers, strings, boolean.
2. It is important to select the data type appropriate to the processing that will be performed on it.
3. Operators are type-specific, their behaviour will vary dependent on data type.
4. It is possible to convert from some data types to others, though this can lead to a loss of information, e.g. when converting from a float to an integer.
5. Type conversion may require some prior knowledge of the contents of the variable to be converted to avoid run-time error.

2.7 Problem Sets

1. Write a computer program that prompts the user to enter their name via the keyboard, then prints to the screen the word 'hello' followed by the name entered. Hint: you will need the input() and print() functions. For example, the console should print a cue: 'Please enter your name: '. If the user types 'Robert' then the console responds with 'Hello Robert'.
2. Write a computer program the prompts the user to enter an integer followed by a second integer. The program will print to the screen the remainder of the first integer divided by the second. Hint: you will need to cast the inputs to the correct type. For example, the console should display the message 'Please enter an integer' and the user can enter a value, say 10. Next the console should display the message 'Please enter another integer' and the user can enter a value, say 3. The console will then print the message 'The remainder is 1'.

3. Write a computer program that prompts a user to enter a boolean value (False or True) from the keyboard, followed by a second boolean value. These human inputs can be regarded as inputs to a digital logic OR gate. The program will print to the screen the correct output of the OR gate for the associated input. Check to ensure you code produces the correct answer for all 4 possible input combinations. The truth table of a two-input OR gate is shown in Table 3.

I_1	I_2	Out
False	False	False
False	True	True
True	False	True
True	True	True

Table 2.3: OR Gate Truth Table

Chapter 3

Branches

Decision making is one of the foundations of computer programming, for the programmer this translates into the ability to conditionally execute a sequence of instructions and this gives rise to the notion of a branch in the execution of code. Conditional execution involves some form of comparison ($x == y, x == 10, x > 10$, x *and* y), the interpreter evaluates the comparison and generates a True or False response. If the condition is True then some optional branch of code is executed before control returns to the main body of the program.

For example, consider a program that multiplies the absolute value of a number by two. The absolute value (i.e. the modulus) is its numerical value ignoring the sign, as shown in Equation 3.1. If a number is positive, its value is already its absolute value. However, if it is negative, we need to remove the sign, which can be achieved by multiplying it by -1.

$$abs(x) = |x| = \begin{cases} x, & x \geq 0 \\ -x, & x < 0 \end{cases} \tag{3.1}$$

The solution to such a problem could be codified as shown in Listing 3.1, it uses the *if* statement (lines 7 & 8) to conditionally execute a branch of code.

```
1  # Prompt user for an input and cast to int
2  ip = int(input(''Enter an integer: ''))
3
4  # On the condition that ip is negative
5  # multiply it by -1 to make it
6  # positive (absolute value)
7  if (ip < 0):
8      ip = ip * (-1)
9
10 # Multiply the absolute value by 2
11 result = ip * 2
```

```
12
13 # Print result to screen
14 print(''Result = '', result)
```

Listing 3.1: A simple decision using the *if* statement

Before we examine the functionality of this code fragment, it is important to understand how the Python programming language demarks instructions. Many programming languages such as C, C++, and Java use a semicolon to demark instructions, i.e. a semicolon will appear at the end of every line of code. In Python a newline is used, i.e. carriage return. Sometimes it is necessary to define a *block* of code; this is useful if we wish to conditionally execute a *sequence* of instructions. This is particularly relevant to *if* statements. In many other languages curly brackets are used to define a block of code ({}), Python however uses **indentation**. To highlight this difference, google the search term "C++ if statement" and compare the general structure of this command with that in Listing 3.1.

Returning to the example of the absolute value of a number, the flow of execution in this solution is depicted in the flowchart in Figure 3.1.

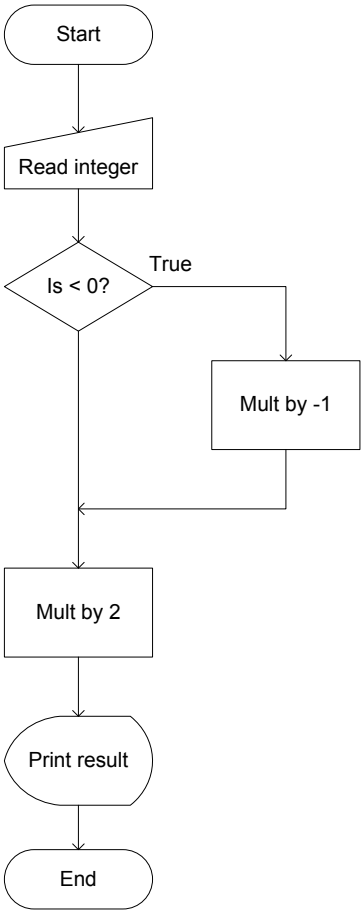


Figure 3.1: Flowchart of a simple decision

Decisions are taken using *if* statements, an *if* statement will permit a sequence of instructions to be executed if its associated condition resolves to the boolean value True, as shown in Figure 3.2. In this case the condition is the return value of the expression *(ip <0)*[1]; recall this produces a boolean data type as discussed in Section 2.3. Thus the code in the branch will be executed whenever the expression within the parenthesis evaluates to True.

[1]The parenthesis are not strictly required but aid readability.

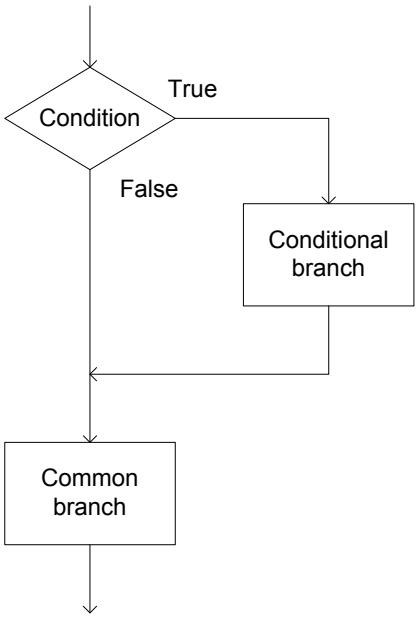


Figure 3.2: Conditional execution

3.1 Mutually Exclusive Execution

Sometimes it is useful to have two mutually exclusive branches of execution, for example consider a fragment of code that determines if a number is odd or even and prints this to the screen. Such an example could be codified as shown in Listing 3.2. This code fragment uses the *else* keyword to provide a second code block of instructions.

```
1  # Prompt user for an input and cast to int
2  ip = int(input(''Enter an integer: ''))
3
4  # If ip is +ve it will have remainder
5  # zero when divided by two
6  if ip % 2 == 0:
7      # Code block for True condition
8      print(''Number is even'')
9  else:
10     # Code block for False condition
11     print(''Number is odd'')
12
13 print(''End of program.'')
```

Listing 3.2: Mutually exclusive execution

To determine if a number is odd or even, we can divide it by two, if the remainder is zero then the number is even, otherwise it is odd. Examine line 6 of Listing 3.2, it uses the modulus operator to check the remainder when dividing by 2. If this condition resolves to True then the branch comprising of lines 7-8 are executed, and if False the branch comprised of lines 10 - 11 are executed. The *print()* statement on line 13 is executed regardless of the outcome of the condition, and this is what is meant by the main body of code in this context.

Pictorially, mutually exclusive branches can be represented by the flowchart shown in Figure 3.3.

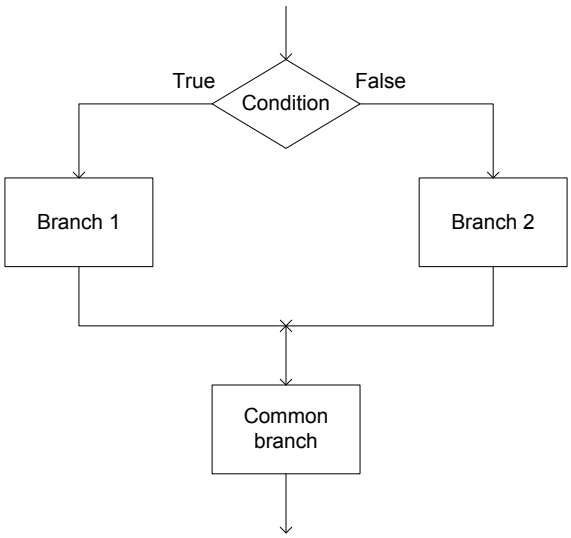


Figure 3.3: Mutually exclusive execution

3.2 Multiple Branches

Both of the previous examples represent a straight forward two-way (binary) decision because the condition is a boolean expression that resolves to only one of two possible values: True or False. However, there are many situations where is is desirable to have more than two mutually exclusive branches of execution.

Consider the hypothetical example of a program that determines if a number is positive, negative, or zero. This can be achieved by *nesting* multiple *if* statements, i.e. one branch of the *if* - *else* construct (line 5) contains another *if* statement (line 12) as shown in Listing 3.3. The program logic for this code is shown in Figure 3.4 where the dotted box highlights the second level of decision making. Pay special attention to how indentation defines a block of code. A block of code is defined between lines 6 and 17 as part of the outer *if* statement, and within this block two further sub-blocks are defined. The first sub-block relates to

the inner *if* statement on lines 13 - 14 and the second sub-block relates to the inner *else* statement on lines 16 - 17. In this case the inner *if* - *else* construct is contained within the outer *else* statement.

```
1  # Prompt user for an input and cast to int
2  ip = int(input(''Enter an integer: ''))
3
4  # First if statement
5  if ip > 0:
6      # Block for 1st branch of 1st condition
7      print(''Number is positive'')
8  else:
9      # Block for 2nd branch of 1st condition
10     # this contains a 2nd if statement with a
11     # 2nd condition
12     if ip < 0:
13         # Block for 1st branch of 2nd condition
14         print(''Number is negative'')
15     else:
16         # Block for 2nd branch of 2nd condition
17         print(''Number is zero'')
18
19 print(''End of program.'')
```

Listing 3.3: Nested if statements

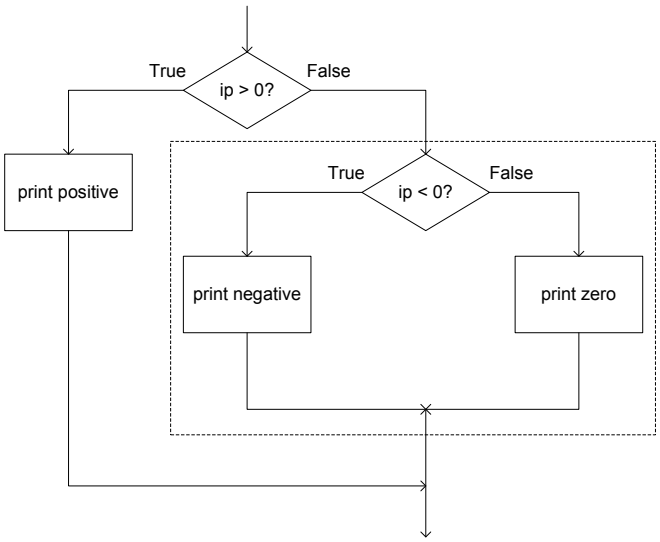


Figure 3.4: Nested if statement

Like many other languages, Python provides a convenient alternative to explicit nesting of decisions. The *elif* statement (derived from else-if) can be used in conjunction with the *if* statement to evaluate multiple conditions and hence permit multiple branches of execution, an example of this is shown in Listing 3.4.

```
1  # Prompt user for an input and cast to int
2  ip = int(input(''Enter an integer: ''))
3
4  # Test value of ip
5  if ip > 0:
6      # Code block for first branch
7      print(''Number is positive'')
8  elif ip < 0:
9      # Code block for second branch
10     print(''Number is negative'')
11 else:
12     # Code block for branch branch
13     print(''Number is zero'')
14
15 print(''End of program.'')
```

Listing 3.4: Multiple branches

This approach to implementing multiple branches is more easy to read and understand than explicit nesting of successive *if* statements. For that reason it is generally preferred; however, there are circumstances where nested decisions may be more appropriate as discussed in Section 3.3.

3.3 Multiple Variables

The examples considered in this chapter thus far have concerned decision making based on the value of a single variable, *ip*. There are many situations where the branch of code to be executed depends on the values multiple variables, i.e. a combination of expressions. For example, Listing 3.5 could be a fragment of code from an employee database program that determines an employee's name prefix, e.g. Miss, Mr, Mrs. The listing assumes that the *gender* variable can hold the values "male" or "female" and the status variable can take the values "married" or "single". Note that in this example, the first condition to be processed depends on *gender* and the second condition depends on *status*. Where the conditions to be assessed depend on multiple variables rather on multiple values of a single variable then a nested decision may be more readable than compared to using successive *elif* statements.

```
1  # Determine name prefix
2  # Perform first test on gender variable
3  if gender == ''female'':
4      # Perform second test on status variable
5      if status == ''single'':
```

```
        print(''Prefix is Miss'')
    else:
        print(''Prefix is Mrs'')
else:
    print(''Prefix is Mr'')


print(''End of program.'')
```

Listing 3.5: Decisions on multiple variables

3.4 Multiple Conditions

As has been mentioned, sometimes it is necessary to conditionally execute code based on multiple conditions, either because multiple variables are involved or because multiple values are involved. In these cases, it is often possible to obviate the need for nested decisions by building compound boolean expressions. This can be achieved using the built-in boolean operators *and*, *or*, and *not*, as show in Listing 3.6; enter this code and verify its output. Branch 1 of the code will only be executed if both parts are True, i.e. *gender* equates to female and *status* equates to single. Building compound expressions can make code more efficient and readable. Indeed Karnaugh Maps can be useful in simplifying complex boolean expressions.

```
# Prompt user for gender
gender = input(''Enter your gender: '')
# Prompt user for marital status
status = input(''Enter you marital status: '')

# Determine name prefix
# Perform first test on gender AND status
if gender == ''female'' and status == ''single'':
    # Branch 1
    print(''Prefix is Miss'')
elif gender == ''male'':
    # Branch 2
    print(''Prefix is Mr'')
else:
    # Branch 3
    print(''Prefix is Mrs'')

print(''End of program.'')
```

Listing 3.6: Compound condition

Another use of combining conditions is to define a range of acceptable values of a variable. Take the somewhat contrived example whereby people are only permitted to drive between the ages of 17 and, say,

69. The code fragment in Listing 3.7 executes conditional code if the variable *age* is within that range. Note the use of the *and* operator to specify two conditions that have to be met.

```
# Prompt user for age
age = int(input(''Enter your age: ''))

# Determine if fit to drive based on age

if age >= 17 and age < 70:
  # Branch 1
  print(''Able to drive'')
else:
  # Branch 2
  print(''Unable to drive'')

print(''End of program.'')
```

Listing 3.7: Multiple values

Most programming languages permit conditions to be combined in this manner using boolean operators. This course is concerned with learning programming in general rather than learning Python. With that in mind, it is strongly recommended to avoid combining operators without the use of boolean operators; this is permitted in Python but not in most other languages. The code fragment in Listing 3.8 chains operators together to define a range. This is valid code in Python and will operate correctly, furthermore, it is arguably more intuitive code to read owing to its similarity to mathematical notation. However, it is strongly advised to avoid chaining operators in this fashion and use boolean operators instead for the simple reason that the latter approach can be used with many other general purpose programming languages.

```
# Prompt user for age
age = int(input(''Enter your age: ''))

# Determine if fit to drive based on age

if 17 <= age < 70:
  # Branch 1
  print(''Able to drive'')
else:
  # Branch 2
  print(''Unable to drive'')

print(''End of program.'')
```

Listing 3.8: Chained operators

3.5 Decision Shorthand

As previously mentioned, each decision is executed using an *if* statement which takes a condition in the form of a boolean expression which resolves to either True or False. Within many computer programming languages, the value of zero (integer 0, or real 0.0) can be substituted for the boolean value False. Similarly, any non-zero value can be substituted for the boolean value True, note this includes negative numbers. Enter some sample values into the program in Listing 3.9 and verify the result.

```
1  # Prompt user for an input and cast to float
2  ip = float(input(''Enter a float: ''))
3
4  # Test value of ip
5  if ip:
6      print(''Number is non-zero'')
7  else:
8      print(''Number is zero'')
9
10 print(''End of program.'')
```

Listing 3.9: Numbers as conditions

It should be apparent from the listing that *ip* is not compared to any other value. In fact this form of decision making represents a form of shorthand that is very common in computer programming. Programmers often exploit this shorthand to write more compact and efficient code, albeit at the expense of readability. The greater efficiency is derived from the fact that fewer operations are required, i.e. the computer does not have to perform a comparison and consequently has fewer operations to execute in the microprocessor.

3.6 Efficiency

Let us now look at writing more efficient (faster) code. Compare and consider the two examples of code in Listings 3.10 and 3.11.

```
1
2  # Define a variable and set to an arbitrary value
3  a = 11
4
5  # An if statement
6  if a > 10:
7      print(''Above threshold'')
8
9  # Another if statement
10 if a <= 10:
```

```
11      print(``Not above threshold'')
12
13  # Back to main body of code
14  print(``End of program'')
```

Listing 3.10: Two *if* statements

```
1
2  # Define a variable and set to an arbitrary value
3  a = 11
4
5  # An if statement
6  if a > 10:
7      print(``Above threshold'')
8  else:
9      print(``Not above threshold'')
10
11 # Back to main body of code
12 print(``End of program'')
```

Listing 3.11: One *if* statement

The code fragment in Listing 3.10 will always test two conditions (one in each *if* statement), whereas the code fragment in Listing 3.11 only requires a single condition to be tested and is therefore faster.

It can be tempting to use an *if* statement to set the value of a boolean variable, e.g. see Listing 3.12 which sets the value of a boolean flag based on some condition.

```
1
2  # Define a variable and set to an arbitrary value
3  a = 11
4
5  # An if statement
6  if a > 10:
7      flag = True
8  else:
9      flag = False
10
11 # Back to main body of code
12 print(flag)
13 print(``End of program'')
```

Listing 3.12: *if* statement - set boolean value

A more efficient approach is to avoid computationally expensive branching altogether by NOT using an *if* statement at all! See Listing 3.13.

```
1 # Define a variable and set to an arbitrary value
2 a = 11
3
4 flag = a > 10
5
6 print(flag)
7 print(''End of program'')
```

Listing 3.13: No *if* statement - set boolean value

3.7 Advanced Material

Where applicable, each lesson will be accompanied with some advanced material that is not examinable and may be of interest to those who can already program in Python or another language. This material is often quite Python-specific; therefore, unlike the mandatory material, it cannot be readily ported to other languages. Please refrain from using this material in any exercises demonstrated in the first semester.

3.7.1 Conditional Expressions

In Python it is possible to conditionally assign a value to a variable using the convenient syntax shown in Listing 3.14. Note, it is essential that this form of conditional assignment contains both *if* and *else* statements. The syntax should be of the form <expr1> *if* <cond_expr> *else* <expr2>.

```
1 # Define a variable and set to an arbitrary value
2 i = 10
3 # Conditionally assign a value to a string variable
4 s = 'greater than 9' if (i > 9) else 'less than or equal to 9'
5 print(s)
```

Listing 3.14: Conditionally assign a variable

Conditional expressions can be quite powerful, the code in Listing 3.15 demonstrates how they can be used within a function call.

```
1 temperature = 21.0
2 print('To the', 'beach' if (temperature > 20.0) else 'cinema')
```

Listing 3.15: Conditional expression within a function

3.7.2 Match Case Statement

As mentioned in Section 3.2, it is often required to have multiple branches of mutually exclusive execution. Python version 3.10.x onwards provides a convenient shorthand for coding multiple branches, particularly when conditioning on the value of a single variable. The *Match - Case* statement, known more generally as a *case* statement in other languages, is best illustrated by way of example. Suppose we wish to write some code that prints to the screen whether a particular person pays half fare or full fare, we could use the *if-elif-else* construct as demonstrated in Listing 3.16. Line 3 evaluates if person resolves to adult, and if so, line 4 is executed then the other elif and else blocks are skipped. If the value of person does not match that expressed in line 3 then the next condition is tested in line 5, and if this resolves to child then line 6 is executed. Finally, the else statement in line 7 is used as a catch all term to highlight an error if no condition has been matched.

```
1  person = 'adult'
2
3  if person == 'adult':
4      print('Pay full fare')
5  elif person == 'child':
6      print('Pay half fare')
7  else:
8      # Executed by default if no conditions match
9      print('Error: unknown person')
```

Listing 3.16: Example elif with default branch

The same functionality can be implemented using the match-case construct as demonstrated in Listing 3.17. The keyword *match* is used before the variable of interest, and the keyword *case* is used before each possible value. Note, the values must be constants, e.g. 'cat', 11, 10.1, etc.; they cannot be a range of values e.g. < 10. The default branch is indicated by a single underline '_': as seen in line 8.

```
1  person = 'adult'
2
3  match person:
4      case 'adult':
5        print('Pay full fare')
6      case 'child':
7        print''Pay half fare')
8      case _:
9        # Executed by default if no conditions match
10       print('Error: unknown person')
```

Listing 3.17: Example match - case statement

3.8 Summary

Based on the lessons presented in this section the following key points should be remembered.

1. It is possible to control the flow of execution in Python using *if*, *elif*, and *else* statements.
2. A decision is based on a condition which is a boolean expression, therefore, it can have only two mutually exclusive outcomes. Each outcome represents a block of code that can be executed based on that condition.
3. In Python the *elif* statement provides a mechanism conditionally execute one of multiple parallel branches of code.
4. It is also possible to nest *if* statements to conditionally execute one of multiple parallel branches of code, but this can be less readable and should be avoided if possible.
5. Where execution is conditional on the value of multiple variables or multiple conditions, they can be combined using the boolean *and*, *or*, and *not* operators.
6. The boolean values of False and True can be replaced by zero and non-zero numerical values respectively. This can lead to more efficient code.

3.9 Problem Sets

1. Write a program that prompts the user to enter a test score in the range 0 — 100. Depending on the value entered, the program should print to the screen 'Your grade is ' followed by the grade as shown in Table 3.1. For example, if the value entered is 65, then the console should print 'Your grade is B'.

Score	Grade
81 — 100	A
61 — 80	B
41 — 60	C
21 — 40	D
0 — 20	E

Table 3.1: Mapping scores to grades

2. Write a program that prompts the user to enter a password. The program should contain a single *if* statement that prints 'Correct' to the screen if the password is either 'secret' or 'Secret', or 'incorrect' otherwise. Hint you may need to use a boolean operator. For example, the console should print the

message ‘Enter the password’. If the user enters, say ‘secret’, then the console prints ‘Correct’ to the screen. Alternatively, if the user enters, say ‘dogs’, then the console prints ‘Wrong’ to the screen.

3. Write a program that prompts the user to enter a value between -100 and 100. The program should print to the screen whether the number is positive, negative, or zero, and whether to number is odd or even. Try to write efficient code. Hint the modulus operator may prove useful. For example, the console should print ‘Enter a value between -100 and 100’. On entering a value, say -30, the console should print ‘You entered a negative even number’.

Chapter 4

Loops and Iteration

In many circumstances a computer program is required to execute a predefined sequence of instructions until a given task has been completed. This form of looping is known as **iteration**. The loop takes two general forms in Python: the *for* loop, discussed in Section 4.2 and the *while* loop, discussed in Section 4.3.

Both types of loop exist in most other general purpose programming languages. This section will provide an overview of their uses from a general programming standpoint, using Python as a reference. The lessons learned in this section are therefore equally applicable to any other general purpose programming language.

4.1 The Need For Iteration

Before we look at the specifics of loops let us consider some preliminaries. Suppose you wish to update the value of a given variable, say add 5 to its current value. The code in Listing 4.1 does this with the line *a = a + 5*, when this line of code is executed the right side of the assignment is evaulated and the resolved value is stored in the variable on the left of the assignment. The old value of *a* has been overwritten or updated. Updating the value of a variable is a common step in computer programming and this is often facilitated with the use of loops.

```
1 # Define a variable and set to an arbitrary value
2 a = 10
3 # Add 5 to it current value
4 a = a + 5
5 print(a)
```

Listing 4.1: Updating a value

This example provides a convenient opportunity to introduce some operators designed for updating the value of a variable: $+=$, $-=$, $*=$, etc. Compare Listings 4.1 and 4.2.

```
# Define a variable and set to an arbitrary value
a = 10
# Add 5 to it current value
a += 5
print(a)
```

Listing 4.2: The $+=$ operator

Experiment by substituting $*=$ for $+=$ in the code, note there is no space between the $+$ and the $=$. Ensure you understand what is going on with the value of the variable before moving on.

Let us look at another example, suppose we need to repeat the same step of an algorithms over and over, say 10 times. Suppose we wish to print the numbers from 0 to 9 to the screen, we could do this using the code in Listing 4.3. Note how this code repeats the same two steps many times, and indeed involves overwriting (updating) the value of a variable.

```
# Define a variable and set to 0
a = 0
# Print value to screen
print(a)
# Increment value of variable
a = a + 1
# Repeat last two steps
print(a)
a = a + 1
# Repeat again
print(a)
a = a + 1
# And again and so on
print(a)
a = a + 1
print(a)
a = a + 1
print(a)
a = a + 1
print(a)
a = a + 1
print(a)
a = a + 1
print(a)
```

Listing 4.3: Repeated steps

Section 4.2 will introduce the concept of a *for* loop that enables the same functionality to be written in a more convenient form (Listing 4.4).

4.2 For Loop

The *for* loop is particularly useful if a sequence of instructions is to be executed a predefined number of times, i.e. the number of iterations is known before hand. The *for* loop operates by pre-setting a counter variable to an initial value (lower bound) and successively incrementing that value by a specified amount (step size) until an upper bound is reached. The flow of execution for a *for* loop is shown in Figure 4.1. The flowchart shows the counter (index variable) being initialised to the lower bound. On each iteration it is incremented and its value compared to the upper bound as part of the condition.

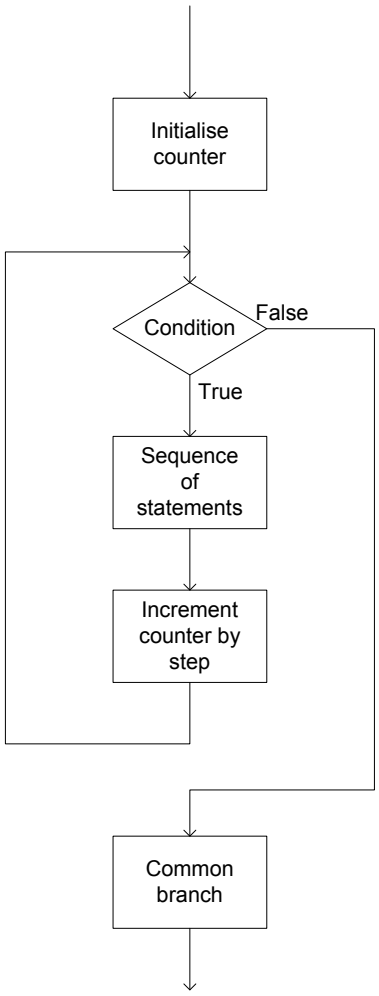


Figure 4.1: *for* loop

4.2.1 The Range Function

Within Python simple *for* loops can be defined with the use of a helper function: *range()*. The *range()* function defines the lower and upper bounds as well as the size of the increments as follows, *range(stop)* will produce a sequence of *num* numbers starting at *0* and ending at *stop-1*; *range(start, stop)* will produce a sequence of numbers starting at *start* and ending at *stop-1*; *range(start,stop, step)* will produce a sequence of numbers between the values *start* and *stop-1*, but only each *step*th value is included. Now that the value returned by the *range()* function has been covered, it is appropriate to discuss its data type. As you may recall, thus far we have considered integers, floating point numbers, strings, and boolean types. The *range()* function returns a more complex data type, which we will refer to as a container object for now, we will

discuss this in greater detail in Section 7. For now it is sufficient to think of the data returned by the *range()* function as a sequence of type range.

4.2.2 For Syntax

In Python, the keyword *for* is followed by an arbitrary variable name, e.g. index or counter. The value of this variable will be updated on each revolution of the loop. It will move to the next value of the sequence (e.g. range) specified after the *in* operator.

At this point it is worth pausing to consider yet another Python keyword, *in*. In is a membership operators. Like the other comparative operators ($==, >, <=$, *and*, *or*), it compares two values and returns True or False based on their respective values. The statement *x in y* returns True if the value contained in x exists in one of the elements in the container object y, otherwise it returns False. We will have more to say about this operator later when we consider string manipulation in Section 6.

Moving on from these technicalities, let's learn by example. Type in and run the code in Listing 4.4, pay special attention to the fact that the final value of the loop is 9; novice Python programmers are often caught out by this. If ever in doubt about the bounds of a loop, include a *print()* statement in its body, and perform a trial run.

The syntax of the *for* loop is fairly simple; the *for* statement is followed by a counter variable, in this case *index*, followed by the *in* keyword, followed by the sequence of numbers to be iterated through, in this case the sequence returned by *range(10)*, followed by a colon.

```
for index in range(10):
    # Body of loop
    print(''Index = '', index)
```

Listing 4.4: *range()* function

Modify the code in Listing 4.4 to print the numbers from 1 to 10 instead of 0 to 9. Lastly, modify the code further to print the numbers 2, 4, 6, 8, 10.

Loops are amongst the most frequently used constructs in programming. Consider a trivial example of calculating the cumulative sum of numbers from 1 to n, i.e.:

$$s_n = 1 + 2 + \cdots + (n-1) + n = \sum_{k=1}^{n} k \tag{4.1}$$

This can be written using a *for* loop as shown Listing 4.5. Type in and test this code.

```
# Prompt user for an input and cast to int
```

```
ip = int(input(``Enter an integer: ''))

# Add 1 to input value to compensate
# for loop ending at end-1
n = ip + 1

# Initialise summation variable
sum = 0

# Set upper bound to n
for index in range(1,n):
    # sum increased by current iteration
    # value
    sum = sum + index

# Print result to screen
print(``Cumulative sum = '', sum)
```

Listing 4.5: Cumulative sum

Modified the code to calculate the factorial of an integer, $n!$, as shown in Listing 4.6.

$$n! = 1 \cdot 2 \cdots (n-1) \cdot n = \prod_{k=1}^{n} k \tag{4.2}$$

```
# Prompt user for an input and cast to int
ip = int(input(``Enter an integer: ''))

# Add 1 to input value to compensate
# for loop ending at end-1
n = ip + 1

# Initialise product variable
# Note 0! = 1
product = 1

# Set upper bound to n
for index in range(1,n):
    # Multiply by current index value
    product = product * index

# Print result to screen
print(``Factorial = '', product)
```

Listing 4.6: Factorial

There are many circumstances where it is necessary to nest loops, i.e. include a loop within a loop. Consider the example to calculate the exponential of an integer value entered via the keyboard. The exponential, e^x can be approximated by the formula given in Equation 4.3, using the first 50 terms.

$$e^x = \frac{x^0}{0!} + \frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{n=0}^{\infty} \frac{x^n}{n!} \approx \sum_{n=0}^{49} \frac{x^n}{n!} \tag{4.3}$$

```
1  # Prompt user for an input and cast to int
2  x = int(input(''Enter an integer: ''))
3
4  # Initialise the result
5  result = 0.0
6
7  for n in range(50):
8      # calc ip to the power n
9      numerator = x ** n
10
11     # Now calc the factorial
12     # denominator is the factorial
13     # Note 0! = 1
14     denominator = 1
15     # Add 1 to n to compensate
16     # for factorial loop ending at end-1
17     for index in range(1,n+1):
18         # Multiply by current index value
19         denominator = denominator * index
20     # Do the cumulative summation
21     # term by term
22     result = result + (numerator / denominator)
23
24 # Print result to screen
25 print(''Exponential of '', x, ''is '', result)
```

Listing 4.7: Nested loop: exponential

The code to calculate the exponential is shown in Listing 4.7. The outer *for* loop (line 7) is used to calculate the first 50 terms (0 - 49) of the infinite summation; it is assumed this will be accurate enough for most purposes. Each term in the summation can be represented by a numerator divided by a denominator[1]: $\frac{x^n}{n!}$. Line 9 takes the input value, x, and raise it to the power of n, this provides the numerator. The denominator in this case is the factorial and uses the same code as that in Listing 4.6; the inner *for* loop (line 17) is used for this purpose. Note that the value passed to the inner loop is not a constant as in the previous cases, it is the current index value of the outer loop, n, therefore, the bounds of the inner *for* loop change on every successive iteration of the outer loop.

[1]Note: $\frac{x^0}{0!} = 1$.

As mentioned, the *for* loop is particularly useful for repetition if the number of iterations is known in advance; however, there are circumstances where the number of iterations cannot be expressed explicitly. In these situations the *while* loop, as described in Section 4.3 may prove useful.

4.3 While Loop

Unlike the *for* loop which iterates a predefined number of times, the *while* loop continues to repeat while some condition is met. Equivalently, the *while* loop will continue to repeat until some condition is violated. The condition is a boolean expression in the same form as used with *if* statements. The flow of execution for a *while* loop is shown in Figure 4.2.

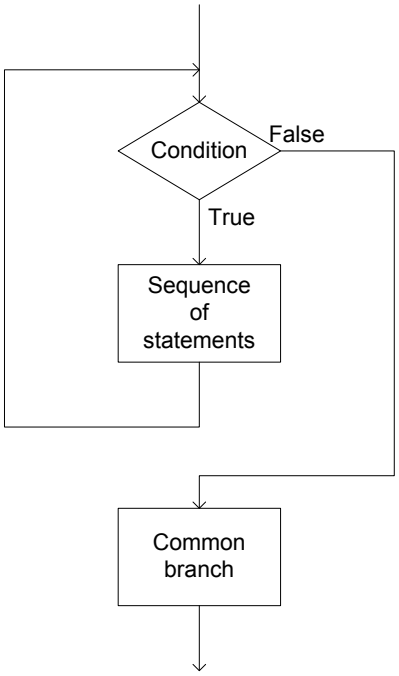


Figure 4.2: *while* loop

The trivial example in Listing 4.8 demonstrates the use of a *while* loop to prompt the user to enter a positive integer from the keyboard. If the user enters a negative integer the loop will repeat and prompt the user once more. The loop will only exit when the user enters a positive integer; on this occurrence the condition will be set to False by the *if* statement in lines 8 and 9. The important point about this form of loop is that the programmer will not know in advance who many attempts the user will take to enter a positive integer, i.e. the number of iterations is determined at run-time, not at the programming stage. Type in the code in Listing 4.8 and test it.

```
1  # Set condition to True to ensure loop executes at
2  # least once
3  condition = True
4  # Loop until sure that input is +ve
5  while condition == True:
6      # Prompt user for an input and cast to int
7      x = int(input(''Enter a positive integer: ''))
8      if x > 0:
9          condition = False
10
11 print(''The positive integer is '', x)
```

Listing 4.8: While loop

The example in Listing 4.8 demonstrates that a *while* loop can be used to accomplish some programming goals that cannot be achieved by a *for* loop. Indeed, anything that can be achieved by a *for* loop can also be achieved by a *while* loop; however, the converse is not true. The example Listing 4.5 can easily be rewritten in the form of a *while* loop to achieve exactly the same objective as shown in Listing 4.9. An index is declared and defined on line 7, this is used as a counter within the loop; the very last line of code in the loop (line 14) increments this counter. Note the use of the less-than or equal-to operator in the *while* loop. Take a few moments to compare and contrast these two approaches.

```
1  # Prompt user for an input and cast to int
2  ip = int(input(''Enter an integer: ''))
3
4  # Initialise summation variable
5  sum = 0
6  # Initialise the index i.e. the counter
7  index = 1
8  # Repeat while index is less than
9  # or equal to the input
10 while index <= ip:
11     sum = sum + index
12     # increment the counter
13     # on each iteration
14     index = index + 1
15
16 # Print result to screen
17 print(''Cumulative sum = '', sum)
```

Listing 4.9: Cumulative sum, revisited

Another useful application of the *while* loop is counting. The code fragment in Listing 4.10 counts the number of attempts it takes for a user to successfully enter a positive integer. Clearly, this functionality cannot be achieved by a *for* loop since the number of iterations cannot be knowable in advance.

```
1  # Set condition to True to ensure loop executes at
2  # least once
3  condition = True
4  # Initialise counter
5  counter = 0
6  # Loop until sure that input is +ve
7  while condition == True:
8      # Increment counter for each attempt
9      counter = counter + 1
10     # Prompt user for an input and cast to int
11     x = int(input(``Enter a positive integer: ''))
12     if x > 0:
13         condition = False
14
15 print(``Number of attempts is '', counter)
```

Listing 4.10: Counting user attempts

4.4 Loop Control

There are situations where it is beneficial to only partially execute the sequence of statements contained in a loop. At some specified point in the execution the programmer may wish to return to the start of the loop or exit the loop altogether.

Take for example the code fragment in Listing 4.11 which can be used to calculate the sum of the square of each number between one and ten. Type in and run this example to verify the output.

```
1  # Initialise running total
2  total = 0
3  # Initialise counter
4  number = 0
5  # Loop while less than ten
6  while number < 10:
7      # Increment counter
8      number = number + 1
9      # Compute the square
10     square = number ** 2
11     # Add squared value to total
12     total = total + square
13
14 # Print to screen
15 print(``The sum of the squares is '', total)
```

Listing 4.11: Square all numbers between 1 and 10

Now suppose instead we wish to calculate the sum of the square of each number between one and ten, except the number five. This can be achieved by checking for the condition that *number* equals the value 5, and if so returning to the top of the loop without adding the squared value to the running total. Listing 4.12 demonstrates how this can be achieved. The *if* statement on line 13 checks to see if the current number being considered is 5, if so it executes the *continue* statement on line 14. It is the *continue* statement that returns the flow of execution back to the top of the loop, as demonstrated in Figure 4.3. In doing so the final line of code in the loop (line 16) is not executed on this particular iteration therefore 5^2 is not included in the summation. Try the code and vary the number that is the exception.

```
1  # Initialise running total
2  total = 0
3  # Initialise counter
4  number = 0
5  # Loop while less than ten
6  while number < 10:
7      # Increment counter
8      number = number + 1
9      # Compute the square
10     square = number ** 2
11     # Go back to top of loop
12     # if the number is 5
13     if number == 5:
14         continue
15     # Add squared value to total
16     total = total + square
17
18 # Print to screen
19 print(``The sum of the squares is '', total)
```

Listing 4.12: Continue statement

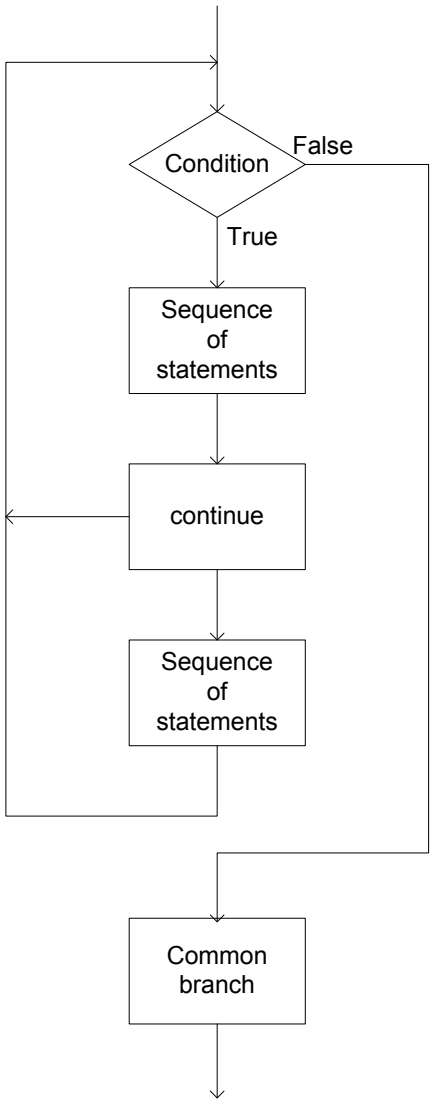


Figure 4.3: *continue* statement

There are other occasions where it is necessary to exit the loop prematurely based on some condition. If this is required then the *break* statement is used, as shown in Figure 4.4. In the example in Listing 4.13, the sum of the squares of the numbers between one and ten is computed and printed to the screen on each iteration; however, in this example the user is prompted to determine if they would like to continue. If they choose to enter "n" then the loop terminates and no further values are considered.

```
1 # Initialise running total
```

```
2  total = 0
3  # Initialise counter
4  number = 0
5  # Loop while less than ten
6  while number < 10:
7      # Increment counter
8      number = number + 1
9      # Calculate the square
10     square = number ** 2
11     # Add squared value to total
12     total = total + square
13     # Print to screen
14     print(``The sum of the squares is '', total)
15     # Prompt user to continue
16     ip = input(``Continue? (y/n): '')
17     if ip == ``n'':
18         break
19
20 print(``End of program.'')
```

Listing 4.13: Break statement

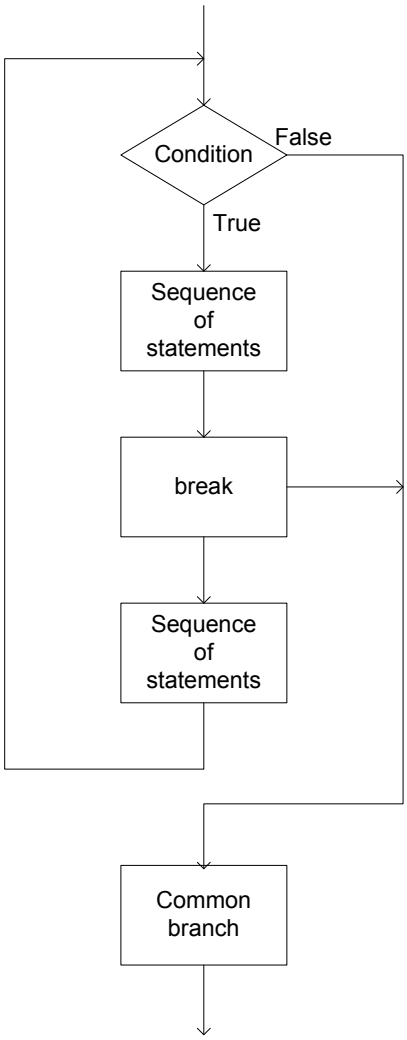


Figure 4.4: *break* statement

Given that the *break* statement can be used to exit a loop when a given condition occurs it is possible to use it for normal termination of a loop. The program that calculates the cumulative sum of a number entered by the user (Listing 4.9) can be rewritten using an infinite loop as shown in Listing 4.14. The decision to exit the loop is not taken at the top of the loop, since the condition is always True. The decision to exit is taken by the *if* - *break* combination in lines 15 and 16. Both loops are equivalent; however, the version in Listing 4.14 would be harder to read by other programmers because they would have to search the code for the exit clause. For that reason it is generally recommended that *break* statements should be used for abnormal termination of a loop (e.g. terminate on error) rather than for normal termination.

```
1  # Prompt user for an input and cast to int
```

```
ip = int(input(''Enter an integer: ''))

# Initialise summation variable
sum = 0
# Initialise the index i.e. the counter
index = 1
# Repeat while index is less than
# or equal to the input
while True:
    sum = sum + index
    # increment the counter
    # on each iteration
    index = index + 1
    if index > ip:
        break

# Print result to screen
print(''Cumulative sum = '', sum)
```

Listing 4.14: Infinite loop

The *continue* and *break* statements can also be applied to *for* loops. Create your own *for* loop to calculate the sum of the squares between 1 and 10 except 5.

4.5 Advanced Material

Where applicable, each lesson will be accompanied with some advanced material that is not examinable and may be of interest to those who can already program in Python or another language. This material is often quite Python-specific; therefore, unlike the mandatory material, it cannot be readily ported to other languages. Please refrain from using this material in any exercises demonstrated in the first semester.

4.5.1 For-Else

A somewhat unusual construct that is available in Python is the *for-else* construct, normally an *else* clause is associated with an *if* statement. Often a loop can be used to determine the location of an item within a collection such as a character string. Execute the code in Listing 4.15, and check that it correctly identifies the location of the letter 'q' (lower case). Next change the search term to 'Q' upper case and observe the result. Finally, delete the *else* clause and re-run.

```
sentence = 'the quick brown fox jumps over the lazy dog'
search_term = 'q'

index = 0
```

```
5  for letter in sentence:
6      if letter == search_term:
7          print(search_term + ' is in position', index)
8          break
9      index = index+1
10 else:
11     print(search_term + ' not found')
```

Listing 4.15: For-else

4.5.2 The enumerate() Function

An useful built-in function when writing *for* loops is *enumerate()*, as shown in Listing 4.16. This code performs the same function as that in Listing 4.15; however, the index variable does not have to be explicitly defined and updated. The function returns two values: index keeps track of the number of iterations of the loop automatically and the next variable (letter) is assigned the item at that index.

```
1  sentence = 'the quick brown fox jumps over the lazy dog'
2  search_term = 'q'
3
4  for index, letter in enumerate(sentence):
5      if letter == search_term:
6          print(search_term + ' is in pos', index)
7          break
8  else:
9      print(search_term + ' not found')
```

Listing 4.16: Using enumerate()

4.6 Summary

Based on the lessons presented in this section the following key points should be remembered.

1. *for* loops are useful when the number of iterations is known in advance.
2. *while* loops are useful when a sequence of instructions should be executed until a condition occurs.
3. Anything that is achievable by a *for* loop can also be achieved by a *while* loop.
4. The *continue* statement moves the point of execution back to the top of the loop.
5. The *break* statement is used to exit the loop.
6. The use of *while True:* and *break* is generally frowned upon because it makes the code less readable.

4.7 Problem Sets

Computer programming requires a range of skills: knowledge of the syntax of a particular language, an understanding of data types and algorithms, the ability to test and debug code, experience in structuring code appropriately, and the ability to translate concepts from the physical world into a working application. The translation of real world problems into working code is perhaps the most creative and frustrating aspect of programming. The problems below are designed to encourage this creativity, and highlight the advantage of having a notepad and pen ready when writing code.

1. Use a *for* loop to print the numbers 1, 2, 3, ..., 9, 10 to the screen. For example, the console should print '1, 2, 3, 4, 5, 6, 7, 8, 9, 10'.

2. Modify the first exercise to print the numbers 2, 4, 6, 8, 10 to the screen. For example, the console should print '2, 4, 6, 8, 10'.

3. Look up the phrase "Taylor Series" in Wikipedia, there should be several expansions for well-known mathematical identities. With reference to equations 4.4, write some code to calculate the hyperbolic cosine ($\cosh$) of a floating point number entered from the keyboard, your code should consider the first 50 terms. Before you begin, carefully consider whether a *for* loop or *while* loop is more appropriate. The objectives of this exercise are to gain experience (i) writing a nested loop, and (ii) to experiment with different step sizes. Hints: work on the inner loop first to ensure it operates as you expect; use *print()* statements during the writing and testing phase within the body of the loop. For example, the console should print 'Enter a real number'. Say the user enters 1.1, the the console responds with 'The hyperbolic cosine of 1.1 is 1.68551855382'. Test you code for a range of values, note $\cosh 1.0$ is 1.54308063482, and the $\cosh 2$ is 3.76219569108. Ensure you get the correct results.

$$\cosh x = \frac{x^0}{0!} + \frac{x^2}{2!} + \frac{x^4}{4!} + \frac{x^6}{6!} + \cdots \tag{4.4}$$

4. Look up the phrase "Fibonacci Number" in Wikipedia. This should describe a sequence of numbers: 0, 1, 1, 2, 3, 5, ... Translate this description into working code that prints the first 10 numbers of the sequence. The objective of this exercise is to demonstrate the necessary thought process required to initialise variables appropriately before entering a loop. Your code should contain a singe *for* loop and a single *print()* statement. For example, the console should print the following to the screen '0, 1, 1, 2, 3, 5, 8, 13, 21, 34'.

Chapter 5

Input Output In Python

This section will review the functionality built into Python for displaying text on the screen and providing keyboard inputs. Of course, it is also possible to interact with a computer program by reading from or writing to data files stored on the hard disk; therefore, a brief overview of file input-output will also be provided.

5.1 Printing To The Screen

In Python printing to the screen is accomplished using the *print()* function: the value of the variable(s) will be printed to the screen if the name of those variables is enclosed between the brackets following the word *print*, as shown in Listing 5.1. Hopefully, this is intuitively obvious given the previous exposure to this command throughout the course.

```
# Declare some variables
i = 10
f = 11.1
s = ''hello''

# Print variables to the screen
print(i)
print(f)
print(s)
```

Listing 5.1: Printing variables

Listing 5.1 defines three variables *i*, *f*, and *s* and prints them to the screen in succession. The same result can be achieved by placing the value of the variable directly into the *print()* function as shown in Listing 5.2.

```
1 # Print values to the screen
2 print(10)
3 print(11.1)
4 print(''hello'')
```

Listing 5.2: Printing literals

The example given Listing 5.2 permits the introduction of some new terminology. Whereas Listing 5.1 prints an *int* variable, a *float* variable and a *string* variable to the screen, Listing 5.2 prints an *int* **literal**, a *float* **literal** and a *string* **literal** to the screen. A literal represents a fixed value in computer programming whereas a variable is a symbol that can hold any value of its type. Using literals in a *print()* function is often more convenient and intuitive than defining a variable beforehand then printing the value of that variable.

Formatting the printed output can be achieved in a number of ways. Type in and compare the printed output of the programs in Listings 5.3 & 5.4.

```
1 # Define two strings
2 s1 = ''hello''
3 s2 = ''world''
4 # Print both strings with a comma
5 print(s1, s2)
```

Listing 5.3: Print format 1

```
1 # Define two strings
2 s1 = ''hello''
3 s2 = ''world''
4 # Print both strings with a plus
5 print(s1 + s2)
```

Listing 5.4: Print format 2

Why are the two outputs different? Perhaps the answer can be found in Section 2.2.

Often when printing strings to the screen, the use of control characters (sometimes referred to as escape sequences) can be useful. The code snippets in Listings 5.5 & 5.6 contain the control characters \n and \t respectively. Enter the code in these listings and observe the result.

```
1 # Define a string
2 s = ''hello \nworld''
3 # Print the string
```

```
4 print(s)
```

Listing 5.5: Newline

```
1 # Define a string
2 s = ``hello \tworld''
3 # Print the string
4 print(s)
```

Listing 5.6: Tab

It should be apparent that the control characters in Listings 5.5 & 5.6 produced a newline or a tab to be printed to the screen. These special characters can be used to control the layout of information printed to the screen.

5.1.1 Inserting values into a string: general purpose approach

It is a common task in computer programming to insert the values of variables directly into a string at some specified point. Consider a problem that requires the the user to enter their age (years and months, e.g. 21 years and 6 months) then prints to the screen "My age is 21 years and 6 months". This can be achieved using the code fragment in Listing 5.7.

```
1 years = int(input(``Enter years :''))
2 months = int(input(``Enter months :''))
3
4 print(``My age is '', years, `` years and '',
5       months, `` months'')
```

Listing 5.7: Age example

A more convenient syntax exists to permit us to insert the integers directly into the string using the %d format specifier; the % symbols here is know as the string interpolation operator. This syntax is demonstrated in Listing 5.8. The programming languages C, C++, and to some extent Java, use a similar syntax. Therefore, it is beneficial to learn how to use these format specifiers given the overall goal of learning to program in general rather than just learning the Python programming language.

```
1 years = int(input(``Enter years :''))
2 months = int(input(``Enter months :''))
3
4 print(``My age is %d years and %d months'' %(years, months))
```

Listing 5.8: %d example

The %d specifier is used to insert integers only into a string. Within a string simply put a %d where you would like an integer to be inserted, note this may be in multiple locations. At the end of the string, use the % sign again and then include the variable names of the integers in brackets. The first integer in the brackets will map to the first occurrence of %d in the string, the second integer with map to the second occurrence of %d, and so on.

The %f and %s format specifiers can be used to insert floats and strings respectively. The code in Listing 5.9 inserts a string and a fixed point number into a string for printing to the screen. Type in and run this code, it should print to the screen *My height is 180.123457*. Note that the value of the floating point number has been truncated (and rounded up) to 7 decimal places.

```
1 sub_string = ``height''
2 my_height=180.123456789
3
4 print(``My %s is %f'' %(sub_string , my_height))
```

Listing 5.9: Float and string example

We can further express the format of specifiers using some modifiers of the form *fieldwidth.precision*. The *fieldwidth* value expresses the minimum width of the printed field and the *precision* expresses how many decimal places are used. Modify the code such that it mirrors that in Listing 5.10, and experiment with different modifier values.

```
1 sub_string = ``height''
2 my_height=180.123456789
3
4 print(``My %s is %3.8f'' %(sub_string , my_height))
```

Listing 5.10: Modifiers

5.1.2 Inserting values into a string: pythonic approach

Although Python permits the use of the %d, %f and %s operators, a more Pythonic approach is permitted. However, this approach is unique to Python and may not work with other programming languages. It is important to be familiar with this approach so that we can read and interpret the code written by other programmers. Modify your code such that it matches Listing 5.11. In this approach a number of integers ($\{0\}$, $\{1\}$) are used as place holders where the variables will be inserted (positional arguments); these are to be inserted within curly braces. Where modifiers are required, a colon is used after the integer as shown in the example.

```
1 sub_string = ``height''
2 my_height=180.123456789
3
4 print(``My {0} is {1:2.8f}''.format(sub_string, my_height))
```

Listing 5.11: Pythonic approach — positional arguments

Perhaps a more convenient approach is to assign a placeholder label where information will be inserted

```
1 name = ``Robert''
2 age = 21
3
4 print(``My name is {n}, I am {a}''.format(n=name, a=age))
```

Listing 5.12: Pythonic approach — named arguments

In fact, there is an even more convenient syntax for string interpolation that has been available since Python 3.6 (check your version), as described in PEP498 (look this up!). This will be covered in Section 5.4.

5.2 Keyboard Input

The *input()* function can be used to read a sequence of alphanumeric characters in from the keyboard. For example the code snippet in Listing 5.13 prompts the user to enter their name then prints hello followed by the name.

```
1 print(``Enter your name'')
2 name = input()
3 print(``hello'', name)
```

Listing 5.13: Basic input

Python and many other languages permits the prompt to be included diectly within the *input()* function, as shown in Listing 5.14. This is perhaps a more convenient and efficient way to provide a user with a prompt for input. Take special note of the inclusion of the control character, \n, what would be the effect of omitting it?

```
1 name = input(``Enter your name\n'')
2 print(``hello'', name)
```

Listing 5.14: Input with message

As has been mentioned already, *the input()* function returns a *string* type; therefore, if some form of numerical input is required then the *string* must be casted to a *float* or an *int*. In Section 2.1 the *type()* function was used to print a data type to the screen (see Listing 2.2). Another useful function is *isinstance()* which permits the programmer to interrogate a variable to determine its type. Consider the code fragment in Listing 5.15; the user is prompted to enter an integer and the result stored in variable *ip*. Line 2 of the code checks to determine if variable *ip* is a string (type str), note the syntax of this function: it takes parameters the variable in question and a type name. The result of the query is printed to the screen. Line 3 performs a query to check if variable *ip* is an integer. Run this code and enter an integer when requested; run it again and this time enter your name instead. Note the output, it should be apparent that the input is always a string regardless of the characters entered.

```
1 ip = input(''Enter an integer: '')
2 print(''A string?: '', isinstance(ip, str))
3 print(''An int?: '', isinstance(ip, int))
```

Listing 5.15: Type determination

The fact that *input()* always returns a string type poses a potential problem for the programmer. Consider the trivial example where the user is prompted to enter two integers, the computer then calculates the sum of them and displays the result on the screen. Listing 5.16 contains code that the novice programmer might expect to accomplishes that goal. Type in and test this code, does it produce the correct answer to integer inputs?

```
1 # Get two ints from the keyboard
2 first = input(''Enter an integer: '')
3 second = input(''Enter another integer: '')
4 result = first + second
5 print(''Sum = '', result)
```

Listing 5.16: Flawed addition

It should be apparent that the code in Listing 5.16 does NOT produce the summation of two integer inputs because the return type of the *input()* function is a string, and consequently the '+' operator performs concatenation. In order to perform addition, the data type of the inputs must be ints. Modify the code in Listing 5.16 such that it performs the type conversion as shown in Listing 5.17.

```
1 # Get two ints from the keyboard
2 first = int(input(''Enter an integer: ''))
3 second = int(input(''Enter another integer: ''))
4 result = first + second
```

```
5 print(''Sum = '', result)
```

Listing 5.17: Integer addition

Test this code to confirm that it will indeed add two integers entered from the keyboard. Now, run the program again and instead of entering integers try entering a real number or a string of text. This should generate an error. Importantly, this error occurs despite the code being syntactically correct, after all it can successfully execute if two integers are entered. This type of error is known as an **exception**. By examining the error message it should be clear that a *ValueError* has occurred; this is a result of the failed conversion of one date type to another (i.e. a failure to convert alphabetical characters to an integer value).

Good, well-constructed computer programs should be designed to deal with exceptions. Fortunately, modern computer programming languages, including Python, provide a mechanism to deal with them. In Python **exception handling** is implemented using the *try* and *except* statements. The use of these statements can be best understood by illustration; examine the code fragment in Listing 5.18. An infinite *while* loop is defined (lines 2 — 9) that can only be broken once the *break* statement (line 6) is executed. Within the loop there are two blocks of code, and these blocks are executed as follows. The Python interpreter attempts (tries) to execute the first block (lines 4 — 6), if NO error occurs then it skips the second block (the except block) and continues on (lines 11 & 12). In this case the break statement would cause the program to leave the loop. If however, an error of some description occurs within the first block, execution of this block is halted and the second block of code (lines 8 & 9) is executed. In fact is is possible to have several *except* blocks, one for each error type. This except block is executed because the interpreter would have returned a ValueError as noted before (check line 7).

```
1  # Stay within loop until integer is entered
2  while True:
3      try:
4          # Block of code defined by indentation
5          ip = int(input(''Enter an integer: ''))
6          break
7      except ValueError:
8          # Block to execute on error
9          print(''This is not an integer, try again'')
10
11 print(''A string?: '', isinstance(ip, str))
12 print(''An int?: '', isinstance(ip, int))
```

Listing 5.18: Syntax of exception handling

Thus, if we wish to write robust code that will add two integers entered from the keyboard, we should write some code which closely resembles that in Listing 5.19. Type in and test this code with a variety of inputs from the keyboard. Pay special attention to the location of the colons in the code.

```
1 # Stay within loop until first integer is entered
2 while True:
```

```
    try:
        # Block of code
        first = int(input(''Enter an integer: ''))
        break
    except ValueError:
        # Block to execute on error
        print(''Try again'')

# Stay within loop until second integer is entered
while True:
    try:
        # Block of code
        second = int(input(''Enter an integer: ''))
        break
    except ValueError:
        # Block to execute on error
        print(''Try again'')

result = first + second
print(''The summation is: '', result)
```

Listing 5.19: Integer input

Now that we have covered input from the keyboard, we shall consider input and output from files.

5.3 Reading And Writing To Files

Many computer programs process data stored in files (word processors, spreadsheets, media players etc.). To demonstrate how to read data from files into a Python program, first save the file *shakespeare.txt*, available on myplace, in your Python folder. Enter and run the code fragment shown in Listing 5.20.

```
my_file = open(''shakespeare.txt'', ''r'')

my_string = my_file.read()

print(my_string)

my_file.close()
```

Listing 5.20: Accessing a file

The first line opens a file named *shakespeare.txt*; note, the use of **"r"** which signifies that it is possible to read from the file (i.e. read-only mode). To write to a file rather than read from it, you can use **"w"** instead (write-only mode). Previously, this course has considered some built-in primitive data types (such

as *floats* and *ints*), *myFile* is a more complex data type known as an *object*. Objects will be covered later in this course; however, for now it is sufficient to regard an object as a special variable which also includes functionality. Line 3 executes an internal function called *read()* which, as the name suggests, reads in the contents of the file and stores the result in a string. In line 5 the string, and hence the contents of the file, is printed to the screen. Finally, line 7 closes the file after use.

Let us now examine the syntax of how to write data to a file. Enter and run the code in Listing 5.21. This code opens a file named written.txt in write mode, and writes the words hello world to that file. Finally, and importantly the file is closed. Experiment what will happen if the programmer forgets to close the file.

```
1  my_file = open(''written.txt'', ''w'')
2
3  my_file.write(''hello world'')
4
5  my_file.close()
```

Listing 5.21: Writing to a file

It is possible to open a file in a range of access modes as specified in the *open()* function, specimen values are shown in Table 5.1.

Mode	Description
r	Read-only
w	Write-only. If file already exists, overwrite it
a	Append. If file already exists, write to the end
r+	Read and write permitted. Overwrite contents from beginning
w+	Read and write permitted. Overwrite existing file
a+	Read and write permitted. Write to the end of the file

Table 5.1: File access modes

Using a text editor, create your own text file containing some arbitrary text. Experiment writing to that file using the w, w+, a and a+ modes. Make a written note of the results for future reference.

5.4 Advanced Material

5.4.1 Formatted String Literals

In Section 5.1.2 we covered string interpolation using named arguments in conjunction with *format()*. Although this represented an improvement over the old C-style syntax, it is still quite clunky. In order to address this shortcoming, Python 3.6 introduced a more convenient syntax that is increasingly being used. If your version of Python is above 3.6, it is recommended that you use this approach. To understand the

latest approach, examine the code fragment in Listing 5.22 , which is a modification of the example code fragment used earlier. Notice how *format()* is used to map placeholder labels to variable names.

```
name = ``Robert''
age = 21

op_str = ``My name is {n}, I am {a}''.format(n=name, a=age)

print(op_str)
```

Listing 5.22: Pythonic approach — named arguments

A more convenient approach is provided by formatted string literals, more commonly referred to as f-strings. This provides a simple, yet somewhat unusal syntax. With is approach, we prefix the string literal (i.e. the open quotes) with the letter *f*. Within the string itself we simply put variable names within the curly braces; there is no need for the *format()* function to perform a mapping/

```
name = ``Robert''
age = 21

op_str = f``My name is {name}, I am {age}''

print(op_str)
```

Listing 5.23: F-string approach

5.5 Summary

Based on the material covered in this section a clearer understanding of input and output should have been achieved. It is recommended that students look for alternative sources of information on the Web in order to gain a fuller understanding of the key concepts presented here. Please note, Python 3 differs from Python 2 on issues pertaining to input and output.

Hopefully, you will have learned:

1. The print function can be used both with variables and literals.
2. When displaying text on the screen (or in a file for that matter), special control characters can be used to format that text.
3. It is possible to write to a file in a similar manner as writing to the screen.
4. It is possible to read from a file in a similar manner as reading from the keyboard.
5. When accessing files, interaction is governed by a mode such as read-only or write-only, etc.

5.6 Problem Sets

The following are a set of problems designed to increase your programming skills, but also to increase your level of initiative and problem solving ability. Thus, you may be required to search for other sources of information on the Web before the problems can be solved.

1. Write some code which writes your surname, date of birth, and registration number to file. Have appropriate headings above each of these pieces of information. Use tabs and newline characters to format the output. The format should be of the following:

Surname	Date of Birth	Registration
Smith	12/12/1980	201411223344

2. Write a second Python program which adds the details of a second fictitious student to the text file created in Q1. You may need to experiment with different access modes. Research how to accomplish this task using the Internet.

Surname	Date of Birth	Registration
Smith	12/12/1980	201411223344
Jones	01/10/1985	201933221144

Chapter 6

Strings

Thus far, four primative built-in data types have been introduced. Integers and floating point numbers have been used to store numerical information and perform arithmetical operations, boolean variables have been used to perform logical operations like store the output of a comparison, and strings have been used to hold textual (alphanumeric) information. In this section, the concept of **string manipulation** will be explored; this will equip the programmer to perform a range of useful operations on the data stored in strings.

6.1 Encodings

Strings can be viewed as an ordered sequence of characters, conveniently they can be used to hold natural language expressions such as sentences and names, etc. Take for example, the string "hello world!" as depicted in Figure 6.1, it contains letters, a space and and exclamation mark. The first character of this string is "h", and is regarded as being character 0. Each character can hold a letter (upper or lower case, e.g. "A" or "a"), a single digit (e.g. "1" or "2"), punctuation symbols (e.g. ";" or "@" or "!"), and even some nonprinting characters such as tab, newline, carriage return etc. It should be remembered that computers can only store and manipulate binary numbers; these numbers can have different representations: decimal, hexadecimal and so forth. In order to permit text to be stored and manipulated, mappings are used to associate each permissible character with a number. Historically, the American Standard Code for Information Interchange (ASCII) was used by computers to map each letter of the Roman alphabet to a 7-bit number, specimen codings are shown in Table 6.1. This was later expanded to 8 bits to permit other symbols to be stored. In order to enable international encodings, e.g. Russian, Arabic, Cyrillic alphabets, a backwards compatible superset of encondings has been devised: UTF-8. By way of example, a space is encoded as decimal 32, and the exclamation mark ("!") as 33, the upper case "A" is 65, and the lower case "a" is 97. A table of UTF-8 mappings can be found on the Internet, look this up.

Code	Char	Code	Char
9	Tab	65	A
32	Space	90	Z
33	!	91	[
48	0	93	]
59	9	97	a
61	=	122	z
62	>	123	{
63	?	125	}

Table 6.1: Sample ASCII codes

Figure 6.1: An example string

In Python, the *ord()* function returns the integer mapping of each character. Enter the code shown in Listing 6.1, and check the output produced with the table of UTF-8 encodings. Change the value of the single character string.

```
# Define a single character string
c = 'a'

# Display result on screen
print(ord(c))
```

Listing 6.1: String encoding

Now that the underlying structure of a string has been covered, let us look at some operations.

6.2 Simple Operations

When data is stored in a string, we can perform a wide range of operations on it. Perhaps we wish to find the location of a word in a sentence (searching), or replace one word in a sentence with another (substitution). In fact these are routine operations that are built into almost all word processor packages. Another frequently used operation is to re-arrange the order of the characters in a string, i.e. arrange the letters in alphabetical order. These operations represent the fundamentals of string manipulation.

Let us begin by reviewing concatenation using the '+' operator; type in an run the code in Listing 6.2. Hopefully, this demonstrates that a third string has been formed by adding together the first two.

```
1 # Define two strings
2 a = ''hello''
3 b = ''world''
4 # Concatenate strings
5 c = a + b
6 # Display result on screen
7 print(c)
```

Listing 6.2: String concatenation

Recall, a string can be viewed as a sequence of characters. When dealing with strings it is often useful to be able to count the number of characters; this can be achieved using the *len()* function which returns the length of the string. Modify Listing 6.2 such that it also prints the length of the resultant string as shown in Listing 6.3. The *len()* function can be particularly useful for iterating through a string using a *for* loop to achieve some goal, as will be described in Section 6.4.

```
1 # Define two strings
2 a = ''hello''
3 b = ''world''
4 # Concatenate strings
5 c = a + b
6 # Display result on screen
7 print(c)
8 print(''string length is '', len(c))
```

Listing 6.3: *len()* function

6.3 String Slicing

The characters within a string can be referenced individually using square brackets; this is known as string slicing. Recall, the first character is at index location 0 not 1. This approach to reading the individual elements (characters) within a string (using []) is common to many programming languages. Modily line 7 of Listing 6.3 to read *print(c[1])*, change the value in the square brackets and note the results.

Python supports accessing a sequence of elements using the same notation as used with the *range()* function covered Section 4.2. We can use $[start : stop]$ to produce the sequence of of characters from positions $start$ to $stop - 1$, and an optional third parameters $step$ can be used to access every $step^{th}$ element in the sequence. Enter the code in Listing 6.4 and experiment with different index values. What happens if you use an index value that is greater than the length of the string?

```
1 # Define a simple string
2 s = ``hello world!''
3 print(s[0:10:2])
```

Listing 6.4: Indexing a string

As mentioned, it is often useful to determine the length of a string prior to performing some operation on it without inducing an error. With that in mind, the code in Listing 6.5 is preferable when processing the whole string.

```
1 # Define a simple string
2 s = ``hello world!''
3 print(s[0:len(s):2])
```

Listing 6.5: Indexing a string with *len()*

Given that it is possible to read the value of a string element using the square bracket notation, this begs the question of whether it is possible to modify a string element. Type in and run the code in Listing 6.6 and take note of the result.

```
1 # Define a simple string
2 s = ``hello world!''
3 print(s[2])
4 s[2] = "p"
5 print(s[2])
```

Listing 6.6: Immutable string

The code should print the third element of the string ('l') to the python shell then generate an error on line 4. The reason for this is not sytax related. In Python, unlike many other programming languages, a string is *immutable*. An immutable variable is one that cannot be changed after is has been defined, note this does not mean that the string cannot be over-written with a new value. Type the code in Listing 6.7 and verify this.

```
1 # Define two strings
2 s = ``hello world!''
3 s = s + " bye bye world!"
4 print(s)
```

Listing 6.7: Over-writing a string

Another way to individually reference the elements of a string is to use a loop, as described in Section 6.4.

6.4 String Iteration

It is also possible to iterate through the elements in a string using a *for* or *while* loop; this is often useful for performing more complex operations on them. In the example in Listing 6.8, a *for* loop is used to iterate through the string and print each character individually. Try this code and observe the results.

```
1 # Define a string
2 my_string = ``This is a string of characters''
3 length = len(my_string)
4 # Iterate through string printing
5 # each character separately
6 for index in range(length):
7     print(my_string[index])
```

Listing 6.8: Iterating through a string

Based on the code in Listing 6.8, could you utilise the *ord()* function and knowlege of character encoding to print capitalised versions of each chatacter to the screen?

6.5 Search And Replace

Sometimes it is useful to check whether a string contains a particular character (or sequence of characters, i.e. a particular word). To put this more formally, we wish to determine if a string contains a particular **substring**. Most modern programming languages provide the functionality to do this and Python is no different, in fact Python has a particularly convenient syntax for this, and uses the membership operator *in*. Listing 6.9 defines a string (‘The quick brown fox jumps over the lazy dog’), and a search term (‘fox’). Line 6 is used to determine if the original string contains a substring that matches the search term, note the syntax when using the *in* operator. If the substring exists then the value True is returned, otherwise the value False is returned. Experiment with this code by changing the search term.

```
1 # Define a string
2 my_string = ``The quick brown fox jumps over the lazy dog''
3 # Define a search term
4 search_term = ``fox''
5 # Does a substring match the search term?
6 result = search_term in my_string
7 # Print to screen result of search
8 print(``Substring found?: '', result)
```

Listing 6.9: Searching for a substring

Anyone who has used a word processing application will appreciate the convenience of being able to search for and replace a given word or phrase. Once again this is a fairly common task in computer programming; therefore, most languages have built-in functionality to achieve this goal. In Python, a function *replace()* is used as shown in Listing 6.10. Type in a run this code; experiment using different search terms, what happens if you search for a substring that does not exist in the original string?

```
1 # Define a string
2 my_string = ``The quick brown fox jumps over the lazy dog''
3 # Define a search term to be replaced
4 search_term = ``fox''
5 # Define the new sequence of characters
6 new_term = ``cat''
7 # Create a new string post substitution
8 new_string = my_string.replace(search_term, new_term)
9 print(new_string)
```

Listing 6.10: Replacing a substring

Note, that line 8 of Listing 6.10 requires a new string variable to be created, this is because strings are immutable!

6.6 Summary

1. In Python a string is a sequence of characters.
2. Each character is mapped to an integer: ASCII/UTF-8.
3. The characters can be letters or numbers.
4. Each character can be accessed individually using [] and an appropriate index.
5. There is a significant amount of functionality to support string manipulation.

6.7 Problem Sets

Before you begin these exercises do some research on your own by using the Python reference manual: find the documentation for string manipulation for *Python 3*. It should be found in Section 4.6.1 of this web page. Examine the various functions such as *replace()*. One of the goals of these exercises is to learn how to use the reference manual. Consulting the manual is a necessary and frequent aspect of professional

computer programming. It is recommended that you avoid looking for code snippets on the Internet as this will rob you of the opportunity to read and understand the manual, but also of the opportunity to solve these problems.

1. Write a short program that prompts the user to enter a sentence from the keyboard. The sentence is then printed to the screen with each letter capitalised. Example: an input string of 'The quick brown fox jumps over the lazy dog' is modified to read 'THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG'.

2. Write a short program that prompts the user to enter a sentence all in lower case. The sentence is printed to the screen with the first letter capitalised. Example: an input string of 'the quick brown fox jumps over the lazy dog' is modified to read 'The quick brown fox jumps over the lazy dog'.

3. Write a short program that prompts the user to enter a sentence all in lower case. The sentence is printed to the screen with the first word capitalised. Example: an input string of 'the quick brown fox jumps over the lazy dog' is modified to read 'THE quick brown fox jumps over the lazy dog'.

4. Write a short program that prompts the user to enter a sentence. The sentence is then printed to the screen in reverse order. Use a loop, rather than string splicing, to achieve this task. Example: an input string of 'The quick brown fox jumps over the lazy dog' is modified to read 'god yzal eht revo spmuj xof nworb kciuq ehT'.

Chapter 7

Data Structures

As you may recall, we have used the analogy that computer programming is like a recipe; the algorithms are the methods and variables (data types) are the ingredients. Thus far we have looked at some of the basic components of algorithms: decisions and repetition. We have also utilised some basic ingredients: integers, floating point numbers, boolean numbers, and strings. Now we shall turn our attention to more powerful structures that can be used to store and manipulate large amounts of data. These data structures can be thought of as **container objects**, in Python they are known as **collections**.

7.1 Arrays

By way of gentle introduction consider an array (sometimes referred to as a vector in mathematics). An example of an array (column vector) is given in Equation 7.1. From a computer programming perspective the array can be viewed as a container holding three elements: x_1, x_2, and x_3.

$$\mathbf{X} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \tag{7.1}$$

Listing 7.1 defines an array of five integers (1, 9, 6, 2, and 0), multiplies them by 2 and displays the result on screen. Pay special attention to line 2 which imports a special numerical processing library called NumPy; the syntax of this line is not important at this stage. The NumPy library contains the definition of a new data type called an array, line 5 demonstrates the syntax used to define an array in Python. Test this code and verify the result. What would happen if one of the elements were changed to become a floating point number?

```
1 # Import the Python numerical library
2 from numpy import *
3
```

```
4  # Define an array with 3 elements
5  x = array([1 , 9 , 6, 2, 0])
6
7  # Manipulate array
8  y = x * 2
9
10 # Print result
11 print(y)
```

Listing 7.1: A simple array of integers

Arrays and matrices are particularly useful for performing a wide variety of linear algebraic tasks such as solving systems of equations, and many programming languages make wide use of them. However, there use is much broader. Arrays can be used to hold strings or indeed any other data type. As a container object, arrays are favoured for the speed at which they can be iterated through, perhaps using a *for* or *while* loop. These data structures are particularly quick due to the fact that all elements are stored sequentialy in contiguous memory, as shown in Figure 7.1.

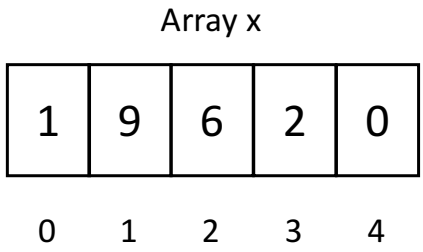


Figure 7.1: An example array

The disadvantage of using an array is that its size is fixed when it is defined. Moreover, it is not possible to add elements to the beginning, end or middle of the array as this would infer a change in size. Python has another powerful data structure known as a **list** that is useful for performing nonmathematical operations on a sequence of data.

7.2 Lists

A list is a collection of data items; these items can be mixed: integers, and strings etc. Listing 7.2 contains a list of forenames; enter this code while paying special attention to the syntax. Line 4 of the code makes use of the *len()* function, which has already been used to determine the number of characters in a string. In this instance it can be used to determine the number of items in the list. In fact, the *len()* function can be applied to any container object. Line 6 prints the contents of the container object (all items) to the screen. Finally, note that the elements of the list can be accessed individually (line 8); recall the first item in a sequence is at index position 0.

```
1 # Define a list of names
2 names = ['jack', 'jill', 'john', 'james']
3 # Calc number of items in list
4 num = len(names)
5
6 print(''Names list = '', names)
7 print(''Number of items: '', num)
8 print(''Item 2 is '', names[2])
```

Listing 7.2: List of forenames

In terms of implementation a list (more formally a linked-list) enables items stored in various locations around the computer memory to be ordered into a chain of items using pointers. A pointer is a special varaible whose value is a memory address. A node is defined which contains three pointers: one points to the memory address of the node representing the previous element in the chain, another points to the node of the next element in the chain, and a third points to the actual element it represents, as shown in Figure 7.2.

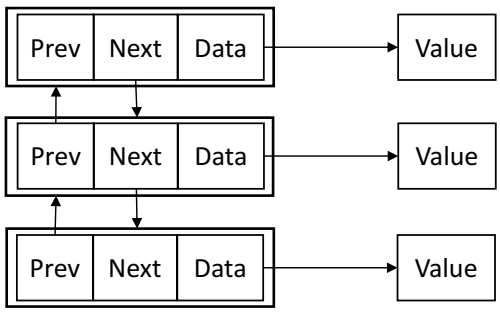


Figure 7.2: Linked-list nodes

There are advantages and disadvantages associated with a list. The advantages are that it can vary in size; therefore, it is possible to add elements to the beginning, middle, or end. Similarly elements can be deleted. All this can be acheived by simply addjusting the values of the pointers. The disadvantage is that to get to the next element in the list, we must first access the memory location of the current element's node, access the next pointer of the node, jump the the node at that memory address, and access the value pointer of that node. This is very slow in comparison to an array because of the number of operations that must be performed..

7.2.1 Iteration

By accessing the elements individually, it is possible to iterate through a list and perform some action on those element. Listing 7.3 iterates through a list using a *for* loop and prints each individual item to the

screen. The variable *i* is used as a counter to index each element of the list successively.

```
1 # Define a list of names
2 names = ['jack', 'jill', 'john', 'james']
3 # Calc number of items in list
4 num = len(names)
5 # Iterate through list
6 for i in range(num):
7     print(names[i])
```

Listing 7.3: Numerical iteration through a list

The *for* loop can be used in another way in Python to access the members of a list more intuitively, as shown in Listing 7.4. Whereas the variable *i* in Listing 7.3 took on a numerical value (an integer) on each iteration of the loop, in this example the variable *name* takes on the value of each successive element in the list (a string). This simple example demonstrates the tremendous power of the *for* loop in Python.

```
1 # Define a list of names
2 names = ['jack', 'jill', 'john', 'james']
3
4 # Iterate through list
5 for name in names:
6     print(name)
```

Listing 7.4: Intelligent list iteration

7.2.2 Modification

Just as indexing can be used to read the elements in a list, it can also be used to modify elements. In this respect, list are said to be **mutable**. The program in Listing 7.5 provides such an example of element modification. Try this code, what happens if you try to overwrite a nonexistent element, e.g. element 4?

```
1 # Define a list of names
2 names = ['jack', 'jill', 'john', 'james']
3 # Overwrite element 2
4 names[2] = ``jim''
5 print(names)
```

Listing 7.5: Element modification

As the example in Listing 7.5 demonstrates the index notation can be used to modify existing elements but it cannot be used to add further elements. Fortunately, Python provides a useful function, *append()*, to permit new elements to be added to a list. Listing 7.6 demonstrates the syntax of the *append()* function that can be used to add elements to the end of a list.

```
1 # Define a list of names
2 names = ['jack', 'jill', 'john', 'james']
3 # Add an element to the end
4 names.append(``jim'')
5 print(names)
```

Listing 7.6: Append element

The limitation of the append function is that it only permits elements to be added to the end of a list. There are many situations where it would be beneficial to add elements elsewhere in the list. In Python this is accomplished using the *insert()* function. This function takes two parameters, as demonstrated in Listing 7.7; this first is the index location of the new element and the second is the value of the new element.

```
1 # Define a list of names
2 names = ['jack', 'jill', 'john', 'james']
3 # Add an element to the middle
4 names.insert(2 , ``jim'')
5 print(names)
```

Listing 7.7: Add element to the middle

A complementary function for deleting members of a list is *pop()*. When used without any parameters is removes the last item on the list. Alternatively, it can be used with an index to specify the element to be used. Type in and test the code in Listing 7.8.

```
1 # Define a list of names
2 names = ['jack', 'jill', 'john', 'james']
3 # Remove element 2
4 names.pop(2)
5 print(names)
```

Listing 7.8: Delete an item

Of course, the *pop()* function requires prior knowledge of the location of the item to be removed, and that is not always available. The *remove()* function can be used to remove the first occurrence of an item, that this does not suggest that all occurrences of an item will be removed. Listing 7.9 provides an example of how to remove an item. Experiment with this code.

```
1 # Define a list of names with two jills
2 names = ['jack', 'jill', 'john', 'jill' , 'james']
3 # Remove element ``jill''
4 names.remove(``jill'')
5 print(names)
```

Listing 7.9: Remove a specified item

Yet another useful function is *sort()* which re-arranges the list in order. For lists of strings this means alphabetical order. Run the code in Listing 7.10 and check the result. What would happen if the strings in the list were replaced by floats or ints?

```
1 # Define a list of names
2 names = ['jack', 'jill', 'john', 'james']
3 # Arrange list alphabetically
4 names.sort()
5 print(names)
```

Listing 7.10: Alphabetically sort the list

7.2.3 Lists and strings

On the subject of sorting data structures, it is also possible to sort a string. Listing 7.11 provides an example using the built-in function, *sorted()*. Type in and run this code. The first two characters of the sorted string should be a space and exclamation mark respectively, followed by the letters of "hello world!" in alphabetical order. In fact the order of the characters is determined by their ASCII/UTF-8 integer representation. Take note that the function *sorted()* takes a string as a parameter but the return type is also a string (see the output of line 6). In fact is true for a number of built-in functions and string functions.

```
1 # Define a string
2 my_string = "hello world!"
3 new_string = sorted(my_string)
4 # Display output and type
5 print(new_string)
6 print(type(new_string))
```

Listing 7.11: List as a return type

A useful function that can be used to transform a string to a list is *split()*. This function identifies items within a string using the whitespace between characters. Each of the items becomes an element within a list. This can be useful for natural language processing where each item in the string represents a word, by

using the *split()* function a list of words can be created for further processing. Type in and run the code in Listing 7.12.

```
1 # Define a string
2 my_string = ``the quick brown fox jumps over the lazy dog''
3 # Convert to a list of strings
4 my_list = my_string.split()
5 # Display list
6 print(my_list)
```

Listing 7.12: Convert a string to a list

A separator character can be used by the *split()* function to demark list items. Run the code in Listing 7.13 and examine the result. In this case a semicolon is used as a separator (line 4); however any character or sequence of characters can be used. Change the semicolon in line 4 to "fox" and rerun the program.

```
1 # Define a string
2 my_string = ``the quick brown fox ; jumps over the lazy dog''
3 # Convert to a list of elements using the semicolon
4 my_list = my_string.split(``;'')
5 # Display list
6 print(my_list)
```

Listing 7.13: Using a separator

In Listing 7.13 a string is processed and the result is a list. The *join()* function operates on a list of elements and produces a string as a result. This function provides a useful mechanism to convert a list of elements to Comma Separated Value (CSV) format as supported by many spreadsheets. Run the code in Listing 7.14 and experiment with different symbols with which to join the elements in the list.

```
1 # Define a list of words
2 my_list = [``the'', ``quick'', ``brown'', ``fox'']
3 # Define a symbol to join words
4 symbol = ``,''
5 result = symbol.join(my_list)
6 # Display list
7 print(result)
8 print(type(result))
```

Listing 7.14: Joining list elements

7.3 Maps

Another data structure that is often used in computation is a **map**. A map is sometimes known as an associative array; in Python it is known as a **dictionary**. Whereas lists use a numerical index value to access elements, maps (dictionaries) use what is known as a **key**. Every element in the map is a key-value pair, the key can be used to retrieve the value of the element. List and array elements can be accessed via an index because those data structures represent conceptual sequences of elements. A map does not hold a sequence of elements, rather is holds a collection of elements, i.e. it is unordered. The word 'map' may conjure up images of an atlas; however, this would be an misleading analogy; the data structure is known as a map because a mapping exists between each element's key and its value.

Perhaps a map is best illustrated by way of example to clarify its potential applications and also the appropriate syntax. Suppose a programmer is required to write an address book application. Such an application requires an association (or mapping) between each contact and the respective telephone extension number. Listing 7.15 contains an example of such an application. Type in and run this program. Note the colon used to specify the mapping between the key (in this case the name) and the element's value (telephone extension number). A comma is used to separate the key-value pairs.

```
1 # Define a map/dictionary of tel ext numbers
2 contact_book={''robert'': 2879, ''martin'':2471, ''david'': 2717}
3 # Print martin's ext number
4 print(contact_book[''martin''])
```

Listing 7.15: A simple map

In common with other container objects it is possible to add and remove elements. Line 4 of Listing 7.16 demonstrates the syntax used to add an element to a map. Since a map is unordered there is no concept of adding the new element at the middle or at the end, location in this context does not exist.

```
1 # Define a map/dictionary of tel ext numbers
2 contact_book={''robert'': 2879, ''martin'':2471, ''david'': 2717}
3 # Add an item
4 contact_book[''james''] = 4072
5 print(contact_book)
```

Listing 7.16: Adding to a map

To remove an element from the map, the element's key is required as would be expected. Listing 7.17 demonstrates the syntax to remove an element, in this case the *martin* entry.

```
1 # Define a map/dictionary of tel ext numbers
2 contact_book={''robert'': 2879, ''martin'':2471, ''david'': 2717}
3 # Remove an item (martin)
```

```
4 del contact_book[``martin'']
5 print(contact_book)
```

Listing 7.17: Deletion from a map

To modify the value associated with a key, we use the same syntax used when adding to an map. The example in Listing 7.18 modifies the number associated with "martin".

```
1 # Define a map/dictionary of tel ext numbers
2 contact_book={``robert'': 2879, ``martin'':2471, ``david'': 2717}
3 # Modify an item
4 contact_book[``martin''] = 1111
5 print(contact_book)
```

Listing 7.18: Modifying a value

As mentioned the *len()* function can be used on a variety of container objects; modify the code in Listing 7.18 to display the number of elements in the map.

Often it is useful to check the membership of a map, e.g. perhaps the programmer wishes to know if the *contactBook* contains an entry for "robert". This can be achieved using the **in** membership operator as shown in Listing 7.19. Hopefully, you will appreciate the simplicity of this syntax. Take special care when using this functionality: it is possible to search for the presence of a key, but not for the presence of an associated value.

```
1 # Define a map/dictionary of tel ext numbers
2 contact_book={"robert": 2879, "martin":2471, "david": 2717}
3 # check membership
4 exists = "robert" in contact_book # returns a bool
5 print("In contact_book? ", exists)
```

Listing 7.19: Membership test

Although a map is not an ordered container it is still possible to iterate through it using a *for* loop in a similar manner to that adopted for list in Listing 7.4. Attempt this on your own and note whether the order in which the values is listed is the same in which they are defined.

The final data structure we shall consider is the set as discussed in Section 7.4.

7.4 Sets

A **set** in computer programming is largely analogous to a set in mathematics: it is an unordered list of elements. A key characteristic of a set is that it can contain only one of a particular element, i.e. $S = \{2, 1, 2, 4, 5\} = \{1, 2, 4, 5\}$. Note, that the element 2 need exist only once in S. The operations performed on a set are often related to determining membership of that set; the *in* membership operator can be very useful in this regard. Sets are also useful in the mathematical sense, a brief review of some mathematical concepts as applied to sets and associated code is covered in Sections 7.4.1 to 7.4.3.

7.4.1 Union

Suppose we have a set, A, which contains all the integers between 5 and 7 inclusive, i.e. $A = \{5, 6, 7\}$. Further, suppose a set, B, contains all the integers between 10 and 12 inclusive, i.e. $B = \{10, 11, 12\}$. This can be represented by a Venn diagram as shown in Figure 7.3. The union of A and B, denoted $A \cup B$, is $\{5, 6, 7, 10, 11, 12\}$, as shown by Figure 7.4.

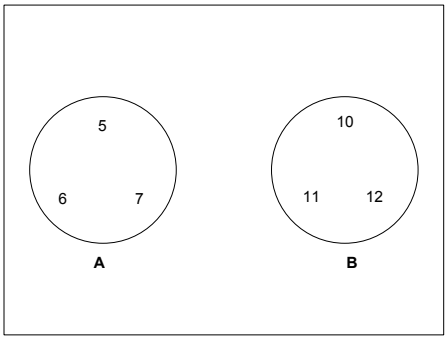


Figure 7.3: Venn diagram of sets A and B

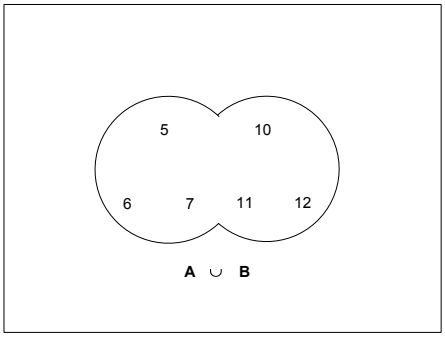


Figure 7.4: Venn diagram of the union of A and B

Let us take a second example. $C = \{1, 2, 3, 4\}$ and $D = \{3, 4, 5\}$; therefore, $C \cup D = \{1, 2, 3, 4, 5\}$. Note that though the elements 3 and 4 are in both sets only one element of each type is permitted to be a member of a set.

Thus, the union is a set which contains all the elements of both sets. Another way to state this is that the union of two sets, A and B, contains every element that exists in set A **or** set B.

The program in Listing 7.20 demonstrates how to define a set (lines 2 & 3). Examination of the syntax would suggest that a set as implemented in Python is a wrapper around a list that provides some specialist functionality. Line 5 performs the operation $A \cup B$, modify the code to perform the operation $B \cup A$ and compare the results. Further modify the code to compute the union of sets C and D, above.

```
1 # Create two sets A and B
2 a = set([5, 6, 7])
3 b = set([10, 11, 12])
4 # Get the union of A and B
5 result = a.union(b)
6 print(result)
```

Listing 7.20: Union of two sets

7.4.2 Intersection

The intersection of two sets is the set of elements that is common to both sets. For example, if we define sets $C = \{1, 2, 3, 4\}$ and $E = \{3, 4, 5\}$ as shown in Figure 7.5, then the intersection is given by $C \cap E = \{3, 4\}$. The intersection of these two sets is represented by the Venn diagram in Figure 7.6 . If there are no common elements then the result is the empty set, as in the example where $A = \{5, 6, 7\}$ and $B = \{10, 11, 12\}$, then $A \cap B = \{\} = \emptyset$.

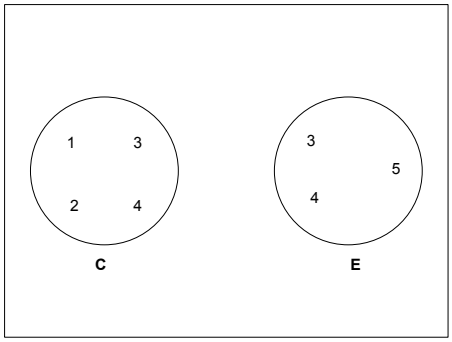


Figure 7.5: Venn diagram of sets C and E

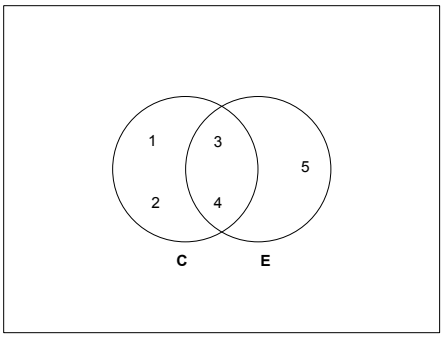


Figure 7.6: Venn diagram of the intersection of C and E

Thus, the intersection is a set which contains only the elements which are members of both sets. Another way to state this is that the intersection of two sets, A and B, contains every element that exists both in set A **and** set B.

The code in Listing 7.21 demonstrates the syntax to calculate the intersection of sets C and E. Modify this code to compute the intersection of E and C and compare the result. Further modify the code to compute the intersection of sets A and B.

```
# Create two sets C and E
c = set([1, 2, 3, 4])
e = set([3, 4, 5])
# Get the intersection of C and E
result = c.intersection(e)
print(result)
```

Listing 7.21: Intersection of two sets

7.4.3 Subsets and Supersets

The term subset describes the *relationship* between two sets. Set X is a subset of set Y if each and every element contained in X is also contained in Y. For example, if $A = \{5, 6, 7\}$ and $G = \{3, 4, 5, 6, 7, 8\}$ then we can state that A is a subset of G; the mathematical notation for this is $A \subset G$. This relationship can be represented by the Venn diagram shown in Figure 7.7. Based on this definition it is clear that G can have many different subsets including $\{3, 4, 5\}$, $\{6, 7\}$, and $\{3, 4, 7\}$. It should also be clear that the set $H = \{9, 10, 11\}$ is not a subset of G, or expressed mathematically $H \not\subset G$.

A superset describes the inverse relationship between two sets, i.e. Y is a superset of set X, if X is a subset of Y: $X \supset Y$ if $Y \subset X$.

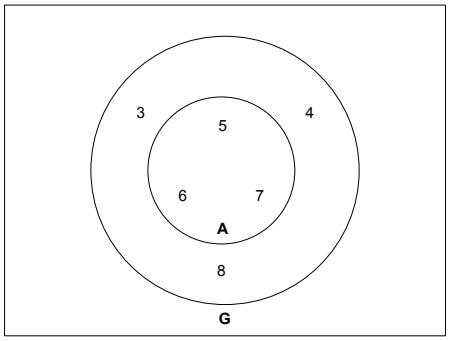


Figure 7.7: Venn diagram of G and its subset A

The code in Listing 7.22 demonstrates the syntax for testing if set A is a subset of G. Type in and run this code. Note that the result is not another set but a boolean value. This suggests that this function *issubset()* performs a membership test. Modify the code to test if set G is a subset of A. To test if one set is the superset of another the equivalent function is *issuperset()*; modify the code further to determine if G is a superset of A and vice versa.

```
# Create two sets A and G
a = set([5, 6, 7])
g = set([3, 4, 5, 6, 7, 8])
# Is A a subset of G
result = a.issubset(g)
print(result)
```

Listing 7.22: Integer type

7.5 Advanced Material

Where applicable, each lesson will be accompanied with some advanced material that is not examinable and may be of interest to those who can already program in Python or another language. This material is often quite Python-specific; therefore, unlike the mandatory material, it cannot be readily ported to other languages. Please refrain from using this material in any exercises demonstrated in the first semester.

7.5.1 List Comprehensions

Now that a basic level of understanding of *for* loops and some data structures (lists, dictionaries, sets, etc.) has been achieved, an advanced construct can be considered: a comprehension. Comprehension are not available in many widely-used languages such as Java, C, C++, etc, but are available in other languages

such as Haskell (where they originated). In EE106, the basic material focusses on constructs and techniques that can be applied more widely to other languages; therefore, comprehensions are not addressed as part of the compulsory material. Nonetheless, they are useful and will be found in pre-existing code, and so are covered here.

Comprehensions are useful ways to create and modify the contents of data structures (collections) such as lists, dictionaries and sets. In this section, the focus is on list comprehensions, but a similar syntax can be utilised for dictionaries and sets. A list comprehension can be used to create and fill a list. The traditional approach to adding multiple elements to a list would involve the use of a *for* loop. For example, to create a list with the integers 0 to 9, the following code could be written as shown in Listing 7.23.

```
1 # Create an empty list
2 my_list = []
3 for i in range (10):
4 my_list.append(i)
5
6 print(my_list)
```

Listing 7.23: For loop list initialisation

We can use the convenient syntax of a list comprehension to achieve the same results as shown in Listing 7.24.

```
1 # Initialise a list using a comprehension
2 my_list = [i for i in range(10)]
3 print(my_list)
```

Listing 7.24: Using a comprehension for list initialisation

To understand Listing 7.24, Figure 7.8 depicts the syntax. We define a list name in the usual manner: list name equals then the list is specified by squared brackets. Instead of list items separated by commas within the squared brackets, we have an expected output and a form of a *for* loop.

list_name = [output_expression for item in iterable]

Figure 7.8: Basic syntax

Figure 7.9 provides means to interpret the syntax of Listing 7.24.

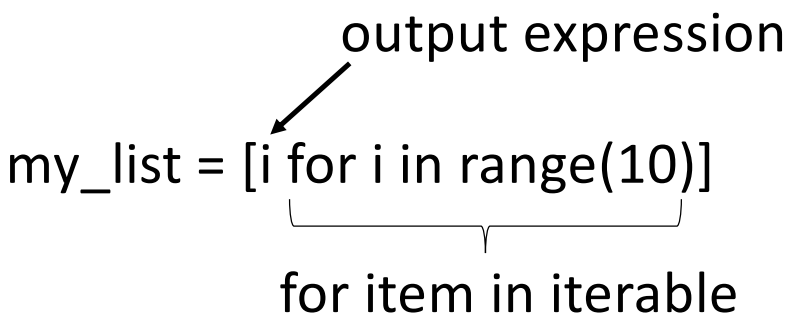


Figure 7.9: Adding simple integer values

Let's look at a slightly more complex example, as shown in Listing 7.25. In this example we add the squares of the values to the list. The syntax is illustrated in Figure 7.10

```
# Initialise a list using a comprehension
my_list = [i**2 for i in range(10)]
print(my_list)
```

Listing 7.25: Using a comprehension for list initialisation

Figure 7.10: Adding the square of integer values

List comprehensions can be used to create a list by processing the elements of another list, as shown in Listing 7.26. Here we create a new list which contains all the values of the old list multiplied by 10.

```
old_list = [1, 3, 5, 7]
# Create a new list from another one using a comprehension
new_list = [item * 10 for item in old_list]
print(new_list)
```

Listing 7.26: Using a comprehension for list initialisation

We can also use conditions when we create a list based on the contents of a pre-existing data structure. The syntax can be extended as shown in Figure 7.11.

list_name = [output_expression for item in iterable if condition]

Figure 7.11: Extended syntax

By example, lets say we wish to extract only the numerical characters from within a string and store them in a list, we could achieve this with a list comprehension by utilising the *isdigit()* function which returns a boolean True if a character has a numerical representation. The syntax for this is shown in Figure 7.12, and the code in Listing 7.27.

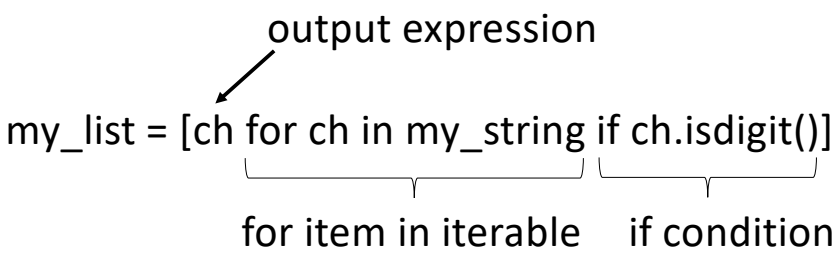


Figure 7.12: Condition initialisation

```
1 my_string = 'This is 798 a string with numbers in it'
2 # Create a  list from a selection of characters in a string
3 my_list = [ch for ch in my_string if ch.isdigit()]
4 print(my_list)
```

Listing 7.27: Using a comprehension for list initialisation

7.6 Summary

1. There are a number of different container objects available in Python, and each has its particular strengths.
2. A list is useful for storing and manipulating a collection of data.
3. Lists can be used in conjunction with strings to assist in processing.

4. A map (or dictionary) is an unordered container. It is useful for accessing information based on a key.
5. A set in computer programming is analogous to a set in mathematics.

7.7 Problem Sets

1. Write a program that prompts the user to enter a sentence, the computer should print the words of that sentence in alphabetical order. Example: an input string of 'the quick brown fox jumps over the lazy dog' is rearranged to read 'brown dog fox jumps lazy over quick the the'.
2. Write a program that uses a list to store 10 numerical values. Python has some special functions that can be used in conjunction with lists in the special case where the list contains only numbers. Determine and use the appropriate functions to print the minimum value, the maximum value and the sum of the values in the list. For example, if the list contains the values: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, then the minimum value should be 0, the maximum value should be 9, and the sum of the values should be 45.
3. Write a program that uses a map (dictionary) to store the first name (key) of five people together with their age (value). Using your own initiative, search the Python documentation for the additional functions which access all keys in the map and all values in the map. Use these functions to print all keys in the map to the screen followed by all values. Using the documentation for a language is an essential part of using that language. Take your time to have a good look at it.
4. Write a computer program that creates a set containing the strings 'apple', 'banana', and 'cherry'. Your code should test the set to determine if the string 'banana' is a member and print the result to the screen.

Chapter 8

Functions

The computer programs considered thus far have been fairly short and simple to implement. However, as the size and complexity of code increases techniques are required in order to manage this complexity. Modularisation is adopted to successively break down the overall goal of the program into smaller intermediate goals; those intermediate goals can be broken down still further and so on. Thus, a divide and conquer approach is adopted in order to manage complexity. One of the main tools in modularisation is the **function**. Functions are often known by other names such as procedures or methods, within the context of this course the terms will be used interchangeably.

Functions are well known in mathematics, and define an operation on inputs, e.g. $y = f(x) = x^2$. Figure 8.1 provides a pictorial representation of this function, each value in the domain (input x) is mapped to a value in the co-domain (output y).

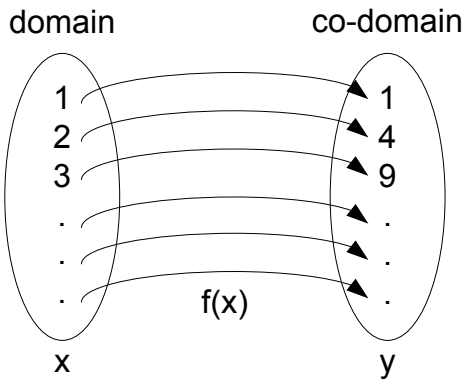


Figure 8.1: A function as a mapping

In this case there is a single input and a single output. However, in mathematics it is possible to have

a function that takes several inputs, for example consider a function that computes the hypotenuse (z) of a triangle given the lengths of the other two sides: x and y. Such a function would be of the form $z = f(x, y) = \sqrt{x^2 + y^2}$. In computer programming it is possible for a function to take zero or more inputs. The function itself will generally return zero or one outputs; if multiple outputs are required then a bespoke data type must be devised to cope with this eventuality; bespoke data types will be covered later.

The first question that may arise is *why define a function that returns zero outputs?* Clearly, such a function would make no sense in pure mathematics, and this is one of the small differences between the functions as used in computer programming and those encountered in more theoretical disciplines. In computer programming, a more general definition of a function is more relevant; a function is merely a sequence of operations performed to realise a given goal, i.e. a procedure. A function that processes the input arguments without providing an explicit return value can be illustrated by way of the example in Listing 8.1. The code fragment defines a string (*my_str*) in line 2, which is used as an argument (an input variable) to the in-built *print()* function on line 5. The *print()* function takes the argument (in this case a string) and displays it on the screen, but no data is returned back to the main body of code.

```
1 # Create an input to the function
2 my_str = ``Hello World''
3
4 # Call the in-built print function
5 print(my_str)
```

Listing 8.1: No return value

Now consider the illustrative example in Listing 8.2 that reads an input from the keyboard, this is achieved by calling the *input()* function (line 2) and storing the return value in *return_var*. This is an example of a function that take no input argument and returns a value back to the main body of the program for use later. This example is not particularly user-friendly and can be improved by prompting the user for an input as shown in Listing 8.3.

```
1 # A function with a return value
2 return_var = input()
3
4 # Call the in-built print function
5 print(return_var)
```

Listing 8.2: Return value of a function

Listing 8.3 is a modified version of Listing 8.2 that defines a string variable, *msg*, on line 2. The *msg* variable is fed into the *input()* function, as an argument, on line 5, where it is used internally to display a message on the screen. In this example, the *input()* function takes a single argument (a string named *msg*) and returns a single value back to the main body of code.

```
1 # Define a message
2 msg = ``Enter a string: ''
3
4 # A function with a return value and argument
5 return_var = input(msg)
6
7 # Call the in-built print function
8 print(return_var)
```

Listing 8.3: Function with argument and return value

These short examples illustrate that a function in a computer programming context can be used to perform a variety of mathematical/arithmetical operations and return a result back to the place in the program which called that function. From this perspective, a function behaves in a very similar manner to its mathematical analogue. Another perspective of a programming function is one that embraces the notion of code re-use: a function can be used to encapsulate a sequence of instructions that the programmer will use many times within a program. From the simple examples considered, it should be apparent that knowledge of both the argument (input) type and return (output) type is needed if a function is to be utilised correctly.

In Python, and indeed in most other general purpose programming languages, it is possible for the programmer to create their own bespoke functions to perform any number of duties. The syntax of a function definition is discussed in Section 8.1.

8.1 Function syntax

In Python a function must be defined before it is utilised (i.e. called). The syntax of a function definition is shown in Figure 8.2. The *def* keyword is used to tell the computer that a new user-defined function is being created, this is followed by a unique function name, then a list of input parameters enclosed in brackets, and then a colon. Indentation is used to define the sequence of instructions to be executed. Finally, the *return* keyword is used to return the point of execution back to the main body of code from where the function was originally called; a value can be returned (outputted) back to the main body.

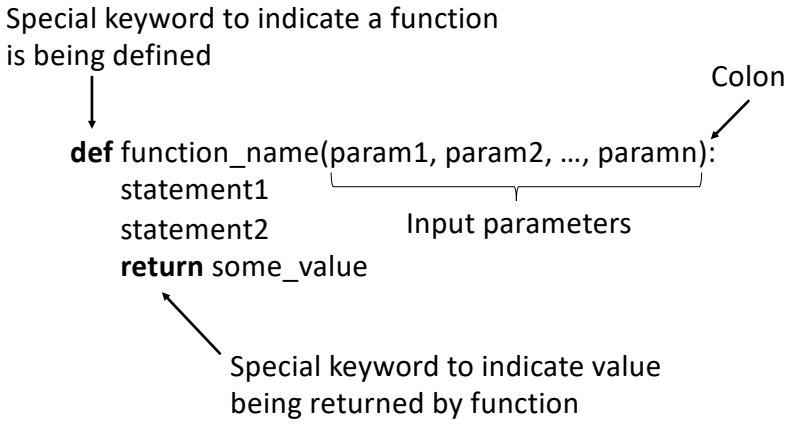


Figure 8.2: Syntax of a function definition

Section 8.2 provides some samples of function definitions to illustrate the syntax more fully. Some guides to usage and tips for best practice are also included.

8.2 Arguments and parameters

By way of a first example of a bespoke function, Listing 8.4 presents function which computes the square of a number; type in and execute this code. The function, *squared()*, is defined on lines 5 — 7, and invoked (or called) on line 14. In most programming languages, a function must be defined in advance of being used; therefore this approach will be adhered to in this course. A note on program execution: although the function definition starts on line 5, the first point of program execution is on line 10.

The function has a single input parameter (*ip*), and returns as an output value stored in *op*. The function is called, i.e. invoked, on line 14 and the return value stored in *y*. When the function is called, the value of the argument (*x*) is copied to the parameter (*ip*); this is an important point that is worth further explanation. A user input will be stored in the variable *x*, and copy of that value will be stored in parameter *ip*, thus two copies of the input value exist: one in the main body of code and another within the function. The copy within the function is not visible outside the function; verify this is true by changing the *y* to *ip* on line 17; any attempt to access *ip* outside the function should generate an error. Similarly when the function exits the value of the local variable (*op*) is copied to the variable defined in the main body of code (*y*). Any attempt to access the value of *op* outside the function will generate an error; satisfy yourself that this is true by modifying line 17 as appropriate. This will be explained more fully in Section 8.3.

```
1  # Define function before it is called
2  # ip is input parameter, and op is
3  # output value
```

```
4
5  def squared (ip):
6      op = ip * ip
7      return op
8
9  # Prompt user for an input
10 x = float(input(''Enter a number: ''))
11
12 # Call the function and store the
13 # return value in y
14 y = squared(x)
15
16 # Print result to screen
17 print(''Squared value is '', y)
```

Listing 8.4: Bespoke function

8.2.1 Multiple parameters

Just like in mathematics, a function defined in a computer programming language can take multiple arguments. Take for example the code in Listing 8.5, which executes the mathematical function $y = x^n$; this function has two parameters: x and n, as defined on line 2. In line 7 of the listing, the function is called and is positionally passed two arguments: a 2 followed by a 3. In the case the first argument (2) is mapped to the first parameter (x), and the second argument (3) is mapped to the second parameter (n), and so the function computes 2^3. Conversely, in line 8, the function is called and is positionally passed two arguments: a 3 followed by a 2. In the case the first argument (3) is mapped to the first parameter (x), and the second argument (2) is mapped to the second parameter (n), and so the function computes 3^2 which is different from 2^3. Clearly when arguments are passed positionally, the order of the arguments matter. Perhaps a less error-prone approach would be to pass the arguments by keyword, as shown in lines 10 & 11; in this case the order does not matter because the mapping is explicitly specified in the function call.

```
1  # Raise x to the power of n
2  def power(x, n):
3      y = x ** n
4      return y
5
6  # Call the power function with positional arguments
7  y1 = power(2,3)
8  y2 = power(3,2)
9  # Call the power function with keyword arguments
10 y3 = power(x=2, n=3)
11 y4 = power(n=3, x=2)
```

Listing 8.5: Raise to a power function

8.2.2 Return values

Not all functions need return a value back to the main body of code, Listing 8.6 is an example of a function that takes a single argument as an input, squares it and prints the result to the screen, i.e. no value is return back to a variable in the line that called it. To call a function that does not return a value simply type its name and remember to use brackets and any parameters that need to be passed into that function as demonstrated on line 13. Compare this line with that of line 14 in Listing 8.4, which uses the form $y = function_name(x)$.

Type in and test the code in Listing 8.6. In this case execution will pass back to the main body of code once the *return* statement on line 5 is executed. Notice that the *return* statement in this example is not followed by a variable name.

```
1  # Define function with no return value
2  def square_and_print(ip):
3      result = ip * ip
4      print(''Square is: '', result)
5      return
6
7  ###### Main body ######
8
9  # Prompt user for an input
10 x = float(input(''Enter a number: ''))
11
12 # Call function to print the square
13 square_and_print(x)
14
15 # Print something to confirm
16 # execution has returned from function
17 print(''End of program'')
```

Listing 8.6: Function with no return value

What would happen if we tried to assign a value to a function that does not return one? Modify Listing 8.6 to accept a value from the function and print it to the screen as shown in Listing 8.7. What happens when y is printed to the screen, take a note of this?

```
1  # Define function with no return value
2  def square_and_print(ip):
3      result = ip * ip
4      print(''Square is: '', result)
5      return
6
7  ###### Main body ######
8
```

```
9  # Prompt user for an input
10 x = float(input(''Enter a number: ''))
11
12 # Call function to print the square
13 # Assign the return value to y
14 y = square_and_print(x)
15
16 # Print y to the screen
17 print(''y = '', y)
18
19 # Print something to confirm
20 # execution has returned from function
21 print(''End of program'')
```

Listing 8.7: Failed reference to a return value

The examples thus far have considered functions with only one *return* statement. There may be situations were it is unavoidable to use multiple *return* statements; however, it should be borne in mind that this will lead to multiple exit points in the function. Consider the simple example in Listing 8.8; a function *above_zero()* is defined which prints to the screen whether an input is either positive or negative; the function contains two *return* statements: one on line 5 and another on line 8. Type in and test the code.

```
1  # Define function with multiple return statements
2  def above_zero(ip):
3      if ip >= 0:
4          print(''Number is positive'')
5          return
6      else:
7          print(''Number is negative'')
8          return
9
10 ###### Main body ######
11
12 # Prompt user for an input
13 x = float(input(''Enter a number: ''))
14
15 # Call function
16 above_zero(x)
```

Listing 8.8: Multiple exit points

Just as the use of *continue* and *break* statements are generally discouraged within loops unless absolutely necessary, so are the use of multiple *return* statements within a function. Multiple exit points make the flow of execution of a function more difficult to read and consequently make future modification of the function more difficult. A better approach would be to rewrite the function as shown in Listing 8.9. In this case only a single *return* statement (line 7) is used, yet the function has exactly the same behaviour as that in Listing 8.8.

```
1  # Define function with a single return statement
2  def above_zero(ip):
3      if ip >= 0:
4          print(''Number is positive'')
5      else:
6          print(''Number is negative'')
7      return
8
9  ###### Main body ######
10
11 # Prompt user for an input
12 x = float(input(''Enter a number: ''))
13
14 # Call function
15 above_zero(x)
```

Listing 8.9: Single exit point

8.3 Scope

The scope of a variable is the region of the program code where it is possible to access that value of that variable. If it is possible to access the variable then it is said to be *in scope*. When a variable to not in scope, it may still be stored in the computer's memory; however, it will not be accessible using its variable's name.

Whilst it is true that the value of variables declared locally, within the function, are not accessible globally; the converse is not true. Variable declared as part of the main body of code are accessible within the function. To test this modify the code in Listing 8.4 to print out the value of the variable *x* declared globally as shown in Listing 8.10.

```
1  # Define function before it is called
2  # ip is input parameter, and op is
3  # output value
4
5  def squared (ip):
6      op = ip * ip
7      print(x)
8      return op
9
10 # Prompt user for an input
11 x = float(input(''Enter a number: ''))
12
13 # Call the function and store the
14 # return value in y
```

```
15 y = squared(x)
16
17 # Print result to screen
18 print(''Squared value is '', y)
```

Listing 8.10: Variable visibility

It is generally considered best practice to use different names for parameters and local variables within the function definition from that used for arguments and return variables in the main body of code. Consider the example in Listing 8.11, this is a modified version of Listing 8.4 in which the parameter and local variable names have been modified to be the same as the argument and return variable names. Take a moment to compare and contrast these two examples. When the code in Listing 8.11 is executed, once again two copies of the input value will be stored in memory: one value will be stored in the variable *x* in the main body of code, and another in the parameter *x* within the function. Thus two variables named *x* will be in existence at the same time! The computer deals with such ambiguous situations by utilising the variable with the local scope first, that is within the function *x* refers to the local version, and within the main body of code it refers to the global version.

```
1 # Define function before it is called
2 # x is input parameter, and y is
3 # output value
4
5 def squared (x):
6     y = x * x
7     return y
8
9 # Prompt user for an input
10 x = float(input(''Enter a number: ''))
11
12 # Call the function and store the
13 # return value in y
14 y = squared(x)
15
16 # Print result to screen
17 print(''Squared value is '', y)
```

Listing 8.11: Arguments and parameters

Should the function not execute as hoped, the programmer will invariably commence debugging by examining the values of various parameters etc. to identify flaws in the program logic. If the function were defined as shown in Listing 8.11 it would be considerably more difficult to examine the values of variables that exist with the same name at different scopes. Is is for this reason that is highly recommended to use different names for parameters and local variable than those used globally.

Another important consideration, then, is the scope of a variable not just its type. To reinforce this point type in and run the code in Listing 8.12. An arbitrary global variable is defined (*temp*) and assigned the value of 10 in line 18. Subsequently, upon entering the function a local variable is defined, on line 10,

and assigned the value of 7. Execute the code and take a note of the values of the variable *temp* that are accessible both locally within the function and globally within the main body of code. Now comment out line 10 in the code by prefixing the line with a '#'; run the code again and compare the result. It should be apparent that defining a local variable with the same name as one that is defined more globally prevents access to that global variable. Or phrased another way: a local variable will mask a global variable of the same name.

```
1  # Define function before it is called
2  # ip is input parameter, and op is
3  # output value
4
5  def squared (ip):
6      op = ip * ip
7      # Create a local variable with
8      # same name as global variable
9      # but with different value
10     temp = 7
11     print(''Local value of temp is '', temp)
12     return op
13
14 # Prompt user for an input
15 x = float(input(''Enter a number: ''))
16
17 # Create another global variable
18 temp = 10
19
20 # Call the function and store the
21 # return value in y
22 y = squared(x)
23
24 # Print result to screen
25 print(''Squared value is '', y)
26 print(''Global value of temp is '', temp)
```

Listing 8.12: Scope resolution

8.4 Code reuse

Modularisation of code helps reduce complexity and facilitate code reuse. Consider the example of computing the factorial of an integer entered from the keyboard as shown in Listing 8.13. If there is a requirement to compute some mathematical function a number of times then it makes sense to convert this code into a function, as shown in Listing 8.14. Type in and run this code to ensure it executes as expected.

```
# Prompt user for an input and cast to int
ip = int(input(``Enter an integer: ''))

# Add 1 to input value to compensate
# for loop ending at end-1
n = ip + 1

# Initialise product variable
# Note 0! = 1
product = 1

# Set upper bound to n
for index in range(1,n):
    # Multiply by current index value
    product = product * index

# Print result to screen
print(``Factorial = '', product)
```

Listing 8.13: Factorial revisited

```
# Define factorial function
def my_fact(ip):
    # Add 1 to input value to compensate
    # for loop ending at end-1
    n = ip + 1

    # Initialise product variable
    # Note 0! = 1
    product = 1

    # Set upper bound to n
    for index in range(1,n):
        # Multiply by current index value
        product = product * index
    return product

##### Main body of code #####

# Prompt user for an input and cast to int
ip = int(input(``Enter an integer: ''))

# Create a variable to store the answer
result = my_fact(ip)

# Print result to screen
print(``Factorial = '', result)
```

Listing 8.14: Functionalised factorial

The utility of this approach may become clearer by revisiting the problem of calculating an exponential. The sample code for computing the exponential, based on a nested loop, is repeated in Listing 8.15

```
# Prompt user for an input and cast to int
x = int(input(''Enter an integer: ''))

# Initialise the result
result = 0.0

for n in range(50):
    # calc ip to the power n
    numerator = x ** n

    # Now calc the factorial
    # denominator is the factorial
    # Note 0! = 1
    denominator = 1
    # Add 1 to n to compensate
    # for factorial loop ending at end-1
    for index in range(1,n+1):
        # Multiply by current index value
        denominator = denominator * index
    # Do the cumulative summation
    # term by term
    result = result + (numerator / denominator)

# Print result to screen
print(''Exponential of '', x, ''is '', result)
```

Listing 8.15: Exponential revisited

It is possible to modify the code in Listing 8.15 by including the functionalised version of the factorial (Listing 8.14). Modify the code in Listing 8.15 to incorporate the factorial function; your code should resemble that shown in Listing 8.16.

```
# Define factorial function
def my_fact(ip):
    # Add 1 to input value to compensate
    # for loop ending at end-1
    n = ip + 1

    # Initialise product variable
```

```
8       # Note 0! = 1
9       product = 1
10
11      # Set upper bound to n
12      for index in range(1,n):
13          # Multiply by current index value
14          product = product * index
15      return product
16
17  ##### Main body of code #####
18
19  # Prompt user for an input and cast to int
20  x = int(input(''Enter an integer: ''))
21
22  # Initialise the result
23  result = 0.0
24
25  for index in range(50):
26      # calc ip to the power n
27      numerator = x ** index
28      denominator = my_fact(index)
29      result = result + (numerator / denominator)
30
31  # Print result to screen
32  print(''Exponential of '', x, ''is '', result)
```

Listing 8.16: Exponential with factorial function

Hopefully, the code in Listing 8.16 should be more readable. In particular, the inner loop can now be replaced by the function call in line 28, thus the main body of code now looks fairly simple. This example highlights the usefulness of libraries — prewritten code, produced by other programmers that can be used to help solve more complex problems. Indeed, the built-in math library has a prewritten *factorial()* function. The beauty of library functions such as the *factorial()* function is that the programmer does not need to know the details of implementation (i.e. how the function does what it does), the programmer only needs to know it behaviour (i.e. what it does) in order to use it.

Much of what is termed 'software architecture' is concerned with defining a chain of functions as described in Section 8.5.

8.5 Function chains

As mentioned, modularisation is an approach to managing complexity whereby a divide and conquer approach is taken to solving large problems. Large problems are broken down into smaller problems, and each of these are broken down into still smaller problems and so on. This approach to design is mirrored in the code produced where one function will typically call a number of other functions and each of these functions may call other functions and so on. When each of these functions exits, control returns to the

calling function, and this continues up the function call chain.

For example, suppose *function1()* calls *function2()* which calls *function3()* which calls *function4()*, as illustrated in Figure 8.3. It is only after *function4()* has completed its task that control is passed back to *function3()*, and similarly it is only when *function3()* is completed that control is passed back to *function2()*, and so on. That is, control passes to the next level in the function chain, not back to the very top.

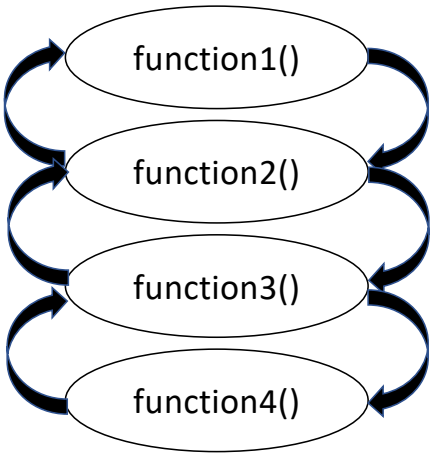


Figure 8.3: A function chain

Although the notion of divide and conquer describes the approach, it is not often clear to the novice programmer when to end this process, i.e. when to stop breaking down functions into other functions. This is where experience and judgement are required as there are no hard and fast rules for this process. In commercial software houses, this job is often given to the most experienced programmers — the software architects. The more junior members of the team are given a list of functions to code and a description of their behaviour.

There are however some rules of thumb that apply to functionalisation. As a general rule a function should perform one simple task; clearly there are many philosophical opinions on what constitutes a simple task, but ultimately this is for the programmer to decide. Another general rule is that if your algorithm cannot be viewed on a standard size monitor with a reasonable font size then it should be decomposed into functions.

8.6 Importing functionality

Once you have written some bespoke functions, it makes sense to store them in one or more files to use later; this is much more convenient than cutting and pasting pre-written functions into new programs. This can be illustrated by a trivial example. The code in Listing 8.17 defines a simple function. We can save this code in a file named, say, mylib.py.

```
# An example of a function to be imported


def print_my_name():
    print(''Robert'')
```

Listing 8.17: A library file containing a function

When you write a new program and would like to utilise the function written in another file, you can use the import command as shown in Listing 8.18. Note the syntax, we use *from <file name> import <function name>*. If the file contains multiple functions then you can import them using *from <file name> import <function name 1>, <function name 2>*. If you wish to import all the functions in a given file you can use *from <file name> import ∗*.

```
# How to import a function

from mylib import print_my_name

print_my_name()
```

Listing 8.18: Importing a function

8.7 Advanced Material

Where applicable, each lesson will be accompanied with some advanced material that is not examinable and may be of interest to those who can already program in Python or another language. This material is often quite Python-specific; therefore, unlike the mandatory material, it cannot be readily ported to other languages. Please refrain from using this material in any exercises demonstrated in the first semester.

8.7.1 Lambda Functions

In simple terms, a lambda function is a small, (often anonymous) function. Perhaps is use its best illustrated by example. The code in Listing 8.19 defines a lambda function called y, which takes an input argument x and returns the value $x + 2$.

```
y = lambda x: x + 2
print(y(5))
```

Listing 8.19: Lambda function that adds 2 to an argument and returns the result

This lambda function is functionally equivalent to the code in Listing 8.20. Compare and contrast the two approaches.

```
def y (x):
    op = x + 2
    return op

print(y(5))
```

Listing 8.20: Standard function that adds 2 to an argument and returns the result

The syntax of a lambda function is shown in Figure 8.4.

lambda argument(s): expression

Figure 8.4: Lambda function syntax

Just like standard Python functions, a lambda function can take multiple arguments as shown in Listing 8.21. In this example, two values are passed to the lambda function and their summation is returned.

```
y = lambda a, b: a + b
print(y(5, 7))
```

Listing 8.21: Lambda function that takes two arguments

A lambda function can be executed as it is defined as shown in Listing 8.22. In this example, the lambda function hasn't been given a name, and cannot be used elsewhere; this is known as an anonymous function. These small anonymous functions can be used to modify the arguments to other functions on the fly, as shown in Listing 8.23. It's worthwhile pausing to consider what is happening in Listing 8.23: the function *my_range()* is being passed another function rather than a value, it is that other function that is being passed a value. This approach to using lambdas should be used with great care: such code is often difficult to debug and difficult for other programmers to read.

```
print((lambda x: x + 2)(5))
```

Listing 8.22: Anonymous lambda function

```
# Define a range fn that gives numbers from 1 to n
# rather than 0 to n-1
def my_range(x, func = (lambda x: x+1)):
    return list(range(1,func(x)))

r = my_range(7)
print(r)
```

Listing 8.23: Anonymous lambda function passed to another function

In mathematics, passing a function to another function is known as *function composition*. By way of illustration, Equation 8.1 defines a simple function $2x + 4$ and Equation 8.2 defines another x^3. We can compose a third function from $f()$ and $g()$ as shown in Equation 8.3. We are adopting a similar approach programmatically with the code in Listing 8.23.

$$f(x) = 2x + 4 \tag{8.1}$$

$$g(x) = x^3 \tag{8.2}$$

$$h(x) = (f \circ g) = f(g(x)) = (2x + 4)^3 \tag{8.3}$$

We could implement this functionality using standard functions (Listing 8.24), or perhaps by using a lambda function in concert with a standard function (Listing 8.25).

```
# Define function f
def f(x):
    return (2*x) + 4

# Define function g
def g(x):
    return x**3

result = g(f(5))
print(result)
```

Listing 8.24: Function of a function

```
# Define function g using lambda
def g(x, func = lambda x: ((2*x) + 4)):
```

```
    return func(x) ** 3

result = g(5)
print(result)
```

Listing 8.25: Function of a function using a lambda

The use of a lambda function does not provide the programmer with any additional programmatical tools, rather it provides a quick and easy syntax to define and use small functions, often as an input to another function. However, lambda functions are quite useful in Python when combined with the built-in *filter()* function. This function takes two input arguments: a function and an iterable data structure (collection) such as a list or a string. The function argument passed in conducts a test on each member of the iterable data structure and returns either a boolean True or False value. The Listing 8.26 demonstrates how the *filter()* function operates in conjunction with a predefined standard function to produce a list of even numbers present in another list. The same functionality can be obtained, perhaps more conveniently, using a lambda function as shown in Listing 8.27.

```
# Define a list of integers
old_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]

def is_even(x):
    return x%2 == 0

# Create a list of even numbers from the old_list
new_list = list(filter(is_even, old_list))
print(new_list)
```

Listing 8.26: Built-in filter function

```
# Define a list of integers
old_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Create a list of even numbers from the old_list using lambda
new_list = list(filter(lambda x: x%2 ==0, old_list))
print(new_list)
```

Listing 8.27: Built-in filter function using a lambda

8.7.2 Functions Within Functions

Just as it is possible to define and use a function in the global scope (namespace), it is possible to define and use a function within the body (and hence scope) of another function in Python. Note, this is not

possible in many other programming languages. From this perspective, we can say that one function (inner function) is nested within another (outer function). The inner function will not be accessible from outside the outer function; therefore, it must be called within its encapsulating outer function. Theoretically, there are advantages to keeping the scope of a function limited if you suspect the name of the function may be used elsewhere or it contains very highly specialised code that will only be of use within the outer function. However, the use nested functions is somewhat controversial within the computer programming community.

Recall, global variables are visible inside functions, but local variables defined inside functions are not visible in the global namespace. The same applied to nested functions: local variables defined within the outer function are accessible within the inner function but not vice-versa. Let's look at the code in Listing 8.28, which defines an inner function within an outer function. When this code is run, it is clear that the inner function is able to access (read) the value of the variable defined in the outer function. Modify the code by adding the line *print(outer_val)* to the end of the listing and verify it produces an error.

```
1  def outer():
2      outer_var = 10
3
4      def inner():
5          print('outer var = ', outer_var)
6
7      # Call inner function from within outer function
8      inner()
9  # Call outer function from main body of code
10 outer()
```

Listing 8.28: A function within a function

Although it is possible to read the value of a local variable defined within the outer function, it is not possible to change its value within the inner function. That is, the variable *outer_var* is read-only. Modify the code such that the variable is modified within the inner function, as shown in Listing 8.29. This illustrates that the value of *outer_var* is masked when another variable of the same name is created at a more local scope, but becomes accessible again when leaving the local scope.

```
1  def outer():
2      outer_var = 10
3
4      def inner():
5          outer_var = 11 # change value of variable
6          print('From inner(), outer var = ', outer_var)
7
8      # Call inner function from within outer function
9      inner()
10     print('From outer(), outer var = ', outer_var)
11 # Call outer function from main body of code
12 outer()
```

Listing 8.29: Scope of nested functions

8.7.3 Closures

So far we have learned that functions can be passed into other functions and that functions can be defined inside other functions. The logical extension to this idea is that an inner function can be retuned by an outer function to the global namespace. This permits the functionality of the inner function to be utilised externally, and is known as a closure. Closures are useful for permitting access to the value of the local variables defined in the outer function within the global scope, or for customising functions (so-called factory function).

The code in Listing 8.30 provides an example of the functionality of the inner function being returned to the global scope and stored in *my_func*. The outer function defines a local variable that is accessible within the outer function. The outer function then defines the inner function and returns it to the global scope. Pay special attention to line 7, parenthesis are not used in conjunction with the inner function name; it says *return inner* not *return inner()*. Back in the global scope, the outer function is called and passed the value of 10 (line 10). This outer function does not return a data type like an *int* or a *float*, in the same manner as most other functions, rather it returns another function which it subsequently calls in line 11. When executed, it is clear that the value of the local variable *outer_var* (11) is accessible in the main body of code.

```
1  def outer(arg):
2      outer_var = arg + 1
3      def inner():
4          print('From inner(), outer var = ', outer_var)
5
6      # Return the inner function from the outer function
7      return inner # Note, no parenthesis after function name!
8
9  # Call outer function from main body of code (global scope)
10 my_func = outer(10)
11 my_func()
```

Listing 8.30: Closure provides access to local variables

Another use of closures is in the definition of so-called function factories: functions that can be used to manufacture other functions. In Listing 8.31, an outer function *power_generator()* is defined which operates as a function factory: this creates a local variable *exponent* that is accessible to the inner function *get_power* and customises it accordingly. We can call the function factory and pass it a parameter that can be used to return customised functions. In this example, we define two customised functions: *squared* and *cubed*. They are subsequently used to raise a number to the power two and three respectively.

```
1  def power_generator(power):
```

```
    exponent = power # explicitly create local variable
    def get_power(number):
        return number ** exponent # raise number to a power

    # Return the inner function from the outer function
    return get_power # Note, no parenthesis after function name!

# Manufacture two functions via function factory
squared = power_generator(2)
cubed = power_generator(3)
# Call and print the output of the functions
print(squared(5)) # should give 25
print(cubed(7)) # should give 343
```

Listing 8.31: Closure as a function factory

8.7.4 Recursion

As mentioned in Section 8.5, it is possible for one function to call another, and so on. Recursion is a form of loop that can be achieved when a function calls itself! It is a seldom used construct because most problems can be solved better using iteration; therefore, it is advised that recursion is used with great care. To illustrate an example, consider the code in Listing 8.32; the code prints to the screen a count down from a given starting point. The danger of recursion is that it can result in an infinite loop which can cause a computer program to hang (crash). This can be avoided with the inclusion of an exit criterion such as that in lines 5 & 6, which quits the recursion when the value reaches zero.

```
1  # Define recursive function
2  def count_down(x):
3      print(x) # print value to screen
4
5      if x == 0: # Evaluate whether to terminate recursion
6          return
7      else:
8          count_down(x-1) # Call recursive function again
9
10     return
11
12 ### Main body of code ###
13
14 start_value = 12
15 count_down(start_value) # Initially call function
```

Listing 8.32: Recursion with exit condition

If lines 5 & 6 were removed then theoretically, the code would print to the screen 12, 11, 10 , ..., 1, 0, -1, ..., -10, ..., -100, ... and so on to $-\infty$. This code is illustrated in Listing 8.33. Type in and run this code, and note the error that it generates.

```
1  # Define recursive function
2  def count_down(x):
3      print(x) # print value to screen
4
5      count_down(x-1) # Call recursive function again and again
6
7      return
8
9  ### Main body of code ###
10
11 start_value = 12
12 count_down(start_value) # Initially call function
```

Listing 8.33: Dangerous recursion, infinite execution

In order to prevent infinite execution, Python provides useful functionality that can be used to limit the depth of recursion, and so enable early failure (with associated error). An example of this is shown in Listing 8.34.

```
1  # Imports
2  import sys
3
4  # Define recursive function
5  def count_down(x):
6      print(x) # print value to screen
7
8      count_down(x-1) # Call recursive function again
9
10     return
11
12 ### Main body of code ###
13 sys.setrecursionlimit(100) # set limited depth here
14 start_value = 12
15 count_down(start_value) # Initially call function
```

Listing 8.34: Limited depth of recursion

As mentioned, iteration is generally a safer and more intuitive approach to looping through a subsection of code, though recursion does have some very niche uses. It should be used with great care to include a suitable exit clause.

8.8 Summary

Based on the lessons presented in this section the following key points should be remembered.

1. A function in computer programming is broadly analogous to a function in mathematics.
2. A function in computer programming need not return a result, rather it can be used to execute some sub-steps of an algorithm.
3. In most programming languages, a function can take multiple parameters as inputs but can return at most one output. Where multiple return values are required, a custom data type must be used.
4. The function is central to code reuse and to managing complexity.
5. There are no set rules for how large or small a function should be. It is the programmers choice whether to continue to break a task down further into more functions.
6. A rule of thumb is that a function should accomplish one goal.
7. A rule of thumb is that if your code takes up more than one screen, then you should consider breaking it up into functions.
8. Where possible strive to have a single exit point to a function.
9. Store functions you write in a separate file for future use.

8.9 Problem Sets

1. Write a function that takes an integer as an argument and computes the sum of all integers between 1 and that argument. Assume the input will always be greater than 1. For example, if the value entered is 5, then the cululative sum returned should be 15; if the value entered is 6 then 21 should be returned.
2. Write a computer program that functionalises the exponential program written earlier in the course in Listing 4.7. A bespoke exponential function (called myExp) should be defined and called within the main body of code. Your function should internally call the bespoke factorial function defined earlier. This exercise illustrates an example of a function calling another function. Test your code to ensure it returns the correct answer: $e^1 = 2.71828182846$, $e^2 = 7.38905609893$.
3. Look up "Gamma function" in Wikipedia. Write a bespoke function to calculate $\Gamma(n)$, based on the factorial function already designed in previous exercises. Assume n is an integer. For example, $\Gamma(5) = 24$.

Chapter 9

Abstract Data Types

Just as it is possible for the programmer to design and implement their own bespoke functions to supplement the built-in functions and library functions that are available within most languages, it is also possible to design and implement custom data types. Custom data types can be regarded as compound types composed of other pre-existing data types. When designing a bespoke data type, the designer (programmer) will have to decide what information that data types should hold, and which other types should be used to store that information.

Consider the hypothetical example of a database of student records. Based on requirements specified by the hypothetical end-users, the designer decides that the bespoke data structure should hold each student's name, registration number and average mark, as illustrated in Figure 9.1.

The next stage in the design process is to determine which pre-existing data types to use to store each of these individual pieces of information. Since the student's first name and surname will contain textual data, using a *string* to hold this information is the obvious choice. The data type to store the registration number is less obvious: it is typically numerical but it is unlikely that any arithmetical/mathematical operations will be performed on it. In this case the designer opts to use a *string*. Finally, the average mark will be numerical but is likely to be the output of some mathematical calculations therefore a *string* is not the appropriate choice. Both the *int* type and *float* type are applicable, however, the *float* type is more general and is considered the most appropriate in this example. The designer will use appropriate variable names to make the code readable: first_name, surname, registration, and score.

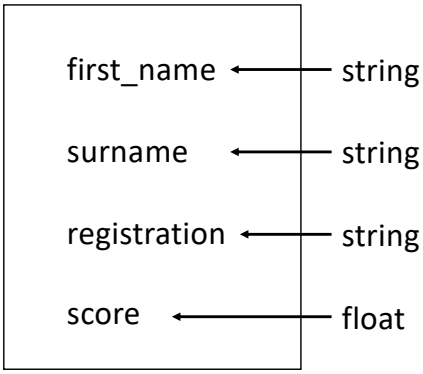


Figure 9.1: Student record

Listing 9.1 illustrates how a student record data type is defined in Python; the listing is explained with reference to Figure 9.2 which demonstrates the basic syntax; enter and run this code. Within Python, and indeed many other languages, a bespoke data structure is known as a **class**; therefore the *class* keyword is used to define a bespoke class as shown on line 2. A unique class name is required followed by a colon, indentation is used to define the contents of the class. Within a class, a special method is required, named *__init__()*, shown on line 4; this method name must begin and end with a double underscore, and the first parameter must be *self*. This is an initialisation method which is automatically executed whenever an instance of the class is created, consequently, it is used to set the initial values of that instance's internal data members (lines 6 — 9). Note the use of the *self.* prefix to denote that an internal data member is being accessed. An instance of the class is created, named *rec*, using the syntax shown on line 14; the class name is required along with the initial values of of the data members. Note, the order in which the initial values are fed to the class must reflect the order in which they are defined. The data members that form part of the class can be accessed using the 'dot' notation as shown on lines 17, 20 and 22.

It is possible to 'get' (i.e. read) the value of a data member using the dot notation as shown on line 17, it is also possible to 'set' (i.e. write) a value using the same notation as shown on line 20. Thus the contents of an instance can be modified after creation.

```
1  # Define an example class
2  class StudentRecord:
3      # Special initialisation method
4      def __init__(self, fn, sn, reg, sc):
5          # Initialise data members
6          self.first_name = fn
7          self.surname = sn
8          self.registration = reg
9          self.score = sc
10
11 ##### Main body #####
12
```

```
13 # Create a class instance
14 rec = StudentRecord("Fred", "Bloggs", "1234", 71)
15
16 # Read an instance's data member
17 print(rec.surname)
18
19 # Modify a data member
20 rec.first_name = "Jim"
21
22 print(rec.first_name)
```

Listing 9.1: studentRecord definition

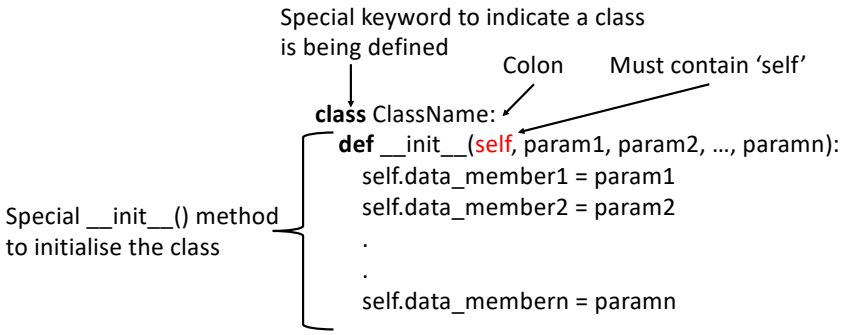


Figure 9.2: Syntax of a basic class

Before proceeding it is worth clarifying some of the nomenclature relating to functions, classes and variables. A variable defined in the main body of code is a global variable, one defined within a function is a local variable (i.e. local to that function), and one defined within a class such as *first_name* in Listing 9.1 is a **data member** of that class. A function defined within a class, such as *__init__()* is a **method**, although it is often referred to as a **member function**; both terms can be used interchangeably. The class defines the specification for a bespoke data type; it can be regarded as the blueprint from which all instances of that type will adhere. A variable[1] created from a particular class, e.g. *rec* in Listing 9.1, is referred to as an **instance** of that class, more generally an instance is also known as an **object**. The relationship between and instance (i.e. an object) and its class is the same as the relationship between a variable and its type. The process of creating an instance of a class is known as **instantiation**.

The ability to create a customised data type permits a convenient means to return multiple values from a function[2]. A class can be defined that has multiple data members and each of these can be one of the return values from the function. Consider the example in Listing 9.2, enter and run this code. This code provides a simple example of how classes can be used to aggregate data. Lines 15 — 23 define a function

[1]The term variable is used loosely here.

[2]In actual fact there is a data type known as a *tuple* that permits this functionality, but we will ignore this fact for the time being since this functionality is not available in many other languages.

get_displacement() that calculates the displacement to a given coordinate point (x, y); the displacement is a vector quantity composed of a magnitude and a phase angle, therefore an aggregated data type (class) is needed to return both of these values from a function. The class *Displacement*, defined on lines 5 — 10, implements this aggregated data type by virtue of its two data members *magnitude* and *angle*, as shown in Figure 9.3. The values of *x* and *y* are defined on lines 29 & 30, and are passed as arguments into the *get_displacement()* function on line 34. The return value is stored in the object *disp*. When the function *get_displacement()* is called it calculates the vector quantities using predefined functions as part of the math library, the results are stored in the local variables *hypotenuse* and *phase* on lines 17 and 18.

An instance of the class *Displacement* is created on line 34 and named *rtn*; the local variables are used to initialise the data members *magnitude* and *angle* of this object. The object containing the aggregated data can then be returned to the main body of code as shown on line 23. Both items of data can be accessed independently using the 'dot' notation as shown on lines 37 and 38.

```
 1  # Import maths functionality
 2  import math
 3
 4  # Define a class to hold 2 values
 5  class Displacement:
 6      # Special initialisation method
 7      def __init__(self, m, theta):
 8          # Initialise both values
 9          self.magnitude = m
10          self.angle = theta
11
12
13  # Define a global function
14  # takes x and y coordinates
15  def get_displacement(ipx, ipy):
16      # Create 2 local varaiables
17      hypotenuse = math.hypot(ipx, ipy)
18      phase = math.atan2(ipy , ipx)
19      # Put values into an instance of
20      # class displacement
21      rtn = Displacement(hypotenuse, phase)
22      # Return both values in a single instance
23      return rtn
24
25
26  ##### Main body #####
27
28  # Set coordinate values
29  x = 1.0
30  y = 1.0
31
32  # Call function which returns
33  # aggregated values
34  disp = get_displacement(x,y)
```

```
35
36 # Print both values to screen
37 print(disp.magnitude)
38 print(disp.angle)
```

Listing 9.2: Aggregated data class

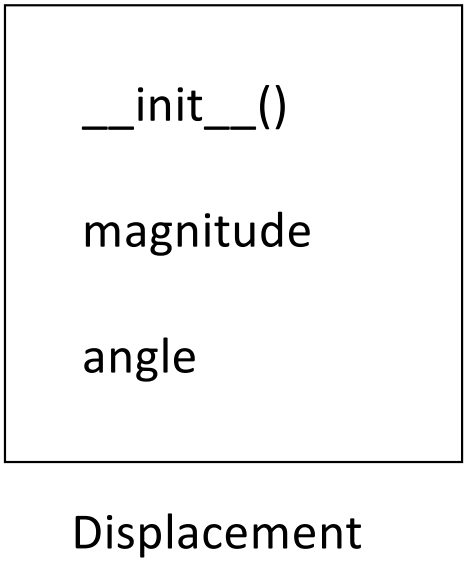


Figure 9.3: *Displacement* class

Customised data types are a powerful tool to enable the programmer to fashion their own data structures to fit their particular needs. It makes sense to create a single data structure to hold student records or employee information in a database, for example. Intelligent class definition can lead to more simplified problem solving by allowing the programmer to think at a higher level of abstraction: functions can now be used to perform operations on the internal state of objects.

From the novice programmer's perspective, one of the most unusual aspects of a class definition is that includes a function (or more correctly a method), i.e. the bespoke data structure contains not only data members but also special internal functionality. The ability to include both information (data) and functionality gives rise to a new and intuitively reasonable approach to designing computer programs: programming using abstract data types as discussed in Section 9.1.

9.1 Programming Using Abstract Data Types

This course has taken an evolutionary approach to structuring code. Initially, all programs consisted of a single procedure (effectively a single function) where a sequence of instructions was executed consecutively. Subsequently, a degree of modularity was introduced through the use of functions: the main function could call another function and that could call other functions and so on. This approach to structuring code is known as a function-based paradigm; *C* is a well known example of a function-based programming language. The ability to combine functionality together with the data upon which that functionality operates, affords designers the opportunity to make code more logical and reusable. A data type (i.e. a class) that includes the functionality to operate upon the internal data is referred to as an **abstract data type**.

To illustrate the benefits of programming using abstract data types, the program in Listing 9.2 will be reformulated through several evolutions to produce a more elegant solution.

9.1.1 Data Aggregation

Consider the program in Listing 9.2 to calculate the displacement of a given coordinate from the origin. In that example, a global function, *get_displacement()*, is used to calculate the displacement. From a design perspective, the function takes some representation of a coordinate (in this case two arguments x and y), performs some operations on them, and returns the displacement vector, *disp*. If we step back and reconsider the problem; x and y are attributes of the coordinate and therefore it may make more sense define a data structure that holds these attributes, as shown in Figure 9.4. In Python this can be encoded as shown in Listing 9.3.

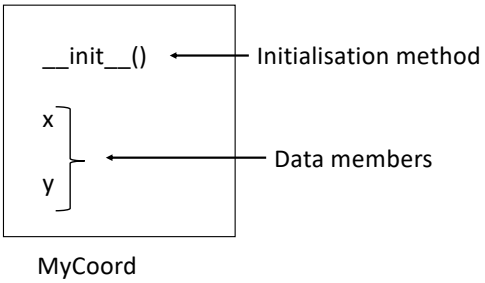


Figure 9.4: Simple *MyCoord* class

```
1 # Define a primitive coordinate class
2 class MyCoord:
3     # Special initialisation method
4     def __init__(self, ipx, ipy):
5         # Initialise both values
6         self.x = ipx
```

```
7            self.y = ipy
```

Listing 9.3: Simple coordinate class

Thus the first step towards an improved solution involves identifying the essential nature of the data to be stored and represented as a class. The next step is to incorporate the necessary functionality as discussed in Section 9.1.2.

9.1.2 Incorporating Functionality

Further consideration of the problem reveals that the *get_displacement()* function operates on the coordinate attributes. The principle of programming using abstract data types is based on incorporating functionality within a class that operates on the data members of that class. Consequently, the class can be redesigned as depicted in Figure 9.5; the code for this improved design is shown in Listing 9.4. Special attention should be paid to the syntax of the *get_displacement()* method on lines 9 — 17: it **must** have a parameter *self* included. There is no need to pass any arguments into this method since the data required for it to perform its duties is already available through the data members: *self.x* and *self.y*. In fact that is the whole point — the data is already available within the object!

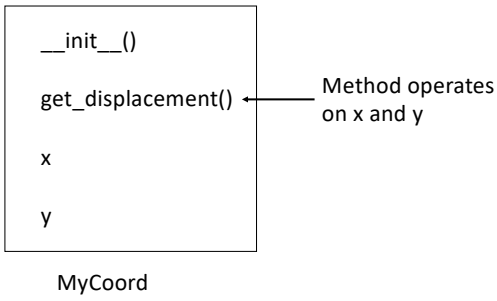


Figure 9.5: Incorporating functionality

```
1  # Define abstract coord class
2  class MyCoord:
3      # Special initialisation method
4      def __init__(self, ipx, ipy):
5          # Initialise both values
6          self.x = ipx
7          self.y = ipy
8
9      def get_displacement(self):
10         # Create 2 local variables
11         hypotenuse = math.hypot(self.x, self.y)
```

```
        phase = math.atan2(self.y , self.x)
        # Put values into an instance of
        # class displacement
        rtn = Displacement(hypotenuse, phase)
        # Return both values in a single instance
        return rtn
```

Listing 9.4: Added functionality

The *get_displacement()* method returns values that are contained within the *Displacement* class, thus this solution includes two distinct classes as represented by Figure 9.6.

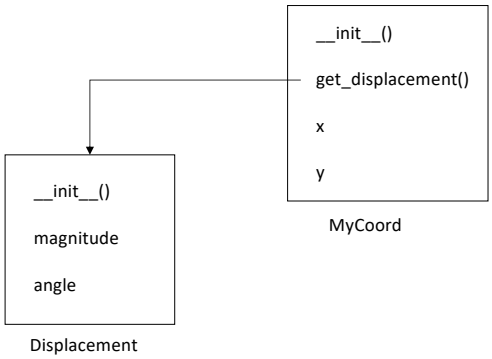


Figure 9.6: Two class solution

9.1.3 Refine The Class Interface

Now suppose there is a need to obtain the displacement of a given coordinate multiple times in a program, this could be achieved by calling the *get_displacement()* method multiple times. This is quite a computationally expensive approach since the displacement would have to be recalculated each time. A better approach would be to calculate and store the displacement initially then simply return the displacement vector on subsequent occasions. This can be achieved by decoupling the calculation and placing that functionality into another method *calc_displacement()*, as depicted in Figure 9.7. The code for this new design is shown in Listing 9.5, Enter and run this code.

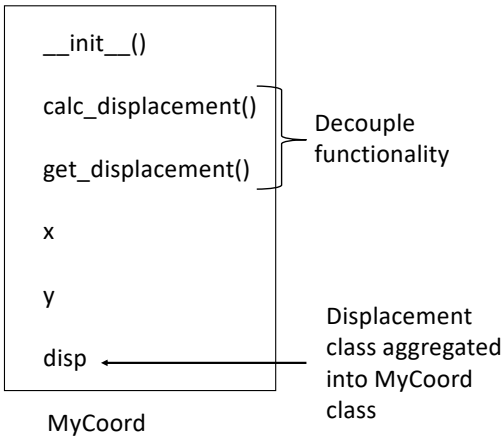


Figure 9.7: Decoupled functionality

```
# Import maths functionality
import math

# Define a class to hold 2 values
class Displacement:
    # Special initialisation method
    def __init__(self, m, theta):
        # Initialise both values
        self.magnitude = m
        self.angle = theta

# Define abstract coord class
class MyCoord:
    # Special initialisation method
    def __init__(self, ipx, ipy):
        # Initialise both values
        self.x = ipx
        self.y = ipy
        self.disp = Displacement(0.0,0.0)

    def calc_displacement(self):
        # Create 2 local variables
        self.disp.magnitude = math.hypot(self.x, self.y)
        self.disp.angle = math.atan2(self.y , self.x)

    def get_displacement(self):
        return self.disp

##### Main body #####
```

```
30
31 # Create an instance of a coordinate
32 point_a = MyCoord(1,1)
33
34 # Call instance method
35 point_a.calc_displacement()
36 vector = point_a.get_displacement()
37
38 print(vector.magnitude)
39 print(vector.angle)
```

Listing 9.5: Simplified design

The *MyCoord* class in Listing 9.5 now also includes a new data member *disp* which is initialised on line 19: this represents the displacement vector and once again it makes sense to include it as a data member of the *MyCoord* class because it is a property of the coordinate. The method *calc_displacement()* is used to calculate and store the correct vector quantities for *disp*, and the method *get_displacement()* is used to return a reference to *disp* back to the main body of code. A significant weakness in this design is that it is essential for the programmer to call the *calc_displacement()* method on line 35 before attempting to gain access to the distance vector (line 36). Failure to do so would return the displacement vector with its internal data members *magnitude* and *angle* set to 0.0. Clearly, this would be an erroneous result. It is worth noting that these types of error are among the most difficult for a programmer to detect since they do not cause the program to stop functioning, rather they simply return false information.

A better approach would be to initialise the values of *disp* appropriately upon the creation of an instance of the *MyCoord* class as described in Section 9.1.4.

9.1.4 Best Practice: Initialisation

The code in Listing 9.5 could be improved still further to ensure that the data members of *disp* are initialised to their correct values when the *myCoord* class is instantiated; lines 19 — 21 in Listing 9.6 demonstrate how this can be achieved. The great advantage of initialising this information at instantiation time is that the programmer does not have to remember to explicitly call method *calc_displacement()* before attempting to access the data members of *disp*.

```
1  # Import maths functionality
2  import math
3
4  # Define a class to hold 2 values
5  class Displacement:
6      # Special initialisation method
7      def __init__(self, m, theta):
8          # Initialise both values
9          self.magnitude = m
10         self.angle = theta
```

```
11
12 # Define abstract coord class
13 class MyCoord:
14     # Special initialisation method
15     def __init__(self, ipx, ipy):
16         # Initialise both values
17         self.x = ipx
18         self.y = ipy
19         mag = math.hypot(self.x , self.y)
20         ang = math.atan2(self.y , self.x)
21         self.disp = Displacement(mag , ang)
22
23     def calc_displacement(self):
24         # Create 2 local variables
25         self.disp.magnitude = math.hypot(self.x, self.y)
26         self.disp.angle = math.atan2(self.y , self.x)
27
28     def get_displacement(self):
29         return self.disp
```

Listing 9.6: Explicit initialisation

If the programmer knows in advance that the values of *x* and *y* will not change after the coordinate object has been created, then the *calc_displacement()* method can be removed from the design altogether. However, if the programmer intends the values to be updated then it is best left in place.

Another way in which the code could be improved is by replacing explicit initialisation of *disp* with a call to the *calc_displacement()* method from within the *__init__()* method, as shown in Listing 9.7. Compare and contrast this approach to that in Listing 9.6. The advantage of this approach may not be clear at first, but consider the situation where the program specification changes such that it is now required to return the angular component of displacement in degrees rather than in radians. The code in Listing 9.6 would have to be modified in two distinct places: lines 20 & 26. The necessity to modify two similar lines of code can lead to errors if the programmer modifies one but forgets the other; something all too common in large programs. Whereas the code in Listing 9.7 will operate correctly if only one line of code is modified: line 25. This approach constitutes best practice since it provides a better foundation for future modification, and hence re-use of existing code.

```
1  # Import maths functionality
2  import math
3
4  # Define a class to hold 2 values
5  class Displacement:
6      # Special initialisation method
7      def __init__(self, m, theta):
8          # Initialise both values
9          self.magnitude = m
10         self.angle = theta
11
```

```
12 # Define abstract coord class
13 class MyCoord:
14     # Special initialisation method
15     def __init__(self, ipx, ipy):
16         # Initialise both values
17         self.x = ipx
18         self.y = ipy
19         self.disp = Displacement(0.0 , 0.0)
20         self.calc_displacement()
21
22     def calc_displacement(self):
23         # Create 2 local variables
24         self.disp.magnitude = math.hypot(self.x, self.y)
25         self.disp.angle = math.atan2(self.y , self.x)
26
27     def get_displacement(self):
28         return self.disp
29
30 ##### Main body #####
31
32 # Create an instance of a coordinate
33 point_a = MyCoord(1,1)
34
35 # Call instance method
36 vector = point_a.get_displacement()
37
38 print(vector.magnitude)
39 print(vector.angle)
40
41 # Modify coord location
42 point_a.x = 2
43 point_a.y = 3
44
45 vector = point_a.get\_displacement()
```

Listing 9.7: Reusable design

Despite the improved design, it is all too easy to use the functionality of class *MyCoord* incorrectly, particularly if the class was designed by another programmer. If the values of x and y are modified (lines 42 & 43), the programmer who is using this class must remember to call the *calc_displacement()* method before attempting to access the *disp* data member (line 45). Failure to do so will result in false information being returned, i.e. the displacement of the previous coordinate values would be returned instead. This problem can be avoided with the appropriate use of an accessor method to modify the values of x and y, then recalculate the displacement vector, *disp*. This new method, to be called *update()* modifies the structure of the class as shown in Figure 9.8.

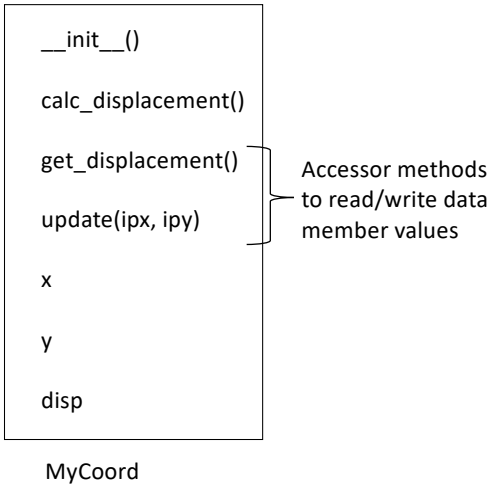


Figure 9.8: Accessor method

Modify the *MyCoord* class such that it resembles that in Listing 9.8. Run and test this code to ensure that the correct values are always returned. The *update()* method (lines 23 — 26) takes two parameters as inputs and uses them to modify the values of the *x* and *y* data members. It then calls the *calc_displacement()* method to ensure that the displacement vector, *disp*, contains the correct values. Thus after the coordinate values are modified (line 47), the correct answer will always be returned (line 48).

```
1
2  # Import maths functionality
3  import math
4
5  # Define a class to hold 2 values
6  class Displacement:
7      # Special initialisation method
8      def __init__(self, m, theta):
9          # Initialise both values
10         self.magnitude = m
11         self.angle = theta
12
13 # Define abstract coord class
14 class MyCoord:
15     # Special initialisation method
16     def __init__(self, ipx, ipy):
17         # Initialise values
18         self.x = ipx
19         self.y = ipy
20         self.disp = Displacement(0.0,0.0)
21         self.calc_displacement()
```

```
22
23      def update(self, ipx, ipy):
24          self.x = ipx
25          self.y =ipy
26          self.calc_displacement()
27
28      def calc_displacement(self):
29          self.disp.magnitude = math.hypot(self.x, self.y)
30          self.disp.angle = math.atan2(self.y , self.x)
31
32      def get_displacement(self):
33          return self.disp
34
35  ##### Main body #####
36
37  # Create an instance of a coordinate
38  point_a = MyCoord(1,1)
39
40  # Call instance method
41  vector = point_a.get_displacement()
42
43  print(vector.magnitude)
44  print(vector.angle)
45
46  # Modify coord location (2,3)
47  point_a.update(2,3)
48  vector = point_a.get_displacement()
49
50  print(vector.magnitude)
51  print(vector.angle)
```

Listing 9.8: Accessor method

9.2 Summary

This chapter has demonstrated the utility of abstract data types for building and structuring code. Abstract data types permit an additional layer of modularisation over and above that permitted by functions alone.

From a programming perspective calling a global function, e.g. *len()*, makes intuitive sense since some sequence of operations will be applied its argument. Syntactically, passing an argument into a function resembles the use of functions in mathematics. If we need to perform the same sequence of operations on another variable or object we simply call the function again. The *len()* function is a good example: it can be applied to various data types such as *lists* and *strings* as shown on lines 4 & 9 of Listing 9.9. Its cross-type applicability suggests it is good design to implement it as a (global) function, rather than as a method within various classes (types).

However, the ability to incorporate functionality into data types gives rise to a different approach to programming. Consider the built-in *string* type, which is used to contain alphanumeric characters. An elegant design would be to include functionality within that data type that may be frequently required by other programmers, an example is the *lower()* method that converts uppercase characters to their lowercase equivalent. Given that this functionality can only be applied to a *string*, it would seem appropriate to include it as a method within the *string* class rather than as a global function.

```
# Define a simple list and print length
num_list = [ 2, 8, 6, 4]

print("List length = ", len(num_list))

# Define a simple string and print length
name = "Fred Bloggs"

print("String length = ", len(name))

# Convert to lower case
print("String value = ", name.lower())
```

Listing 9.9: Functions and methods

1. Bespoke data types can be used to aggregate several other types.
2. Bespoke data types are useful as a container to pass multiple values between functions.
3. Abstract data types contain functionality (methods) to operate on the internal data members. Thus the data members can be used to store state, i.e. historical values.
4. Careful initialisation of data members is essential.
5. Consider accessor methods to properly update data member values.

9.3 Problem Sets

1. Define a class called *Parallel* that is initialised with the values of three resistors. The class should contain a method, *get_resistance()*, which returns the resistance of the three resistors connected in parallel.
2. Define a class called *Circle* that is initialised with a single value representing its radius. The class should contain the methods *get_circumference()* and *get_area()* which return the circumference and area respectively. It is to be expected that the radius can change, therefore an appropriate accessor method is required.
3. Define a class called *MyFibonacci* that can be used by other programmers to obtain the next value in the Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, ... The class should contain a method called *get_value()*

which returns the first value of the sequence when first called. Subsequent calls to the method will return the next value in the sequence. Note, each call to the method should return a single number.

4. Write a program that prompts the user to enter the location of two coordinates, and prints to the screen the equation of the straight line, $y = mx + c$ that connects them. A bespoke class should be designed to represent a coordinate (this should hold two values), and another class to represent a straight line (this should contain two coordinates and any required functionality).

Chapter 10

Object Orientated Programming

Aggregation is a powerful tool to express "**has a**" relationships in code, i.e. relationships where one entity **contains** another. For example, a class *vehicle* could be defined that aggregates two other classes: *engine* and *chassis*, as depicted in Figure 10.1. This makes logical sense since a vehicle contains an engine and also contains a chassis. Unfortunately, aggregation is not well suited to expressing in code "**is a**" relationships, i.e. relationships where one entity can be regarded as a **type of** another. For example, the class *car* could be defined that shares the attributes of a more general *vehicle* class, as represented by Figure 10.2. This makes logical sense since a car is a type of vehicle. Aggregation is an inappropriate tool to express this relationship since a car is not contained within a vehicle.

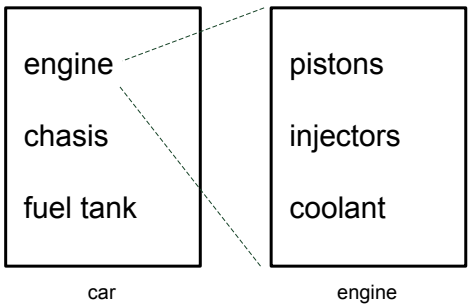


Figure 10.1: Aggregation example

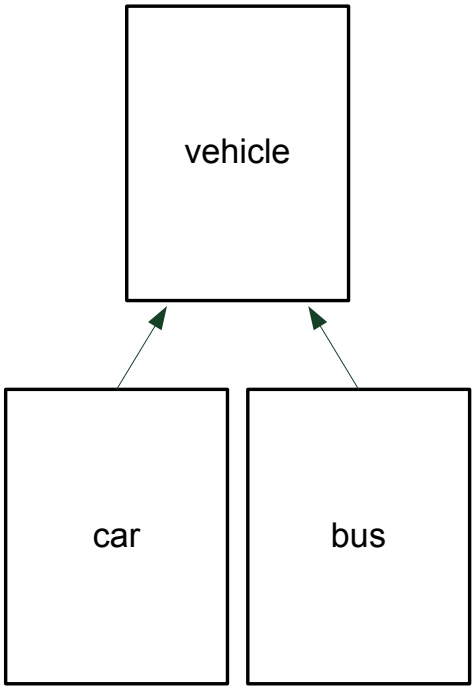


Figure 10.2: Inheritance example

Object oriented programming is an extension of programming using abstract data types that permits the expression of "type of" relationships. A **child class** can be defined that inherits behaviour of a more general **parent class**. Within this context, behaviour is inherited via the data members and methods of the parent class. The relationship between the parent class and one or more child classes is illustrated in Figure 10.3

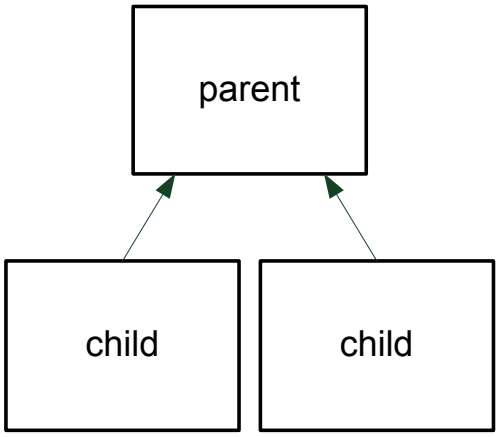


Figure 10.3: Inheritance relationship

Inheritance permits code reuse by sharing data members (attributes) and methods (functionality) with other classes. The specification of various cars in computer code may identify that all cars have an identical power steering system, fuel tank, and braking system. However, saloons have a petrol engine and rear wheel drive, but SUVs have a diesel engine and four wheel drive. Such as relationship is depicted in Figure 10.4. Further subtypes of saloon could inherit the *saloon* class, and refine its definition further.

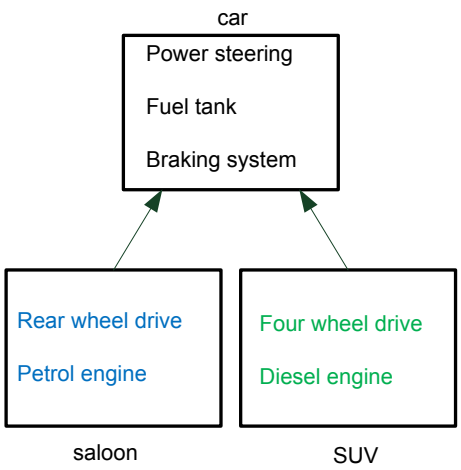


Figure 10.4: Inheritance car parts

The concept of inheritance is perhaps best explained with reference to an example. Consider the hypothetical case of a personnel database required by a university to hold the details (i.e. records) of students and academics. Academic personnel records contain: name, age, job title, salary and NI number; student personnel records contain: name, age, registration number, and average mark. The structure of these records is

illustrated in Figure 10.5.

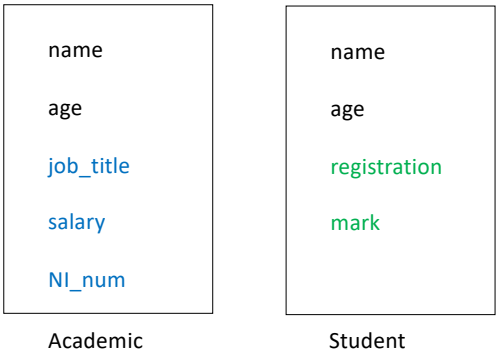


Figure 10.5: Personnel records

It is clear from the specification of both types of record that some similarity exists: *name* and *age* data members are common to both, as illustrated in Figure 10.6. This permits a parent *Person* class to be defined that contains these data members, and both the *Academic* class and the *Student* class can inherit them. This approach obviates duplication of code since these data members only need to be defined once.

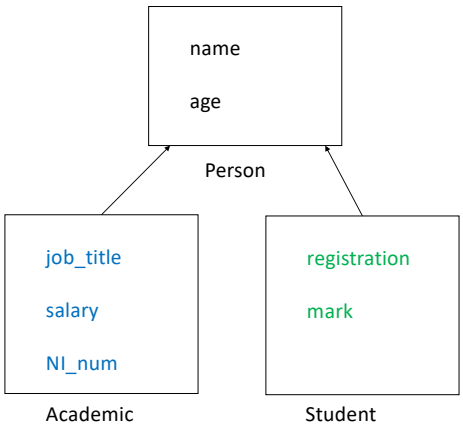


Figure 10.6: Commonality between records

The code in Listing 10.1 provides an example of how inheritance can be used to implement these personnel records. Type in and execute this code.

```
# Define base class
class Person:
```

```
    # Special initialisation method
    def __init__(self, par_name, par_age):
        self.name = par_name
        self.age = par_age

# Define a child class
class Academic(Person):
    def __init__(self, ip_name, ip_age, ip_title, ip_salary, ip_NI):
        self.job_title = ip_title
        self.salary = ip_salary
        self.NI_num = ip_NI
        # call init of parent
        Person.__init__(self, ip_name, ip_age)

    def print_out(self):
        print("Name: ", self.name)
        print ("Age: ", self.age)
        print("Job title: ", self.job_title)
        print("Salary: ", self.salary)
        print("NI number: ", self.NI_num)

# Define another child class
class Student(Person):
    def __init__(self, ipName, ipAge, ipRegistration, ipMark):
        self.registration =  ip_registration
        self.mark = ip_mark
        # call init of parent
        Person.__init__(self, ip_name, ip_age)

    def print_out(self):
        print("Name: ", self.name)
        print ("Age: ", self.age)
        print("Registration: ", self.registration)
        print("Mark: ", self.mark)


##### Main body #####

# Create instances of each child class
a = Academic("John Smith", 60, "Lecturer", 50000, "AA 12345 B")
s = Atudent("Fred Bloggs", 21, "201154321", 70.0)

# Examine contents
a.print_out()
s.print_out()
```

Listing 10.1: Database records

The code in Listing 10.1 defines an *Academic* class and a *Student* class, both inherit the attributes of the

Person class. Lines 2 — 6 define the parent class *person* which contains the two data members *name* and *age* to be inherited. Lines 9 — 15 define the child class *Academic* which contains the job_title, salary, and NI_num data members; the syntax of this derived class is illustrated with reference to Figure 10.7. The parent class name is inserted between parenthesis (line 9), following the name of the derived class. When a child class is defined, its *__init__()* method (lines 10 — 15) can optionally invoke the *__init__()* method of its parent class as shown on line 15. The *Academic* class also includes a method *print_out()*, lines 17 — 22, which prints the values of its data members to the screen. A second child class *Student* is defined on lines 25 — 36 which has different data members from that defined in the *Academic* class. Its *print_out()* method reflects this difference. Instances of each child class are created on lines 42 & 43, and the *print_out()* method of each is called on lines 46 & 47. Note that both child classes have a method with the same name, *print_out()*, which semantically does the same thing but is customised to the data members of each class; this is an example of **polymorphism**.

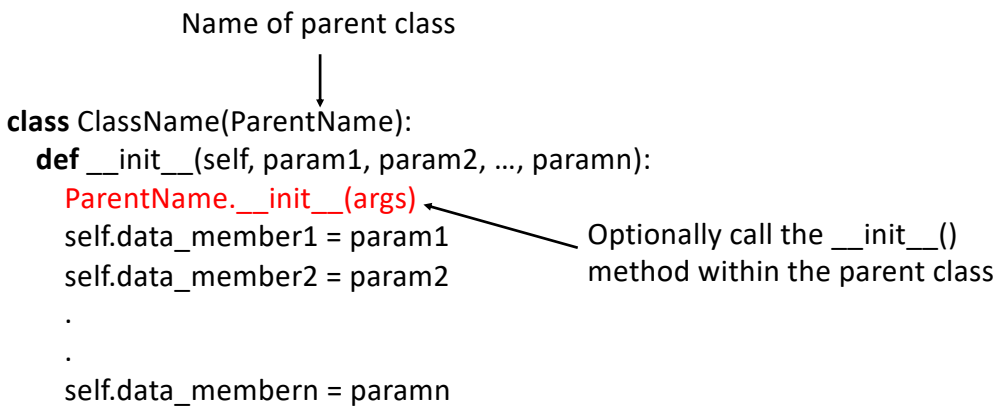


Figure 10.7: Generalised class syntax

Polymorphism permits both the *Academic* class and the *Student* class to implement the same interface, i.e. to have methods with the same name that perform similar operations that are tailored to the requirements of that class.

10.1 Design Methodology

Designing a hierarchy of classes where inheritance is used by lower level classes to further refine the characteristics of higher level classes is nontrivial and is generally undertaken by the senior programmers in many commercial software houses. Although experience will greatly improve the design of class hierarchies some basic steps can be followed:

1. Determine the methods and data members required for each class.
2. Identify which methods and data members are common across multiple classes. These should be included in a parent class.

3. Child classes that require access to these methods and data members should inherit the parent class.

4. Methods can be refined as required by child classes to facilitate polymorphism: the ability to apply a common interface to related classes.

5. Repeat from step 2.

This process will be demonstrated in Section 10.2

10.2 Design Example

Consider the hypothetical case where a software architect is required to construct a class hierarchy to represent various shapes. The specification states:

- That it must be able to handle squares, equilateral triangles, pentagons, and circles from the outset, but other shapes will be added later.

- Each class must contain a method *print_area()* which prints the area of the shape to the screen. This method must print a custom message: 'Triangle area is: 10', or 'Square area is: 25'.

- The square, equilateral triangle, and pentagon classes must include a method *print_sides()* which prints the shape's number of sides to the screen. This method may print a more generic message: 'Shape has 3 sides'.

- The circle class must include a method *print_circumference()* which prints its circumference to the screen.

- The square, equilateral triangle, and pentagon classes will take the length of their sides at instantiation.

- The circle class will take its radius at the time of its instantiation.

- The area of each shape should be calculated as soon as is practical after instantiation.

The specification can be viewed diagrammatically, as shown in Figure 10.8. Each shape can be assumed to have a custom designed *calc_area()* method since the equation to calculate the area of each shape is distinct. Similarly, the *print_area()* method will also be custom designed to allow the shape name to be printed to the screen. However, the *print_sides()* method is common to the *EquiTriangle*, *Square*, and *Pentagon* classes; furthermore, the *length* and *num_sides* data members are also common to these three shapes. Consequently, the *print_sides()* method as well as the *length* and *num_sides* data members can be placed into a parent class from which these three shapes can be derived as shown in Figure 10.9.

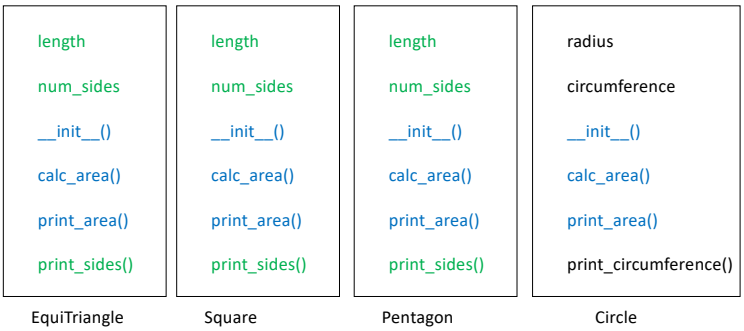


Figure 10.8: Shape class specification

Figure 10.9 represents a first draft of a class hierarchy with a new class *RegularPolygon* introduced to hold the data members and methods common to the *EqiTriangle*, *Square*, and *Pentagon* classes. The design choice to include these data members and method within a parent class avoids the need to create identical equivalents in each of the original classes; this has several benefits.

1. The overall amount of code entered by the programmer is reduced leading to a more readable solution.
2. Fewer lines of code lead to reduced memory requirements.
3. The data members and methods can now be reused by other regular polygons such as heptagons and octagons should they be required at a later date. Thus the code is more readily reusable by other programmers.

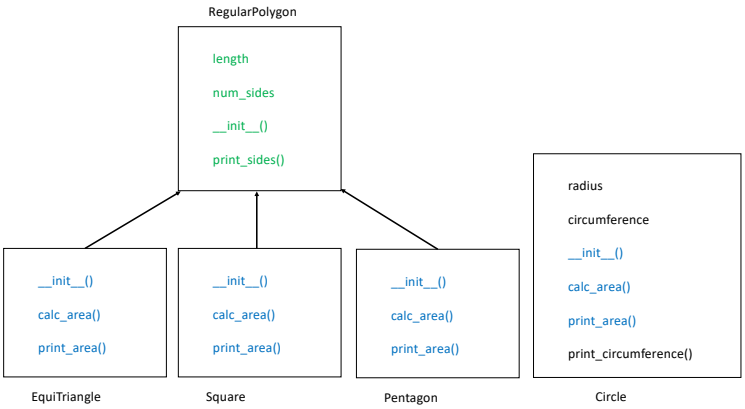


Figure 10.9: Inherited characteristics

Each class will have its own unique method to calculate its area reflecting the different formulae used in this calculation. For a side of length l, the area of an equilateral triangle is given by Equation 10.1.

$$A_t = \frac{\sqrt{3}}{4} l^2 \tag{10.1}$$

The area of a square is given by Equation 10.2.

$$A_s = l^2 \tag{10.2}$$

The area of a pentagon is given by Equation 10.3.

$$A_p = \frac{\sqrt{25 + 10\sqrt{5}}}{4} l^2 \tag{10.3}$$

Finally, for a given radius r, the area of a circle is given by Equation 10.4.

$$A_c = \pi r^2 \tag{10.4}$$

One of the design requirements is to calculate the area as soon as practical after instantiation. This requirement translates into the problem: from where should the method *calc_area()* be called? There are three candidate locations and each will be considered in turn.

1. From within the *__init__()* method of each class, i.e. *Square*, *EqiTriangle*, *Pentagon*, and *Circle*? Clearly, this would involve duplication of code and would not give rise to the advantages highlighted above.

2. From within the *__init__()* method of the *RegularPolygon* class? This approach would accrue some benefits: limited code reuse, and any future shape that is added will automatically have its area calculated as long as it is a regular polygon. However, this approach would require the *Circle* class to call *calc_area()* from within its own *__init__()* method, hence leading to fractured design.

3. From within a new class from which all future shapes must be derived? This approach makes intuitive sense: it works for all existing shapes, furthermore as long as any new shape inherits from this top level class then its area will be calculated automatically upon instantiation. This approach is depicted in Figure 10.10 where a new *Shape* class has been introduced.

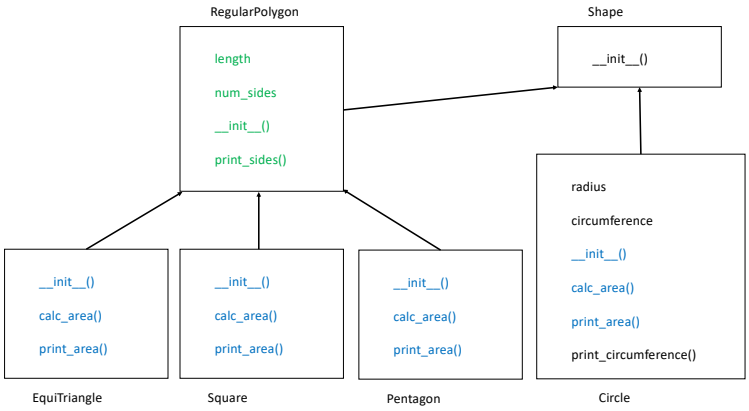


Figure 10.10: Shape class hierarchy

10.2.1 Design Example Implementation

Now that the design as been defined from a software architecture perspective, it is now possible to begin coding the various classes. The various classes can be considered in turn and indeed this highlights the divide and conquer approach to computer programming.

The *EqiTriangle* class can be implemented as shown in Listing 10.2. From its definition (line 2) it can be seen that it inherits from the *RegularPolygon* class as will be described in Listing 10.6. The *__init__()* method (lines 3 — 5) takes as a parameter the length of its sides. The method calls its parent's equivalent (line 4), then sets its *num_sides* data member to 3 (line 5). A class-specific method *calc_area()* to calculate the area of the triangle is defined on lines 7 — 9 according to Equation 10.1, and another class-specific method *print_area()* is defined on lines 11 — 13 that prints the area of that shape to the screen.

```
1  ### Define equilateral triangle class ###
2  class EqiTriangle(RegularPolygon):
3      def __init__(self, argl):
4          RegularPolygon.__init__(self, argl)
5          self.num_sides = 3
6
7      def calc_area(self):
8          area = math.sqrt(3) * (self.length ** 2) / 4
9          return area
10
11     def print_area(self):
12         print("Triangle area is: ", self.area)
13         return
```

Listing 10.2: *EqiTriangle* class

The *Square* class can be implemented as shown in Listing 10.3. It also inherits the *RegularPolygon* class (line 2). It is broadly similar to the *EqiTriangle* class in structure, but differs in its number of sides (line 5), the calculation of area method *calc_area()* (lines 7 — 9) which is based on Equation 10.2, and has its own distinct *print_area()* method (lines 11 – 13).

```
1  ### Define square class ###
2  class Square(RegularPolygon):
3      def __init__(self, argl):
4          RegularPolygon.__init__(self, argl)
5          self.num_sides = 4
6
7      def calc_area(self):
8          area = self.length ** 2
9          return area
10
11     def print_area(self):
12         print("Square area is: ", self.area)
13         return
```

Listing 10.3: *Square* class

The *Pentagon* class can be implemented as shown in Listing 10.4. Like the *EqiTriangle* and *Square* classes, it inherits from *RegularPolygon*, and indeed has a customised methods.

```
1  ### Define pentagon class
2  class Pentagon(RegularPolygon):
3      def __init__(self, argl):
4          RegularPolygon.__init__(self, argl)
5          self.num_sides = 5
6
7      def calc_area(self):
8          area = math.sqrt(25 + (10 * math.sqrt(5))) * \
9                 (self.length ** 2) / 4
10         return area
11
12     def print_area(self):
13         print("Pentagon area is: ", self.area)
14         return
```

Listing 10.4: *Pentagon* class

The *Circle* class can be implemented as shown in Listing 10.5. The *Circle* class inherits from the *Shape* class (line 2). Its *__init__()* method takes the radius as an input parameter rather than the length of a side, reflecting the essential characteristics of a circle. Within this method, a call is made to the top level (base class) equivalent within the *Shape* class (line 6). Note: this class does NOT inherit the *RegularPolygon* class.

```
### Define circle class
class Circle(Shape):
    def __init__(self, r):
        self.radius = r
        self.circumference = 2 * math.pi * r
        Shape.__init__(self)

    def calc_area(self):
        area = math.pi * (self.radius ** 2)
        return area

    def print_area(self):
        print("Circle area is: ", self.area)
        return

    def print_circumference(self):
        print("Circumference is: ", self.circumference)
        return
```

Listing 10.5: *Circle* class

The *RegularPolygon* class can be implemented as shown in Listing 10.6; it inherits from the *Shape* class (line 2). Its *__init__()* method defines and initialises data members *length* and *numSides* (lines 7 & 8) reflecting the nature of all regular polygons as recognised within the field of geometry; these data members will be inherited by all child classes. A call is made to the Shape class's *__init__()* method on line 9 to complete the initialisation. A method *print_sides()* is defined on lines 11 — 13 which prints the number of sides of any child class.

```
1  ### Define a polygon class ###
2  class RegularPolygon(Shape):
3      # Special initialisation method
4      def __init__(self, argl):
5          # All polygons defined by number and
6          # length of their sides
7          self.length = argl
8          self.num_sides = 0
9          Shape.__init__(self)
10
11     def print_sides(self):
12         print("Shape has ", self.numSides, " sides")
13         return
```

Listing 10.6: *RegularPolygon* class

Finally, the *Shape* class can be implemented as shown in Listing 10.7. This class is the base class for

all types of shape and performs the important duty of ensuring that the area of the shape is calculated at instantiation time. The *__init__()* method on line 3 is explicitly called by the *__init__()* methods of the *Circle* class (line 6 of Listing 10.5) and the *RegularPolygon* class (line 9 of Listing 10.6). This method calls the appropriate *calc_area()* method depending on the child class. A data member *area* is defined thus any class that inherits from *Shape* will also have this data member.

```
1  ### Define a generic shape class ###
2  class Shape:
3      def __init__(self):
4          self.area = self.calc_area()
```

Listing 10.7: *Shape* class

10.3 Polymorphism & Inheritance

As mentioned polymorphism is the ability to perform the same semantic behaviour within related objects. That is, related objects will have the same interface (method names) but with customised functionality for their respective class. Take for example the design discussed in Section 10.2.1, if instances of each shape (lines 4 — 7) were created and stored in a container object such as a list (line 10), as shown in Listing 10.8, then it would be possible to iterate through that list and instruct each object to print its area on the screen (lines 13 & 14). Test this for yourself by encoding the various classes in Listings 10.2 to 10.7, together with the main body of code in Listing 10.8; it is recommended that the highest level of the class hierarchy be placed at the beginning of the program. Alternatively, use the file *shapes.py* available on myplace.

```
1  ##### Main body #####
2
3  # Create instances of various shapes
4  tr = EqiTriangle(5)
5  sq = Square(7)
6  pt = Pentagon(4)
7  c = Circle(6)
8
9  # Store shapes in a list
10 shapeL_lst = [tr, sq, pt, c]
11
12 # Iterate through list to print area
13 for shape in shape_list:
14     shape.print_area()
```

Listing 10.8: Polymorphism

Listing 10.8 demonstrates the power of polymorphism: different types of object can be requested to perform the same general set of operations through a common interface (i.e. a common method call).

What would happen if the main body of code were modified to call the method *print_sides()* instead of *print_area()*, as shown in Listing 10.9? Modify your code accordingly and test it.

```
##### Main body #####

# Create instances of various shapes
tr = EqiTriangle(5)
sq = Square(7)
pt = Pentagon(4)
c = Circle(6)

# Store shapes in a list
shape_list = [tr, sq, pt, c]

# Iterate through list to print num sides
for shape in shape_list:
    shape.print_sides()
```

Listing 10.9: Error

This code would generate an error as it iterates through the list: although classes *Square*, *EqiTriangle*, and *Pentagon* shapes have a method *print_sides()*, the *Circle* class does not and this results in an error. Clearly, it is not possible to invoke a method within a class that does not exist. This begs the question: how should such an eventuality be addressed to prevent error?

One approach is architectural and can be applied across various programming languages: include a method of that name that notifies the user that some error has occurred (or silently ignores the error). In the case of this specific example this translates into the requirement to give the *Circle* class access to a *print_sides()* method which prints some kind of error message to the screen. This method could be included within the *Circle* class directly; however, whenever a new shape is added that is not a polygon then it too would require an identical method to be included directly in its design. To facilitate code re-use a better approach would be to include the method in the *Shape* class as shown in Listing 10.10. Any future shape that is added will then be able to access this method. Modify your code for this class accordingly.

```
### Define a generic shape class ###
class Shape:
    def __init__(self):
        self.area = self.calc_area()

    # Default method for non polygons
    def print_sides(self):
        print("Shape does not have sides")
        return
```

Listing 10.10: Modified *Shape* class

This approach highlights the power of inheritance. When an object's method is invoked, it will execute the version of that method defined within the object's class. If however, that class does not contain a definition for that method, then the version defined in the parent class will be executed. If no method is defined within the parent, then the version within the parent's parent is executed and so on. If the method is not defined anywhere in the class hierarchy then an error will be generated. This is illustrated in Figure 10.11; when an attempt is made to execute method *print_sides()* in an object of class *Circle*, the object checks to determine if the class *Circle* contains a definition for *print_sides()*, if not then it checks the parent class, *Shape*. Within the *Shape* class a version of *print_sides()* is defined and that version is executed. When an attempt is made to execute *print_sides()* in an object of class *Square*, the object checks to determine if the class *Square* contains a definition for *print_sides()*, if not then it checks the parent class, *RegularPolygon*. Within the *RegularPolygon* class a version of *print_sides()* is defined and that version is executed. Note, there is no need for class *RegularPolygon* to check its parent class *Shape* since it has redefined the method of its parent. Thus when a call is made to an object's method, the lowest method in the hierarchy (i.e. the version in the most specialised class) is invoked.

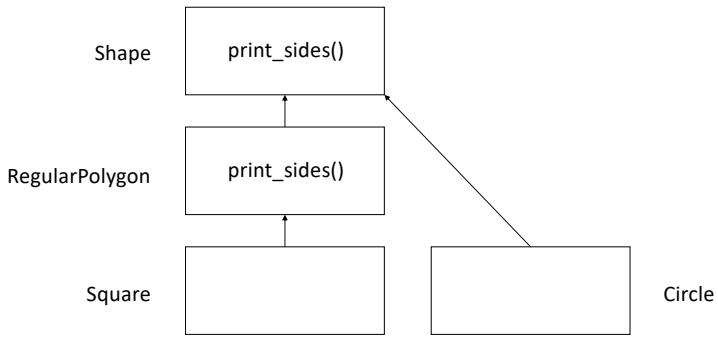


Figure 10.11: Inheritance call chain

Another approach which is specific to Python is based on run-time type checking, whereby an object's class is determined prior to attempting to call a method within that class. In Python this is achieved via the special built-in function *isinstance(object, class)*, which takes two arguments; the first is the object being interrogated, and the second is the class of interest. This function will return the boolean value *True* if the object is of that class or if it is derived from that class. To test this, remove the *print_sides()* method from the *Shape* class and modify the main body of code in accordance with Listing 10.11.

```
##### Main body #####

# Create instances of various shapes
tr = EqiTriangle(5)
sq = Square(7)
pt = Pentagon(4)
c = Circle(6)

# Store shapes in a list
shape_list = [tr, sq, pt, c]
```

```

# Iterate through list to print num sides
for shape in shape_list:
    if isinstance(shape, RegularPolygon):
        item.print_sides()
```

Listing 10.11: Run-time type checking

10.4 Summary

Based on the lessons presented in this section the following key points should be remembered.

1. Aggregation is useful for **has a** relationships.
2. Inheritance is useful for **is a** relationships.
3. Polymorphism is used to perform the same semantic behaviour but customised to a particular class.
4. Object Orientated Programming is a design philosophy that promotes code reuse by permitting related objects to have a common interface.

10.5 Problem Sets

1. Based on the pre-exisiting code for the *Shape* classes. Add a new shape class called *Hexagon* that implements a six sided shape. Carefully consider the appropriate class to use as a parent to maximise code reuse.
2. Based on the pre-exisiting code for the *Shape* classes. Add a new shape class called *Rectangle* that implements a common rectangle. Carefully consider the appropriate class to use as a parent to maximise code reuse.
3. Based on the pre-exisiting code for the *Shape* classes. Add a new shape class called *SemiCircle*, that implements half a circle. Carefully consider the appropriate class to use as a parent to maximise code reuse.